

TOWARDS LARGE LANGUAGE MODELS FOR REAL-WORLD
INFORMATION-SEEKING AT SCALE

by

Fangyuan Xu

A DISSERTATION SUBMITTED IN PARTIAL FULFILLMENT
OF THE REQUIREMENTS FOR THE DEGREE OF
DOCTOR OF PHILOSOPHY
DEPARTMENT OF COMPUTER SCIENCE
NEW YORK UNIVERSITY
AUGUST, 2026

Dr. Eunsol Choi

© FANGYUAN XU

ALL RIGHTS RESERVED, 2026

ACKNOWLEDGMENTS

[TODO]

ABSTRACT

Large language models (LLMs) are increasingly deployed to support real-world information-seeking tasks at scale. Building systems for this setting raises two challenges: making inference efficient to serve user requests at scale, and developing evaluation methods that capture the open-ended and interactive nature of real-world use. This thesis addresses both. It first presents two methods for improving inference-time efficiency. RECOMP augments retrieval-augmented language models with a lightweight compressor that produces shorter representations of retrieved documents reducing the amount of context passed to the model at inference time. RefreshKV improves efficiency during decoding by alternating between attending to the full Key-Value (KV) cache and attending to a small dynamically selected subset, periodically refreshing the subset based on the model’s own attention patterns. Next, it turns into evaluation, studying settings where real-world information needs diverge from prior evaluation methods. It first examines long-form question answering, where the space of valid answers is large. Through a detailed analysis of automatic and human evaluation protocols, it identifies flaws in prior practices and argues for multi-aspect evaluation as a more reliable alternative. It then introduces KIWI, a dataset of interaction traces between knowledge workers and LLMs on multi-turn, knowledge-intensive writing tasks. Analysis of these traces reveals error patterns specific to multi-turn settings, including inconsistencies across turns and failures to follow precise instructions. Together, these contributions advance both the methods for building efficient LLM-based information-seeking systems and the evaluation practices needed to measure their progress.

Contents

Acknowledgments	iii
Abstract	iv
List of Figures	xi
List of Tables	xv
List of Appendices	xxiii
1 Introduction	1
1.1 Motivation	1
1.2 Thesis Outline	3
2 Background	8
2.1 Long-context language models	8
2.1.1 Retrieval augmented language models	9
2.1.2 Efficient inference methods	9
2.2 Evaluation of language models	11
2.2.1 Prior evaluation: automatic metric	11
2.2.2 Prior evaluation: human evaluation	12
2.2.3 Prior evaluation: single turn evaluation	13

3	RECOMP: Improving Retrieval-Augmented LMs with Compression and Selective Augmentation	14
3.1	Introduction	14
3.2	Problem Formulation: RECOMP	17
3.3	Learning the compressors	18
3.3.1	Extractive Compression	18
3.3.2	Abstractive Compression	20
3.3.2.1	Creating Training Dataset For Distillation	21
3.4	Experimental Settings	23
3.4.1	Language Modeling	23
3.4.2	Open-domain QA	23
3.4.3	Baselines and Oracles	24
3.5	Results	25
3.6	Analysis and Discussions	28
3.7	Related Work	31
3.8	Conclusion	32
4	RefreshKV: Updating Small KV Cache During Long-Form Generation	33
4.1	Introduction	33
4.2	RefreshKV for Long-Form Generation with Long-Context LLMs	36
4.2.1	Background and Setting	36
4.2.2	Methodology and Implementation	37
4.3	Experiment Setup	40
4.4	Results	42
4.4.1	Language Modeling	42
4.4.2	Downstream Tasks	44

4.5	Ablation Studies	45
4.5.1	Adaptive stride vs. Fixed stride	45
4.5.2	Impact of full attention steps	46
4.6	Continued Pre-training with RefreshKV	47
4.7	Conclusion	48
5	A Critical Evaluation of Evaluations for Long-form Question Answering	50
5.1	Introduction	51
5.2	Background and related work	53
5.3	How do domain experts evaluate long-form answers?	54
5.3.1	Collecting expert judgments	55
5.3.2	Quantitative results	57
5.3.3	What makes one answer better than another?	57
5.3.3.1	Do models understand justifications of human preferences?	60
5.4	Do automatic metrics correlate with human judgments?	61
5.4.1	Text generation metrics	61
5.4.1.1	General-purpose generation metrics	62
5.4.1.2	Trained LFQA metrics	63
5.4.2	Evaluating automatic metrics	64
5.4.3	Results	66
5.5	Conclusion & Future Work	68
6	KIWI: A Dataset of Knowledge-Intensive Writing Instructions for Answering Research Questions	69
6.1	Introduction	70
6.2	Interaction Design	72
6.2.1	Design goals	72

6.2.2	Interaction process	72
6.3	Dataset Collection	75
6.3.1	(Question, Document set) creation	75
6.3.2	Interaction data collection	76
6.4	Analysis of KIWI	78
6.4.1	Dataset overview	78
6.4.2	Fine-grained instruction analysis	81
6.4.3	Error analysis	82
6.5	Experiments: Automatically evaluating instruction following	83
6.6	Related Work	85
6.7	Conclusion	86
7	Conclusion and Future Work	88
7.1	Summary of The Thesis	88
7.2	Future Directions	90
7.2.1	From Input Compression to System-Level Efficiency	90
7.2.2	Beyond Point Evaluation	91
7.2.3	From Diagnosing Failure to Adapting Models	92
8	Appendices	94
8.0.1	Compressor training data generation	94
8.0.2	Compressor Training Details	96
8.0.3	Inference speed	97
8.0.4	Compressing documents retrieved by different retrieval models	97
8.0.5	More analysis on abstractive compressor for the QA tasks	98
8.0.5.1	Is the teacher model a good summarizer?	98
8.0.5.2	Distillation performance analysis	99

8.0.5.3	Transferring learned compressors to another model for QA tasks	101
8.0.6	Implementation details	110
8.0.7	Memory and time requirement comparison	111
8.0.8	Attention score aggregation for models with GQA	112
8.0.9	Tuning s for query similarity schedule	112
8.0.10	Effective stride	114
8.0.11	Chain-of-key task set-up	114
8.0.12	Results on LongProc tasks with short inputs	115
8.0.13	Detailed RULER results	117
8.0.14	Expert Annotation	120
8.0.14.1	Justification Analysis	121
8.0.14.2	Example of comments mentioning different aspects for Section 5.3.3	122
8.0.15	Automatic Metric Implementation Details	122
8.0.15.1	Learned Metrics	128
8.0.16	Previously Collected Human Evaluation Data	128
8.0.16.1	LFQA systems	129
8.0.16.2	Evaluation aspects	129
8.0.16.3	GPT-3 Two-shot	130
8.0.17	Implementation details for the interaction system	132
8.0.18	Examples	135
8.0.19	Implementation details for automatic instruction analysis with GPT-4	136
8.0.20	Implementation details for error analysis	136
8.0.21	Implementation details for automatic evaluation of instruction following	137
8.0.21.1	GPT-4	137
8.0.21.2	T5	141

8.0.21.3	Additional experiments on three-way rating prediction	141
8.0.22	Analysis on question generation with GPT-4	142
Bibliography		151

List of Figures

3.1	An illustration of RECOMP , which compresses retrieved documents into a textual summary before prepending it as input to a language model at inference time. The compressed summary guides the LM to generate the correct answer, while reducing the computation costs required to encode the documents. We describe training procedures for the compressors in Section 3.3.	16
3.2	Histogram of abstractive summary length (# tokens) distribution.	29
4.1	Left: Illustration of RefreshKV (with $L = 5$, $K = 3$ and a stride $S = 3$) compared to baseline (SnapKV and Full KV) when generating four tokens. The figure shows the computation complexity of attention operation, and the size of the KV cache used at each decoding step for each method. Our approach alternates between inferencing with the partial cache($t=1,2,4$) and the full cache($t=3$). Compared to eviction-based method (e.g. SnapKV) which completely discard the evicted tokens, RefreshKV updates the partial cache based on attention scores over the entire context during the full attention steps. Right: An example of the chain-of-key task and performance of RefreshKV and the baselines. RefreshKV maintains performances across different length while eviction-based baselines' performance degrades when generating a chain with more than one key.	34

4.2	Pseudocode for RefreshKV . The model prefills the prompt with full attention and initialize the partial cache C_p cache with attention scores of the last token. For each partial attention step, we decode with the partial cache and append the KV pairs of the input token to the partial cache. We evict the token with the lowest attention score to maintain a fixed-sized partial cache. For the full attention step, we first update the full KV cache with the new tokens decoded with the partial cache, then decode with the full cache and refresh the partial cache.	38
4.3	We plot the perplexity ratio against the vanilla baseline for RefreshKV (with stride of 10) and SnapKV based on the tokens generated (x axis). While the ratio is similar at the beginning of the sequence, as the generation goes SnapKV’s perplexity diverges from vanilla approach while that of RefreshKV is relatively stable.	43
5.1	Answer length distribution in the comparison of model-generated and human-written answers (H/M) in our expert-annotated dataset. History is the hardest domain for models and also has the largest discrepancy between model and human answer length. There are 75 questions and 75 human-written and model-generated answers.	58
5.2	We manually analyzed 50 justifications each from both experts and WEBGPT crowd annotators. We report nine frequently-mentioned fine-grained aspects here. The plot shows that experts and crowdworkers disagree on which aspects are more decisive, and that experts are more sensitive to factuality and completeness. . . .	58
5.3	Pairwise automatic metric correlation.	67

6.1	An example interaction session between a user and our system. Given a question q and a set of documents $\mathbf{D}$, the model first generates an initial answer a_0 . At each turn t , the user issues an instruction i_t , for which an updated answer a_t is generated. The user provides a rating r_t and a comment c_t for the generated answer, and optionally edits the answer (such as a'_3 in turn 3) such that the edited answer completely follows the instruction.	70
6.2	Top: distribution of annotator ratings. The left columns represent ratings for stylistic instructions (s), the middle columns for info-seeking instructions (i) and the right columns for overall (o) ratings. Bottom: distribution of instruction types across the session.	80
C1	We report the data distribution on NQ dev set, TriviaQA dev set and HotpotQA dev set comparing the end task performance when prepending the oracle compression method (oracle sentence or GPT-3 summaries) and when not prepending anything for the base model (Flan-UL2).	95
C2	Learning an extractive compressor for QA task. The Score here is the exact match between the decoded answer and the gold answers.	103
C3	Learning an abstractive compressor for QA task. The Score here is the exact match between the decoded answer and the gold answers.	103
C4	Histogram of abstractive summary length (# tokens) distribution for testing data of NQ,TQA, HotpotQA and Wikitext.	104
D1	Effective stride across layer for Llama-3.1-8B (similarity threshold=0.85) and Qwen2-7B (similarity trheshold=0.95) in three datasets. We sample 10 examples from each dataset to esimate the effective stride.	114
A1	Screenshot of annotation interface for collecting expert evaluation.	120

B1	We present the workflow for collecting KIWI . Detailed description for each step is in §6.2.	145
B2	A screenshot of our annotation interface.	150

List of Tables

3.1	Results on language modeling task. We report results on GPT-2, GPT2-XL and GPT-J with compressors trained with GPT-2.	25
3.2	Open-domain QA results with Flan-UL2 (20B) as the LM M . We report number of tokens provided as in-context evidence document, excluding the in-context examples. We train separate compressors (one extractive, one abstractive) for each dataset. Extractive compressor selects one sentence for NQ/TQA, and two sentences for HotpotQA.	27
3.3	Analysis on usage of in-context evidence for NQ. For the last column, we report frequency of model copying from the evidence on a subset where gold answer is present / when it is not.	30
3.4	Manual analysis on abstractive summaries generated for NQ, TQA and HotpotQA (HQA) dataset.	31
4.1	Perplexity results and latency on language modeling task for LLama-3.1-8B and QWEN-2-7B. We report results on Arxiv and Book corpora with input context length of 16K tokens. We set $K = 2048$	42
4.2	Downstream task performance. In each cell, the first number represents the performance of Llama-3.1 model and the second number for QWEN-2 model. *We report performance of Llama-3.1-70B and Qwen-2-72B for the chain-of-key task, as the smaller variants cannot perform the task even in vanilla setting.	43

4.3	Results comparing fixed stride and dynamic stride based on query similarity. In all tasks, dynamic stride shows better task performance while performing full attention step fewer times.	45
4.4	Ablation study on LLama-3.1-8B. We report perplexity for Arxiv and F-1 score for the HTML to TSV task.	45
4.5	Results on continued pre-training with RefreshKV for LLaMA-3.1. The context size is 8k and we report perplexity on the last 50 tokens. We report the performance for each setting before (the number on the left) and after (the number on the right) CPT.	48
5.1	Examples of two fine-grained aspects, factuality (top) and completeness (bottom), that were decisive factors in our expert annotators' preference of one answer over another. The <i>human</i> answers are from the r/explainlikeimfive subreddit and the <i>model</i> answers are generated zero-shot by text-davinci-002. See Table A5 for more examples.	52
5.2	Results of our expert annotation of seven domains, where the two candidate answers are either both human-written (H/H) or human-written vs. model-generated (H/M). We report how often the highly-upvoted answer was preferred in H/H, and how often the model-generated answers are preferred in H/M.	54
5.3	Results on masked justification reference prediction: Original comments, Flipped comments and Random comments. The larger LMs can identify references in justifications better.	60

5.4	Accuracy of automatic metrics for imitating human judgments of overall answer preference, coherence, and factuality. h/m denotes comparisons between human-written answers and model-generated answers, while m/m denotes comparisons between pairs of model-generated answers. [†] These metrics are calculated on 109 pairs of comparisons, where comparisons of History are removed because there are only one answer available on the subreddit and hence no reference answer to compare. [◇] We estimate the human performance with a pairwise agreement for two-way and three-way expert annotations. [★] This pairwise agreement is reported by WEBGPT [Nakano et al. 2021], estimated on a subset of the data.	65
6.1	An example ($q, \mathbf{D}$) annotation (§6.3.1). Given a source paper, the annotator first selects the related work paragraph(s) that imply the question, then writes the question and extracts evidence papers cited.	77
6.2	Data statistics of the KIWI . We collect a total of 234 sessions (78 sessions per model). We report the total number of turns, number of turns with edited answers, number of turns per session, and the distribution of instruction types for each model. . . .	79
6.3	Definitions and example instructions for the fine-grained instruction types (§6.4.2). We report the distribution classified by GPT-4 and % good rating (color-coded based on the % good) across the three models.	79
6.4	Fine-grained error categories alongside example instruction and corresponding annotator comment pairs. We report the distribution classified by GPT-4 on interaction turns that are rated as neutral or bad.	83
6.5	Test set results on predicting ratings for instruction following edits (§6.5). We report the % of good and bad ratings in the model predictions in the last column.	85

C1	Training data statistics for abstractive and extractive compressors. “% filtered” refers to the percentage of examples not included. For extractive compressor, we filter examples for which prepending sentences with top contriever score leads to the correct answer. For abstractive compressor, we filter examples for which prepending GPT-3.5-turbo generated summary and no retrieval both produce incorrect answers. $ N $ refers to the average number of negative examples for constrastive training.	96
C2	Inference speed of FLAN-UL2 on NQ dev set.	97
C3	Compressing retrieved documents for contriever and DPR on NQ.	98
C4	Comparing prepending top 5 documents and prepending GPT-3.5-turbo compressed documents.	98
C5	Manual analysis of when compressing with GPT-3.5-turbo leads to different performance compared to prepending top 5 documents. We report % examples belonging to each reason.	99
C6	Analysis on distillation performance, grouped by whether the generated summary is an empty summary. The left two column denotes whether the summaries generated by T5 or oracle GPT-3.5-turbo is empty (E) or not (N). We report the EM for the T5 system and the oracle system (EM(O)) and the percentage of data belong to this slice.	100
C7	Open-domain QA results on LLaMA-13B. We report the results of oracle compressions with Flan-UL2, the base model for the compressors, (<i>Oracle w/ FLAN</i>) and the oracle compression results for LLaMA-13B.	102
C8	Example abstractive and extractive compression on wikitext-103 dev set and NQ.	105
C9	Example input to the Flan-UL2 for NQ with in-context examples and retrieved documents.	106

C10	Prompts used to generated summaries from GPT-3.5-turbo. query and docs represent the actual input query and retrieved documents.	106
C11	Case study of how compressing the retrieved documents helps the model to identify the right answer from NQ dev set.	107
C12	Example summaries and their manual analysis labels. See Table C13 for more example.	108
C13	Example summaries and their manual analysis labels (continued).	109
D1	Dataset statistics. We report the number of tokens for both the input context and output generation for each dataset, as well as total number of examples.	111
D2	Comparing memory (KV cache size for L input tokens) and time (attention computation for generating the next T tokens) of RefreshKV and baselines. We denote S as stride and use the same KV cache size (K) for the partial cache for our method and complete cache for eviction-based baselines.	111
D3	Results comparing different methods to aggregate attention scores for GQA models. We experiment with taking the attention score of the first query head, the average and max attention scores of the query heads in the same group to select topK KV cache. For StreamingLLM and RefreshKV, we set $K = 1/8L$ and stride as 10.	112
D4	Results of different similarity threshold s on the held-out set of the Book dataset across two QC stride.	113
D5	Breakdown of HTML tasks based on output length.	115
D6	Example input for the chain-of-key task where $W = 2$ and $T = 10$	116
D7	Example output for the chain-of-key task where $W = 2$ and $T = 10$ and their score. Keys that are not in the context are highlighted in red.	116
D8	Performance on long-context tasks with short outputs from LongProc benchmark for LLaMA-3.1-8B-Instruct and Qwen-2-7B-Instruct.	117

D9	Detailed performance of RULER subtasks with $L = 32K$. For non-vanilla methods, we set the $K = 1/8L$	119
A1	Statistics of the text length in our expert-annotated dataset. For each category, there are 20 questions. Each question has either a pair of human-written answers (H/H) or a pair of human-written and model-generated answers (H/M). The domain of history has 15 questions in the H/M setting and 5 in H/H. The other six domains have 10 questions in each setting. There are 140 questions, 205 human-written answers, and 75 model-generated answers.	121
A2	Data statistics for computational analysis of free-form justifications. The span refers to the masked reference of candidate answer in the justifications.	122
A3	Free-form justifications written by experts and their corresponding aspects. . . .	123
A4	Example annotations by domain experts comparing long-form answers, either generated from GPT3 or human written, showing their preferences, free-form justifications and aspects. The first two examples illustrate that experts disagree with each other because they value different aspects.	124
A5	Examples of some of the aspects that occur in the manual analysis described in Section 5.3.3.	125
A6	Performance of 2 shot question answer evaluation using GPT3 text-davinci-003. Consistency reports the percentage of the model generate the same preferred answer across three API calls. Accuracy compares the majority votes among the three API calls against the human preference.	127

A7	Data statistics for human preference data used to train and evaluate the learned metric (Section 5.4.1.2). We collapse human rating such that the answer preferred is assigned score 1 and the other 0. Nakano et al. [] included tie data and assign them 50% soft labels, and excluded them from evaluation. However, we didn't find them beneficial for model training and hence removed them from both training and valuation.	127
A8	Data statistics of answers compared in the human evaluation data. The number of comparison data can be found in Table 5.4. $ q $, $ a $, $ d $ and $ j $ represent the average number of words for question, answer paragraph, retrieved documents and justification. For WebGPT, justifications are only on a subset of comparison data. WebGPT and expert annotation data take both the title and the description of the reddit post as question following [], whereas Hurdles data only considers the title as question (hence shorter $ q $).	127
A9	Data Statistics for human comparison evaluation data for each aspect. In all studies, overall score was mandatory but coherence / factuality scores were optional and hence the number of evaluation data available varies among different aspects. All human evaluation data is one-way annotated.	128
B1	We report statistics for each individual turns in our dataset: number of words in the instruction, number of words in the previous answer (src); number of words in the answer (target); the average length fraction between the source and the target; the token-level edit distance; and the fraction between the edit distance and the source text. We filter out turns where the response is rated as neutral or bad and there isn't an edited answer.	137
B2	Test set results on automatic three-way evaluation for instruction following edits.  stands for gpt-4 . We report the % of good , neutral and bad ratings in the model predictions in the last column.	142

B3	Example questions annotated (§6.3.1).	144
B4	Example turns rated as good, neutral. The bad example is in Table B5.	146
B5	Example turns rated as bad by the annotator.	147
B6	Prompt for automatic instruction analysis.	148
B7	Prompt for automatic comment analysis.	149

List of Appendices

Appendix C: Supplementary Material for Chapter 3	94
Appendix D: Supplementary Material for Chapter 4	110
Appendix A: Supplementary Material for Chapter 5	120
Appendix B: Supplementary Material for Chapter 6	132

1 | INTRODUCTION

1.1 MOTIVATION

Large language models (LLMs) have rapidly evolved from research prototypes into systems deployed at scale, accessed by millions of users [Anthropic 2026; Google 2025; OpenAI 2026] across diverse use cases [Appel et al. 2025]. Over the past few years, language models have progressed from specialized systems designed for relatively simple tasks [Han et al. 2021; Kwiatkowski et al. 2019; Nallapati et al. 2016], to chat-bots capable of responding to a broad range of user queries [Nakano et al. 2021; Ouyang et al. 2022], to agents that can accomplish complex, long-horizon tasks [Asai et al. 2024a; Jimenez et al. 2024; Xie et al. 2024]. This trajectory raises two key challenges for deploying language models at scale across real-world use cases. The first concerns how **efficiently** models can be deployed: increasingly capable models require increasingly expensive inference, especially as context lengths grow. The second concerns what it means to **evaluate** progress at all, as deployment moves into open-ended and interactive settings, where reference-based metrics and static benchmarks no longer apply. This thesis addresses these two challenges in the setting of information-seeking tasks.

We first turn to **inference-time efficiency**. Much of the progress in language models has been driven by scaling along multiple axes, including model size, training data [Hoffmann et al. 2022; Kaplan et al. 2020] and more recently, context length [Anthropic 2026; Google 2025; OpenAI 2026]. While scaling has delivered substantial capability gains, it has also made inference

progressively more expensive. These costs are increasingly incurred at test time with long input contexts, reasoning chains, and agentic loops. In this thesis, we focus on one particular lever for improving inference-time efficiency: reducing the effective input sequence length.

Our first observation is that given a long input, only a subset of the input is typically useful for the request at hand. This is particularly true for one prevalent use of long-context, retrieval augmentation, which typically contains noisy and irrelevant documents[Liu et al. 2023a; Mallen et al. 2022]. Building on this observation, we propose RECOMP in Chapter 3, a method that augments the language model with an external, light-weight compressor module. The compressor rewrites lengthy input documents into concise textual summaries, which are then passed to the LM,

Conceptually, RECOMP is related to a line of work on compressing the Key-Value (KV) cache of language models [Li et al. 2024; Xiao et al. 2023; Zhang et al. 2024b], as both approaches rely on the assumption that only a subset of input tokens is useful for decoding. However, this assumption does not hold for all decoding scenarios. For example, the task of summarizing a book [Zhang et al. 2024a] may require the entire book as input. We observe, however, that although the entire input may be useful over the course of generation, not every token is useful at *every* decoding step. Building on this observation, we propose RefreshKV in Chapter 4, a method that alternates between attending over the entire input and attending over a subset of tokens.

The methods above improve efficiency on tasks where progress is measurable with well-established benchmarks [Bai et al. 2023; Hsieh et al. 2024; Zhang et al. 2024a]. But as language models are deployed in increasingly open-ended and interactive settings, the assumptions underlying our **evaluation methods** themselves begin to break down. Traditional automatic metrics, such as accuracy against a reference answer, rely on assumptions about both the reference set and the comparison procedure: namely, that an enumerable set of valid answers exists and is accessible, and that model outputs can be reliably scored against this set. Tasks that satisfy these assumptions are limited, such as short-form question answering [Joshi et al. 2017b; Rajpurkar et al.

2016] or summarization [Nallapati et al. 2016]. As a result, reference-based metrics fail to capture performance on the vast majority of tasks for which language models are now used. In Chapter 5, we examine one representative task: generating long-form answers to open-ended questions [Fan et al. 2019a]. Through a detailed analysis of both automatic metrics and human evaluation, we find that prior metrics do not meaningfully capture human preferences, although they can still measure specific aspects of answer quality. Our findings further show that the open-ended nature of the task introduces an additional challenge: even human annotators may disagree about which answer is better due to differences in personal preferences.

A second evaluation challenge arises from the mismatch between real-world usage and academic benchmarks [Ouyang et al. 2023; Zheng et al. 2023a] in its interaction structure. While most benchmarks consist primarily of single-turn requests [Hendrycks et al. 2021; Liang et al. 2023], real-world interactions often unfold over multiple turns, requiring models to interpret and respond to follow-up requests. In Chapter 6, we study one prevalent real-world use case: language models as writing assistants. To study this setting, we design an interactive interface and collect interaction traces between expert annotators and language models. Our analysis reveals error patterns that arise specifically in multi-turn interactions, including inconsistencies across turns and failures to follow precise instructions.

Together, these works address two central challenges in deploying language models for real-world use at scale. The first half develops inference-time efficiency methods for long-context models, first by compressing lengthy inputs before generation, then by reducing attention cost during decoding. The second half revisits evaluation for settings that are open-ended and interactive, where reference-based metrics and static benchmarks fall short.

1.2 THESIS OUTLINE

The following chapters are structured to address each of these directions.

CHAPTER 2: BACKGROUND. We begin with a brief overview of the two directions addressed in this thesis. First, we review the development of long-context language models and the computational challenges they introduce. We then survey prior and concurrent efforts to improve language model efficiency. Second, we turn to evaluation, reviewing existing methods for assessing language model performance, including automatic metrics and human evaluation protocols.

CHAPTER 3: CONTEXT COMPRESSION FOR RETRIEVAL-AUGMENTED LANGUAGE MODELS. Retrieval augmentation is one prevalent use case of long-context language models and has been shown to improve performance on knowledge-intensive tasks [Lewis et al. 2020b; Shi et al. 2023b; Wang et al. 2023b]. However, prepending retrieved documents to the input increases the number of tokens that the model must process, thereby increasing inference-time computation. In Chapter 3, we introduce RECOMP, a method for improving the efficiency of retrieval-augmented language models (RALMs) while preserving downstream performance. RECOMP augments a RALM with an external compressor module that preprocesses lengthy retrieved documents before passing them to the language model. We instantiate this module with two types of compressors and train them using signals derived from the language model’s downstream performance.

CHAPTER 4: SPARSE ATTENTION FOR EFFICIENT LONG-CONTEXT GENERATION. While RECOMP improves the efficiency of RALMs on tasks such as question answering, it relies on the assumption that only part of the input is relevant to the query at hand. This assumption does not hold for requests that require access to the *entire* input, such as summarizing a book [Bai et al. 2023; Zhang et al. 2024a]. In Chapter 4, we introduce RefreshKV, a sparse attention mechanism for efficient long-context generation. RefreshKV preserves the full context in the Key-Value (KV) cache, but attends to only a subset of tokens at each inference step. It dynamically constructs this subset during generation, reducing inference-time computation while improving performance over prior methods.

CHAPTER 5: EVALUATING THE EVALUATION OF LONG-FORM ANSWERS. As language models become increasingly capable of handling complex questions, evaluation techniques designed for simpler tasks no longer provide meaningful signals [Krishna et al. 2021]. Chapter 5 studies evaluation practices for long-form question answering, a representative open-ended task in which models generate paragraph-level answers to user questions. The chapter first surveys a range of previously proposed automatic metrics and measures their correlations with human preferences. It finds that *none* of the individual automatic metrics correlates well with overall human preference, although some metrics better capture specific aspects of answer quality. The chapter then revisits the standard protocol for human evaluation itself, showing that divergence among annotator preferences does not necessarily indicate poor annotation quality, but can instead reflect genuine differences in what annotators value. Building on these findings, we advocate for targeted, multi-faceted evaluation practices for open-ended question answering.

CHAPTER 6: EVALUATING MULTI-TURN INSTRUCTION FOLLOWING. Language models are increasingly used as assistants that collaborate with users to accomplish specific tasks [Ouyang et al. 2023; Zheng et al. 2023a]. In these settings, models must move beyond answering one-shot questions and instead handle *multi-turn* instructions, including follow-up requests, refinements, and constraints introduced over the course of an interaction. However, it remains unclear what types of instructions arise in such settings and how well language models handle them. In Chapter 6, we design a multi-turn interaction task and collect KIWI, a dataset of interaction traces between users and language models. Our analysis both quantitatively measures model success rates and identifies error patterns specific to multi-turn interaction, including inconsistencies across turns and failures to follow precise constraints.

CHAPTER 7: FUTURE DIRECTIONS. We conclude the dissertation by summarizing key findings and outlining future directions for developing effective and efficient language models.

PRIOR PUBLICATION

The research presented in this dissertation is heavily based on the following jointly authored prior publications.

1. **Fangyuan Xu**, Weijia Shi, Eunsol Choi. “RECOMP: Improving Retrieval-Augmented LMs with Context Compression and Selective Augmentation.” In: *The Twelfth International Conference on Learning Representations*. 2024. [\[pdf\]](#) — In [Chapter 3](#)
2. **Fangyuan Xu**, Tanya Goyal*, Eunsol Choi*. “RefreshKV: Updating Small KV Cache During Long-form Generation.” In: *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics: ACL*. 2025. [\[pdf\]](#) — In [Chapter 4](#)
3. **Fangyuan Xu***, Yixiao Song*, Mohit Iyyer, Eunsol Choi. “A Critical Evaluation of Evaluations for Long-form Question Answering.” In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics: ACL*. 2023. [\[pdf\]](#) — In [Chapter 5](#)
4. **Fangyuan Xu**, Kyle Lo, Luca Soldaini, Bailey Kuehl, Eunsol Choi, David Wadden. “KIWI: A Dataset of Knowledge-Intensive Writing Instructions for Answering Research Questions.” In: *Findings of the Association for Computational Linguistics: ACL*. 2024. [\[pdf\]](#) — In [Chapter 6](#)

The following jointly authored prior publications are also briefly mentioned in this dissertation.

5. Mina Huh, **Fangyuan Xu**, Yi-Hao Peng, Chongyan Chen, Hansika Murugu, Danna Gurari, Eunsol Choi, Amy Pavel. “Long-Form Answers to Visual Questions from Blind and Low Vision People.” In: *First Conference on Language Modeling*. 2024. [\[pdf\]](#) — In [Chapter 7](#)
6. **Fangyuan Xu**, Rujun Han, Yanfei Chen, Zifeng Wang, I-Hung Hsu, Jun Yan, Vishy Tirumalashetty, Eunsol Choi, Tomas Pfister, Chen-Yu Lee. “SAGE: Steerable Agentic Data Generation for Deep Search with Execution Feedback” In: *Findings of the Association for Computational Linguistics: EACL*. 2026. [\[pdf\]](#) — In [Chapter 7](#)

7. **Fangyuan Xu**, Sihao Chen, Zinan Lin, Taiwei Shi, Sydney Graham, Pei Zhou, Mengting Wan, Alex Stein, Virginia Estellers, Charles Chen, Morris Sharp, Richard Speyer, Tadas Baltrusaitis, Jennifer Neville, Eunsol Choi, Longqi Yang. “DP-RFT: Learning to Generate Synthetic Text via Differentially Private Reinforcement Fine-Tuning” Under submission. 2026. [\[pdf\]](#) — In [Chapter 7](#)
8. Yuhan Liu, **Fangyuan Xu**, Vishakh Padmakumar, Daphne Ippolito, Eunsol Choi “No Single Best Model for Diversity: Learning a Router for Sample Diversity” Under submission. 2026. [\[pdf\]](#) — In [Chapter 7](#)

2 | BACKGROUND

In this chapter, we provide a brief overview of long-context language models, efficient inference methods and evaluation methods of large language models.

2.1 LONG-CONTEXT LANGUAGE MODELS

Recent language models have substantially extended the context lengths they can process, from a few thousand tokens to millions of tokens [Anthropic 2026; Google 2025; OpenAI 2026]. This extension has been driven by both architectural advances — improvements to positional encoding [Chen et al. 2023b; Su et al. 2023], sparse and linear attention variants [DeepSeek-AI 2025; Gu et al. 2022; Yang et al. 2025]. This supports a growing set of applications, including retrieval-augmented generation [Jin et al. 2025], long-document understanding [Bai et al. 2023], and agentic workflows [Liu et al. 2025]. Despite these advances, long-context inference remains expensive: both prefill cost (processing the input) and decode cost (attending over a large Key-Value cache during generation) scale with context length, motivating a range of methods to reduce inference cost without sacrificing the capabilities that long context enables. We organize related work along two complementary lines: retrieval-augmented approaches, and efficient-inference methods that reduce the cost of processing long context directly.

2.1.1 RETRIEVAL AUGMENTED LANGUAGE MODELS

Rather than processing arbitrarily long inputs end-to-end, retrieval-augmented language models access external information through a retriever that selects relevant documents for each query [Lewis et al. 2020b; Shi et al. 2023b]. This paradigm has a long history in open-domain question answering [Karpukhin et al. 2020a; Mallen et al. 2022], where retrieval was originally used to compensate for the limited parametric knowledge of smaller models, and has since been extended to a wide range of knowledge-intensive tasks [Asai et al. 2024b; Lewis et al. 2020b]. Recent work has improved both components of the pipeline: stronger retrievers based on dense representations [Karpukhin et al. 2020a] and better training signals [Shi et al. 2023b]. Improvements of the language models have also been proposed, including training procedures that teach the model to use retrieved evidence faithfully and make better use of the retrieved content [Asai et al. 2024b; Yoran et al. 2024]. Even with long-context language models, retrieval remains valuable: it concentrates the relevant content into a shorter input, reducing both inference cost and the model’s susceptibility to distraction by irrelevant context [Liu et al. 2023a; Shi et al. 2023a]. Our work in [Chapter 3](#) builds on this paradigm, treating retrieval as the starting point and adding a compression step between retrieval and generation to further reduce the input length the language model must process.

2.1.2 EFFICIENT INFERENCE METHODS

A complementary line of work reduces the cost of long-context inference directly, without changing what the model processes. Various techniques have been proposed in this direction. FlashAttention [Dao 2024] achieves significant gains in inference speed by optimizing attention computation on GPUs, reducing memory movement through tiling and recomputation. A line of work [Hooper et al. 2024; Liu et al. 2024b; Xiao et al. 2022] proposes to quantize KV caches to reduce both memory and computation cost; while recent work [Liu et al. 2024a; Wan et al. 2025;

Wang et al. 2025b] proposes to merge KV cache of similar tokens.

KV CACHE EVICTION A growing line of work studies strategies for evicting tokens from the KV cache to reduce memory and compute during long-context decoding. Position-based methods retain only sink and recent tokens [Xiao et al. 2023]; attention-based methods retain tokens with high accumulated attention scores [Zhang et al. 2024b]. Query-aware variants use the attention scores of the last few prompt tokens to select which tokens to keep [Chen et al. 2024; Li et al. 2024], and other methods design eviction strategies tailored to attention patterns across different heads [Ge et al. 2024; Xiao et al. 2024b] or layers [Cai et al. 2024; Wan et al. 2025; Yang et al. 2024b]. These methods reduce the cache by removing tokens; in contrast, our work in Chapter 4 retains the full cache and dynamically selects which subset to attend to at each decoding step, preserving access to tokens that turn out to be needed later.

SPARSE ATTENTION A separate line reduces inference cost by computing attention over only a subset of tokens, applied at different stages of the model lifecycle. Earlier work trains language models with fixed sparse attention patterns, such as sliding windows [Beltagy et al. 2020a; Zaheer et al. 2020], to reduce computational complexity at training time. Training-free methods such as Unlimiformer [Bertsch et al. 2023] and InfLLM [Xiao et al. 2024a] extend the effective context window of pretrained models by attending only to tokens with the highest attention scores. More recent work targets inference-time efficiency for models already trained on long contexts: MInference [Jiang et al. 2024] identifies head-specific sparsity patterns to accelerate prefill, while SparQ [Ribar et al. 2024] and Quest [Tang et al. 2024] achieve decode-time speedups by attending to dynamically selected subsets of tokens, using specialized kernels to approximate attention and identify critical tokens. Our work falls in this last category but takes a different approach: rather than approximating attention with specialized kernels, we use the attention patterns of previously decoded tokens to select which tokens to attend to in subsequent decoding steps.

PROMPT COMPRESSION A complementary line of work compresses long contexts before they are processed by the language model. One direction compresses contexts into soft prompts [Chevalier et al. 2023; Mu et al. 2023; Wingate et al. 2022] that replace plain-text context with shorter learned representations, reducing inference cost. A related line, context distillation [Choi et al. 2022; Padmanabhan et al. 2023; Snell et al. 2022], injects context directly into model parameters. Both approaches require gradient access and produce representations tied to specific models. Our work in Chapter 3 instead produces textual summaries that are interpretable, transferable across language models, and applicable to black-box LMs without gradient updates. Token-level prompt compression methods [Jiang et al. 2023; Li et al. 2023b] leverage a small model to prune out redundant tokens. A related line addresses the same problem at the document level rather than the token level, either by filtering irrelevant retrieved documents [Wang et al. 2023c] or fine-tuning the language model to handle them robustly [Yoran et al. 2024].

2.2 EVALUATION OF LANGUAGE MODELS

2.2.1 PRIOR EVALUATION: AUTOMATIC METRIC

Many generation tasks, such as machine translation and summarization, are primarily evaluated with reference-based metrics that measure similarity between generated and reference text, including BLEU [Papineni et al. 2002], ROUGE [Lin 2004], and BERTScore [Zhang et al. 2019]. For open-ended generation tasks such as story generation, comparing a generated text against a single reference is not meaningful, since many distinct outputs may be equally valid. To address this, reference-based metrics have been extended to measure distributional similarity between model-generated and human-written text [Pillutla et al. 2021], comparing populations of outputs rather than individual ones. A separate line of work develops reference-less metrics that measure specific aspects of text quality. Factuality metrics for summarization [Barrantes et al. 2020;

Goyal and Durrett 2020; Kryscinski et al. 2020; Laban et al. 2022] capture the relationship between source document and summary without requiring a reference summary. Other work trains automatic metrics to emulate human judgments using either gold human preferences or synthetically generated data [Sellam et al. 2020; Zhang et al. 2022a; Zhong et al. 2022]. Our work in Chapter 5 examines how well these families of automatic metrics, reference-based, distributional, and learned — capture human judgments on long-form question answering, finding that none reliably predicts overall preferences, though some capture specific aspects of answer quality. Most recently, there are also a line of work that explore using LLM-as-a-judge to evaluate model responses [Li et al. 2023a; Zheng et al. 2023b].

2.2.2 PRIOR EVALUATION: HUMAN EVALUATION

Prior human evaluations of long-form generation typically rely on non-expert crowdworkers to judge highly domain-specific answers, either with no access to external information [Krishna et al. 2021] or with access only to model-retrieved evidence documents [Nakano et al. 2021]. Both settings are problematic: non-experts cannot reliably judge the correctness of answers in isolation, and they cannot be expected to thoroughly comprehend evidence documents or assess their validity and relevance to a candidate answer [Gao et al. 2022a]. While Nakano et al. [2021] solicit optional free-form justifications from workers to explain their preference judgments, it remains unclear how well non-expert workers can judge correctness in domains outside their expertise. Our work in Chapter 5 addresses this by using expert annotators to evaluate long-form answers, and examines what divergence among expert preferences reveals about the evaluation task itself.

2.2.3 PRIOR EVALUATION: SINGLE TURN EVALUATION

Most existing benchmarks for evaluating language models consist of single-turn interactions, in which the model receives a single query and produces a response [Hendrycks et al. 2021; Liang et al. 2023]. This format has enabled large-scale, reproducible evaluation across diverse tasks, but it diverges from how language models are actually used: real-world interactions frequently span multiple turns, with users refining their requests, asking follow-up questions, and incorporating model outputs into ongoing tasks [Zhao et al. 2023a; Zheng et al. 2023a; b]. A growing body of work attempts to evaluate language models in multi-turn settings. One direction evaluates models in agentic settings where multi-turn interaction with an environment is intrinsic to the task, such as web navigation [Zhou et al. 2024], tool use [Wang et al. 2023a], and coding [Jimenez et al. 2024]. These benchmarks evaluate the model’s ability to take multi-step actions, but the turns are typically driven by the model’s interaction with an environment rather than with a human user expressing evolving needs. A smaller line of work studies multi-turn interaction between language models and human users directly, often through interaction logs collected from deployed systems [Zhao et al. 2024; Zheng et al. 2023a; b]. These logs reveal the diversity of real-world use but typically lack the structured annotations needed to characterize specific failure modes. Our work in Chapter 6 contributes to this line by designing an interactive interface to elicit multi-turn interactions between expert annotators and language models on writing tasks, and by collecting structured annotations on the resulting traces that reveal error patterns specific to multi-turn settings, including inconsistency across turns and failures to follow precise instructions.

3 | RECOMP: IMPROVING RETRIEVAL-AUGMENTED LMS WITH COMPRESSION AND SELECTIVE AUGMENTATION

The introduction motivates two challenges in deploying language models at scale. We begin with the first: making inference more efficient as context lengths grow. This chapter focuses on retrieval-augmented generation, one of the most common settings where long contexts arise, and observes that retrieved documents typically contain substantially more text than the language model needs to answer a given query. We propose RECOMP, which trains a lightweight compressor to rewrite retrieved documents into concise summaries before they are passed to the language model. The compressor is trained against signals derived from the downstream language model’s performance. This chapter is based on work with Weijia Shi and Eunsol Choi [Xu et al. 2024b].

3.1 INTRODUCTION

Retrieval-augmented language models (RALMs) [Borgeaud et al. 2022; Izacard et al. 2022; Khandelwal et al. 2019; Lewis et al. 2020b] have shown impressive performance on knowledge-

intensive tasks [Kwiatkowski et al. 2019; Petroni et al. 2021]. Simply prepending retrieved documents to the input without updating the language models (LMs) [Ram et al. 2023; Shi et al. 2023b; Si et al. 2022] allows retrieval augmentation even for black-box LMs, but such approach comes with limitations. First, it increases computational costs as LMs now encode substantially more tokens. Second, even if we manage to adapt LMs to incorporate long context [Beltagy et al. 2020b; Zaheer et al. 2020], these models struggle to use all information in the context, frequently missing information placed in the middle [Liu et al. 2023a]. Third, prepending multiple documents in-context can further *confuse* LMs with irrelevant information, degrading model performances [Mallen et al. 2022; Shi et al. 2023a].

To overcome such limitations, we propose **RECOMP** (**R**etrieve, **C**ompress, **P**repend), an intermediate step for RALMs which compresses retrieved documents into a textual summary prior to in-context augmentation. Figure 3.1 illustrates our approach. The generated summary should be concise to maximize efficiency, be faithful to the retrieved evidence documents, and guide RALM to generate desired outputs when prepended to the input. We train compressors with a learning objective that encourages both efficiency and effectiveness for a target LM. Our framework enables selective augmentation, by generating an empty summary when the retrieved documents are irrelevant or unhelpful for target task and the target LM.

We propose two kinds of compressors: (1) *Extractive compressor* which selects relevant sentences from retrieved document set; (2) *Abstractive compressor* which generates a summary synthesizing information from multiple retrieved documents. Both compressors implement multi-document query-focused summarization [Xu and Lapata 2020], where we summarize retrieved evidence document set with respect to the input query. As we aim to enable RALM to generate correct output when summary is prepended to the input query, we design training schemes to optimize the end task performance. Our extractive compressor is trained with a contrastive learning objective to identify sentences that lead to target outputs, and our abstractive compressor is distilled [West et al. 2022] from an extreme-scale LM (e.g. GPT-3), which achieves impressive

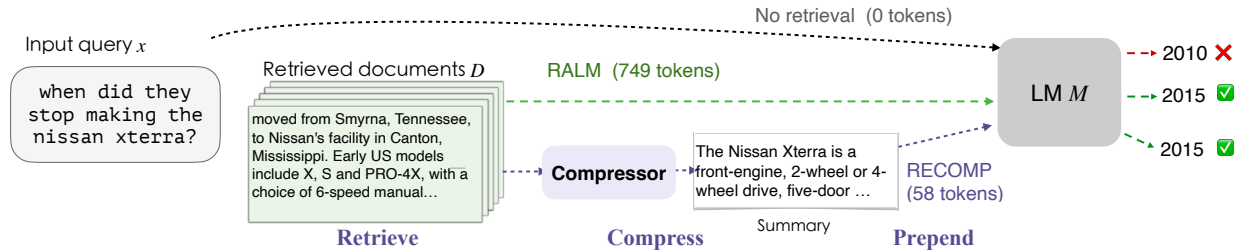


Figure 3.1: An illustration of **RECOMP**, which compresses retrieved documents into a textual summary before prepending it as input to a language model at inference time. The compressed summary guides the LM to generate the correct answer, while reducing the computation costs required to encode the documents. We describe training procedures for the compressors in Section 3.3.

summarization performance.

Our experiments show that **RECOMP** can improve performance of frozen LMs on language modeling [Merity et al. 2016] and three question answering datasets (Natural Questions [Kwiatkowski et al. 2019], TriviaQA [Joshi et al. 2017a] and HotpotQA [Yang et al. 2018]), while prepending significantly fewer tokens compared to RALM without compression. We present two oracle compression methods – an extractive oracle which selects a sentence in evidence documents that leads to the best task performance and an abstractive oracle which chooses between a summary generated by extreme-scale LLM (e.g. GPT-3) and no retrieval augmentation that leads to the best task performance. Both oracle methods achieve a compression rate as low as 6% and significantly *outperforms* prepending full documents. Our trained compressors also show promising results. For language modelling, both trained compressors achieve a compression ratio of 25% with minimal performance drop. When applied to QA datasets, our best model compresses the documents to 5 - 10% of the original tokens with at most less than 10% relative performance drop and 2x inference time speed up. We conclude with careful analyses of our approach that reveal both its strength and weaknesses, thereby building foundation for future work.

3.2 PROBLEM FORMULATION: RECOMP

Given an input sequence $\mathbf{x}$, a target output sequence $\mathbf{y}$ and a set of N retrieved documents D ($[d_1, d_2, \dots, d_N]$),¹ **RECOMP** compresses retrieved documents D with respect to $\mathbf{x}$ into a summary $\mathbf{s}$ which captures core information in D relevant to $\mathbf{x}$ with significantly fewer tokens than D . Our architecture consists of two modules: compressor c_θ and LM M . In this work, we assume a blackbox LM and train the compressor. Given the set of retrieved N documents ($[d_1, d_2, \dots, d_N]$) and the input sequence $\mathbf{x}$, a compressor returns a token sequence $\mathbf{s}$. We design our compressor to be substantially smaller than LM M , as we aim to reduce computational costs of encoding a set of retrieved documents.

The output from compressor, $\mathbf{s}$, should be: (1) **Concise**: The summary should be as short as possible to optimize efficiency. If the retrieved documents do not contain relevant information or retrieval augmentation is not necessary, $\mathbf{s}$ can be an empty sequence. (2) **Effective**: when $\mathbf{s}$ is prepended to input sequence $\mathbf{x}$ and provided to LM M as a prompt, LM should generate the target output sequence $\mathbf{y}$. (3) **Faithful**: $\mathbf{s}$ should be a faithful and interpretable summary of the input document set (i.e., $\mathbf{s}$ must be entailed by the input document set ($[d_1, d_2, \dots, d_N]$)). We focus on training compressors for conciseness and effectiveness. We summarize the key ideas for our two compressors, extractive compressors and abstractive compressor here, and discuss their training schemes formally in Section 3.3.

EXTRACTIVE COMPRESSOR Given n sentences $[s_1, s_2, \dots, s_n]$ in the input document set ($[d_1, d_2, \dots, d_N]$), we train a dual encoder model enc_θ which embeds sentence s_i and the input sequence $\mathbf{x}$ into fixed-dimensional embeddings respectively. Their inner product represents how helpful it would be for the LM M to prepend s_i to the input $\mathbf{x}$ to generate $\mathbf{y}$. The final summary $\mathbf{s}$ from the compressor will be a concatenation of top N sentences ranked by their inner product with the input. As this

¹Improving retriever is not the focus of this work, so we assume a set of retrieved documents are provided.

approach is extractive, we assume the faithfulness criteria is mostly satisfied.²

ABSTRACTIVE COMPRESSOR We train an encoder-decoder model encdec_θ to serve as an abstractive compressor, which takes the input sequence $\mathbf{x}$ and a concatenation of retrieved document set $D [d_1; d_2; \dots d_N]$ and output a summary $\mathbf{s}$. Although we do not have human annotations to train this model, prior work [Chen et al. 2023a; Goyal et al. 2022a; Potluri et al. 2023] suggests that the extreme-scale LMs can generate good query-focused summaries when prompted carefully. Yet, using an extreme-scale model as the compressor is not desirable as we want the compressor to be substantially smaller than the LMs. Thus, we perform **distillation** [Hinton et al. 2015] of extreme-scale LMs to build a lightweight abstractive compressor encdec_θ . We do not train specifically for faithfulness, but later manually evaluate the faithfulness in Section 6.4.

3.3 LEARNING THE COMPRESSORS

Our compressor resembles text summarization models in that the output should be faithful to the original input, yet the main goal is different. Instead of capturing salient information for humans readers, compressors aim to produce a concise text that are useful for a LM on an end task. In this section, we describe how to train the extractive compressor (§3.3.1) and the abstractive compressor (§3.3.2) leveraging end task signals. Further training details can be found in the Appendix 8.0.2.

3.3.1 EXTRACTIVE COMPRESSION

As we formulate extractive compression as a ranking problem, training extractive compressor resembles training a reranker for the retrieved documents³ with two differences. First, our

²Recent work [Zhang et al. 2022b] shows that extractive approach does not always preserve faithfulness, but such cases are still rare compared to abstractive approaches which can easily hallucinate.

³Ram et al. [2023] proposes a document reranker based on a cross-encoder model, which is a similar set-up to our sentence selector, but less compute efficient.

Algorithm 1 Learning an extractive compressor for language modeling task.

Input: Base LM M , Compressor enc_θ , Training data $\{\mathbf{x}_i, \mathbf{S}_i, \mathbf{y}_i\}_1^T$ where $\mathbf{x}_i$ is input, $\mathbf{S}_i = \{\mathbf{s}_j\}_1^n$ is a set of candidate sentences from the retrieved documents for $\mathbf{x}_i$, $\mathbf{y}_i$ is the target answer, and score threshold ϵ .

Output: An updated extractive compressor encoder enc_θ

```
1:  $\mathcal{T} \leftarrow \emptyset$ 
2: for  $i \in \{1, \dots, T\}$  do
3:    $\mathbf{p}_i \leftarrow \text{argMax}_{\mathbf{s}_j \in \{\mathbf{S}_i\}} \text{Score}(M, \mathbf{y}_i, [\mathbf{s}_j; \mathbf{x}_i])$ 
4:   for  $j \in \{1, \dots, n\}$  do
5:      $\mathcal{L} \leftarrow \emptyset$ 
6:     if  $\text{Score}(M, \mathbf{y}_i, [\mathbf{s}_j; \mathbf{x}_i]) + \epsilon < \text{Score}(M, \mathbf{y}_i, [\mathbf{p}_i; \mathbf{x}_i])$  then
7:        $\mathcal{L} \leftarrow \mathcal{L} \cup \mathbf{s}_j$ 
8:     end if
9:   end for
10:  if  $|\mathcal{L}| > 0$  then
11:     $\mathcal{N}_i \leftarrow \text{argTop5}_{\mathbf{s}_j \in \mathcal{L}}(\langle \text{enc}_\theta(\mathbf{s}_j), \text{enc}_\theta(\mathbf{x}_i) \rangle)$ 
12:     $\mathcal{T} \leftarrow \mathcal{T} \cup \{(\mathbf{x}_i, \mathbf{p}_i, \mathcal{N}_i)\}$ 
13:  end if
14: end for
15:  $\text{enc}_\theta = \text{Finetune}(\text{enc}_\theta, \mathcal{T})$ 
```

compressor considers a different granularity of input (sentence) compared to the initial retrieval unit (paragraph). Second, the sentence is evaluated based on whether it is useful as input for the LM M on the downstream task [; Shi et al. 2023b].

MODEL We train a dual-encoder model enc_θ which encodes the input context $\mathbf{x}$ and the candidate sentence $\mathbf{s}_i$ separately. We obtain an embedding of $\mathbf{x}$ and $\mathbf{s}_i$ by taking the representation of the [CLS] token respectively, and compute their similarity by their inner product. We initialize our model with the contriever checkpoint [Izcard et al. 2021]. This model consists of 110M parameters, satisfying the efficiency desideratum of compressor.

TRAINING 1 presents pseudocode for training an extractive compressor with contrastive loss for the language modeling task. For each input query $\mathbf{x}_i$, we identify positive and negative sentences from retrieved documents.

For each pair of input sequence $\mathbf{x}_i$ and candidate sentences $\mathbf{s}_j$, we measure

$$\text{Score}(M, \mathbf{y}_i, [\mathbf{s}_j; \mathbf{x}_i]) = \log p_M(\mathbf{y} | [\mathbf{s}_j; \mathbf{x}_i]),$$

the log likelihood assigned to target output according to LM M when candidate sentence is prepended to the input. We consider the sentence with the highest log likelihood as a positive example $\mathbf{p}_i$ (line 3). To construct negative examples $\mathcal{N}_i = \{\mathbf{n}_k\}_{k=1}^5$, we choose up to five sentences with top contriever score that has the log likelihood lower than the positive sentence for a threshold (line 6).

Training a compressor for QA task works similarly, but scoring will evaluate whether the LM will generate the correct answer with summary prepended (change in line 6). Pseudo code for the QA tasks is in Figure C2 the Appendix. We train our encoder with a contrastive loss [Karpukhin et al. 2020b], maximizing the similarity between positive pairs $(\mathbf{x}_i, \mathbf{p}_i)$ and minimize negative pairs $(\mathbf{x}_i, \mathbf{N}_i)$. The training objective is to minimize $-\log \frac{e^{\text{sim}(\mathbf{x}_i, \mathbf{p}_i)}}{e^{\text{sim}(\mathbf{x}_i, \mathbf{p}_i)} + \sum_{\mathbf{n}_j \in \mathbf{N}_i} e^{\text{sim}(\mathbf{x}_i, \mathbf{n}_j)}}$.

DATA For the language modeling task, we generate training data using the training split of the Wikitext-103 dataset, selecting the top 20 sentences from the top 5 BM25 retrieved documents for each input context $\mathbf{x}$. For the QA tasks, we generate training data using the training split and consider the top 20 sentences from the top 5 contriever-ms-marco⁴ retrieved documents. We report detailed statistics for the training data in Table C1 in the appendix. For each sentence from the retrieved documents, we prepend the Wikipedia page title to it to for decontextualization.

3.3.2 ABSTRACTIVE COMPRESSION

We distill the query-focused summarization ability of extreme-scale LM by generating training dataset with it, filtering the generated data, and training an encoder-decoder model from the

⁴<https://huggingface.co/facebook/contriever-msmarco>

Algorithm 2 Learning an abstractive compressor for language modeling task.

Input: Teacher LM M_t , LM M , Summarization prompt set $\{\mathbf{p}_i\}_1^n$, Compressor encdec_θ , Training data $\{\mathbf{x}_i, \mathbf{D}_i, \mathbf{y}_i\}_1^T$ where $\mathbf{x}_i$ is input, $\mathbf{D}_i$ is the set of retrieved document for $\mathbf{x}_i$, $\mathbf{y}_i$ is the target answer.

Output: An updated encdec_θ

```

1:  $\mathcal{T} \leftarrow \emptyset$ 
2: for  $i \in \{1, \dots, T\}$  do
3:    $v_r \leftarrow -\infty$ 
4:   for  $j \in \{1, \dots, n\}$  do
5:      $\mathbf{s}_j = \text{Decode}(M_t, [\mathbf{p}_j; \mathbf{x}_i; \mathbf{D}_i])$ 
6:      $v_j = \text{Score}(M, \mathbf{y}_i, [\mathbf{s}_j; \mathbf{x}_i])$ 
7:     if  $v_j > v_r$  then
8:        $\mathbf{s}_t \leftarrow \mathbf{s}_j, v_r \leftarrow v_j$ 
9:     end if
10:  end for
11:   $v_d = \text{Score}(M, \mathbf{y}_i, [\mathbf{x}_i])$ 
12:  if  $v_r < v_d$  then
13:     $T \leftarrow T \cup \{(\mathbf{x}_i, \mathbf{D}_i, \emptyset)\}$ 
14:    continue
15:  end if
16:   $T \leftarrow T \cup \{(\mathbf{x}_i, \mathbf{D}_i, \mathbf{s}_t)\}$ 
17: end for
18:  $\text{encdec}_\theta = \text{Finetune}(\text{encdec}_\theta, T)$ 

```

filtered dataset [West et al. 2022]. In contrast to prior work [Jung et al. 2023] which use intrinsic summarization metric for filtering, we filter with the LM’s end task performance with the generated summaries prepended. Fig. 2 presents pseudo algorithm for abstractive compressor training.

3.3.2.1 CREATING TRAINING DATASET FOR DISTILLATION

GENERATION FROM TEACHER MODEL For the language modeling task, we manually construct four prompts to summarize evidence document set $(\{\mathbf{p}_i\}_1^n)$.⁵ Given an input $\mathbf{x}_i$, a retrieved document set $\mathbf{D}_i$, and a prompt $\mathbf{p}_j$ to summarize the document set with respect to the input, GPT-3.5⁶ generates a summary (line 3).

⁵The exact prompts can be found in Table C10 in 8.0.1.

⁶We use gpt-3.5-turbo in all our experiments.

FILTERING WITH CRITIC After generating a summary for each prompt template, we select the summary which results in the highest end task performance for each example (s_t) as the target summary (line 4-10). $\text{Score}(M, y_i, [s_j; x_i])$ is the same as the extractive compressor above. We then compare the end task performance with the target summary prepended and with input x_i only (i.e. no retrieval) on base model M (line 6). If the end task performance gets worse (e.g., increase in perplexity) when prepending the summary, we set the target summary to an empty string (line 7), otherwise we add the target summary to the training set (line 10). This allows for selective augmentation and mitigates the risk of prepending irrelevant documents.

Constructing training datasets for the question answering tasks works similarly, with the following modifications. As summarization for the question answering task is more straightforward, we use a single prompt for each dataset. We filter out examples where prepending the summary does not lead to performance improvement. Pseudo code for the QA tasks is in Figure C3 in the Appendix.

MODEL & TRAINING We use encoder-decoder LM (775M), initialized from T5-large checkpoint [Raffel et al. 2020a]. This model has been trained with summarization datasets [Hermann et al. 2015].

DATA We summarize top 5 retrieved documents for both language modeling and question answering tasks. We generate training examples using subsets of the training set for the Wikitext-103 dataset, the entire NQ training set and TriviaQA training set. For HotpotQA, we only generate abstractive summaries for the subset of training data (56%) where the gold answer is in the retrieved documents to reduce API costs. We report percentage of data filtered and empty summaries in Table C1 in 8.0.1.

3.4 EXPERIMENTAL SETTINGS

We evaluate our approach on language modeling and open-domain QA following prior work [Ram et al. 2023; Shi et al. 2023b]. For both tasks, we report the task performance as a measure of effectiveness and the number of tokens provided in context as a measure of efficiency.

3.4.1 LANGUAGE MODELING

We evaluate language modeling perplexity on WikiText-103 [Merity et al. 2016] benchmark on three open-sourced LMs of varying scale: GPT2 (117M), GPT2-XL (1.5B; Radford et al. [2019]) and GPT-J (6B; Wang and Komatsuzaki [2021]). We train our compressors using GPT2 as the base model and evaluate whether the trained compressor transfer to GPT2-XL and GPT-J. We use the BM25 retriever [Robertson and Zaragoza 2009] to retrieve from the Wikipedia corpus from Dec. 20, 2018 [Karpukhin et al. 2020b]. The articles are truncated into non-overlapping documents of 100 words. During retrieval, articles containing the input sequence x is removed from the corpus to prevent data contamination. Following [Ram et al. 2023], we perform retrieval every 32 tokens.

3.4.2 OPEN-DOMAIN QA

DATASETS We evaluate our model on three benchmark datasets: Natural Questions (NQ) [Kwiatkowski et al. 2019], TriviaQA [Joshi et al. 2017a], and HotpotQA [Yang et al. 2018]. We report results on development set of NQ, test set of TriviaQA and randomly sampled 500 examples from HotpotQA development set. We report Exact Match (EM) and token-level F1 of answer strings.

BASE LANGUAGE MODELS & RETRIEVAL CORPUS We use Flan-UL2 (20B)[Chung et al. 2022], a large scale instruction-tuned LM. We use contriever model trained on MS MARCO dataset [Campos et al. 2016] as a retriever on Wikipedia corpus from Dec. 20, 2018 for all three datasets. The articles are truncated into non-overlapping documents of 100 words.

PROMPT FORMAT We include few-shot in-context examples in the prompt, followed by the retrieved documents and the question. We use five randomly sampled in-context examples from training set, which constitutes 110, 147, and 149 tokens on average for NQ, TQA and HotpotQA respectively. We concatenate retrieved documents in ascending order of retrieval score, with the highest scored one closest to the question [Si et al. 2022]. We do not include the retrieved documents for in-context examples as it did not improve performance. An example input can be found in Appendix Table C9.

3.4.3 BASELINES AND ORACLES

BASELINES We first consider two heuristic token and phrase-level compression methods: **BoW**, which converts the retrieved documents to a list of ordered unigram and concatenates them together and **Named Entities (NE)**, which extracts a list of ordered named entities from retrieved documents and concatenates them. For the extractive compressor on the language modeling task, we use **BM25** and **Contriever** [Izacard et al. 2021], which rank the sentences by their similarity to the input x as baselines. For the QA datasets, we report results using **BM25**, **Contriever** finetuned on MS MARCO and **DPR** [Karpukhin et al. 2020c] fine-tuned on NQ. We also report a **Random** baseline which randomly selects a sentence from the retrieved documents. For abstractive compression, we report the performance of the off-the-shelf **T5 (large, 770M)** model and that of **GPT-3.5** model. As we experimented with multiple prompts for the language modeling task, we report the performance of the summaries generated by **GPT-3.5** model with the best single prompt.

ORACLE We explore the performance upperbound of compression by considering two oracle approaches. For the extractive approach, we construct oracle compressor by considering all sentences s_i in the evidence document set and choosing the sentence that leads to the best end task performance for each example. For the abstractive approach, we consider summaries generated

Table 3.1: Results on language modeling task. We report results on GPT-2, GPT2-XL and GPT-J with compressors trained with GPT-2.

In-context Evidence	In-Domain		Out-Domain			
	GPT2 (117M)		GPT2-XL (1.5B)		GPT-J (6B)	
	# tokens	PPL	# tokens	PPL	# tokens	PPL
-	0	37.84	0	19.89	0	11.44
<i>RALM without compression</i>						
Top 1 document	141	32.90	141	17.86	141	10.57
Top 5 documents	512	35.53	-	-	-	-
<i>Phrase/token level compression</i>						
Top 1 document (BoW)	66	36.13	66	18.85	66	10.97
Top 1 document (NE)	34	37.23	33	19.67	33	11.39
<i>Extractive compression of Top 5 documents (select top 1 sentence)</i>						
Oracle	32	30.36	32	16.58	31	9.92
Oracle (w/ gpt2)	32	30.36	32	16.99	32	10.22
Random	27	36.98	27	19.55	27	11.32
BM25	33	36.63	33	19.02	33	11.08
Contriever	33	35.54	33	18.98	33	11.05
Ours (init. w/ Contriever)	31	33.67	31	18.19	31	10.73
<i>Abstractive compression of Top 5 documents</i>						
Oracle	68	30.67	66	16.87	65	10.10
Oracle (w/ gpt2)	68	30.67	68	17.23	68	10.37
GPT-3.5	33	34.84	33	18.70	33	10.96
T5	15	37.80	15	19.92	15	11.5
Ours (init. w/ T5)	15	33.64	15	18.09	15	10.66

from different prompts ($\{s_j\}_1^n$ in Figure 2) and an empty summary, and choose the one that leads to the best end task performance. As oracle compression is model dependent, we also report model-independent results by using GPT-2 as a reference LM (*Oracle w/ gpt2*) to test how well oracle sentences for one model transfer to other models for the language modeling task.

3.5 RESULTS

LANGUAGE MODELING Table 3.1 reports the results on language modeling task. *All* retrieval augmentation methods improve perplexity over no retrieval setting across three LMs. Heuristic

token / phrase-level compression methods (BoW and NE) are worse than prepending uncompressed documents, potentially due to the disfluency of the prepended text.

Both oracle settings show substantial gain over prepending the entire document set, with only 6-13% of tokens. More tokens are not always better: prepending top 1 document outperforms prepending top 5 documents. This confirms that the naive retrieve-and-prepend approach has a significant room for improvement, as prepending irrelevant documents can hurt performances.

Our trained extractive compressor significantly outperforms other extractive baselines (Contriever and BM25) across all three LMs, while prepending slightly fewer tokens. Compared to prepending one document, we achieve a compression ratio of 25% at minimum performance drop. Our trained abstractive compressor performs the best across the board, achieving the lowest perplexity **and** the highest compression ratio. Our abstractive compressor achieves high compression rate through selective augmentation, prepending summaries to only 33% of examples (length distribution of generated summaries in Fig. C4).

OPEN-DOMAIN QA We report the results on QA tasks in Table 3.2. Similar to the language modeling task, all retrieval augmentation methods improve performance over no retrieval setting, across three datasets, consistent with previous study on other LMs [Mallen et al. 2022; Shi et al. 2023b; Si et al. 2022]. Unlike language modeling, prepending five documents shows significant gains over prepending a single document, motivating the use of compression to incorporate more documents.

We find that extractive oracle outperforms the abstractive one in all datasets. Extractive oracle selects the best one from N candidate sentences, while abstractive oracle selects from two options – prepending GPT-3.5 summary or prepending nothing. Both oracles show improvements over prepending all information, suggesting that removing irrelevant information benefit the model.⁷

Among extractive baselines, DPR performs the best as it has been fine-tuned on high-quality

⁷We provide an example where our compressed summary yields correct answer while prepending full document does not in Table C11 in the appendix.

Table 3.2: Open-domain QA results with Flan-UL2 (20B) as the LM M . We report number of tokens provided as in-context evidence document, excluding the in-context examples. We train separate compressors (one extractive, one abstractive) for each dataset. Extractive compressor selects one sentence for NQ/TQA, and two sentences for HotpotQA.

In-Context evidence	# tok	NQ		# tok	TQA		HotpotQA		
		EM	F1		EM	F1	# tok	EM	F1
-	0	21.99	29.38	0	49.33	54.85	0	17.80	26.10
<i>RALM without compression</i>									
Top 1 documents	132	33.07	41.45	136	57.84	64.94	138	28.80	40.58
Top 5 documents	660	39.39	48.28	677	62.37	70.09	684	32.80	43.90
<i>Phrase/token level compression</i>									
Top 5 documents (NE)	338	23.60	31.02	128	54.96	61.19	157	22.20	31.89
Top 5 documents (BoW)	450	28.48	36.84	259	58.16	65.15	255	25.60	36.00
<i>Extractive compression of top 5 documents</i>									
Oracle	34	60.22	64.25	32	79.29	82.06	70	41.80	51.07
Random	32	23.27	31.09	31	50.18	56.24	61	21.00	29.86
BM25	36	25.82	33.63	37	54.67	61.19	74	26.80	38.02
DPR	39	34.32	43.38	41	56.58	62.96	78	27.40	38.15
Contriever	36	30.06	31.92	40	53.67	60.01	78	28.60	39.48
Ours	37	36.57	44.22	38	58.99	65.26	75	30.40	40.14
<i>Abstractive compression of top 5 documents</i>									
Oracle	51	45.68	53.66	37	71.01	76.38	102	35.80	46.25
GPT-3.5	56	37.12	46.35	41	62.03	69.66	107	31.60	42.65
T5	10	25.90	34.63	7	55.18	62.34	7	23.20	33.19
Ours	36	37.04	45.47	32	58.68	66.34	64	28.20	37.91

NQ data. On NQ, selecting the top 1 DPR ranked sentences from top 5 documents outperforms prepending top 1 document, with much fewer tokens (39 vs. 132). However, its performance degrades in out of domain datasets. Off-the-shelf summarization model (T5) boasts the highest level of compression, achieving 4-6 points gains in EM while adding mere 7-10 tokens.

The trained compressors, both extractive and abstractive, shows promising performances. On NQ and TQA, the abstractive approach is more effective. On NQ, it achieves a compression ratio of 5% tokens while losing 2 EM points compared to prepending full documents. On TQA, we observe similar trends, compression ratio of 5% tokens while losing 3.7 EM points compared to prepending full sets of documents. On HotpotQA which requires multihop understanding of documents, we find extractive approach to be more helpful, achieving 11% compression rate while losing 2.4 EM points compared to prepending full documents. We find that learning an abstractive compressor for more complex tasks, such as HotpotQA, demands further study. While extreme-scale LLM boasts competitive summarization performance for single document, they are not good at synthesizing information from multiple documents (Shaib et al. [2023]; See Section 6.4 for further analysis).

3.6 ANALYSIS AND DISCUSSIONS

TRANSFERRING ACROSS DIFFERENT LMS One benefit of textual summary is that they can transfer to other LMs, unlike approaches such as soft prompts [Chevalier et al. 2023; Mu et al. 2023; Wingate et al. 2022]. We evaluate whether our compressors trained to achieve high performance with respect to a specific LM (GPT2 for language modeling, FlanUL2 for open domain QA) can transfer to other LMs. For language modeling, we find that trained compressor transfers well to other LMs (GPT2-XL and GPT-J), despite they are much larger LMs (Table 3.1. For open domain QA, we tested transferring our compressors to LLaMA-13B [Touvron et al. 2023] (Table C7 in the appendix). Overall, the performance is worse than the LM from which compressors are trained

on, sometimes unable to outperform other compression baselines (e.g., no clear gain from using contriever vs. our trained contriever on TQA/HotpotQA), leaving considerable gap to the oracle compression for LLaMA itself. Yet, on NQ/TQA, our compressor obtains 5% compression ratio with less than 5 EM drop compared to full document setting, showing the robustness of our retrieve-compress-prepend paradigm.

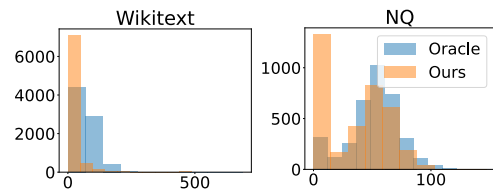


Figure 3.2: Histogram of abstractive summary length (# tokens) distribution.

HOW DO THE LENGTH OF THE SUMMARIES VARY? Can the learned compressor reliably determine when LMs require retrieved documents? As retrieved documents were **hurting** the model performances for some input queries, 4-24% of training examples for abstractive compressors contain empty summary. Fig. 3.2 presents the

length distribution of abstractive summaries on NQ and Wikitext (histograms for other datasets is in Fig. C4 in the appendix). The input document lengths do not vary significantly across examples, yet we find the abstractive summary vary significantly in length, suggesting abstractive compressor enables selective retrieval augmentation. We have not experimented selective compression with extractive compressor, fixing the number of prepended sentences for the entire dataset. We leave enabling adaptive augmentation with extractive summarizer to future work.

HOW DOES MODEL LEVERAGE THE IN-CONTEXT DOCUMENTS? We evaluate whether retrieval augmented LMs tend to copy answers verbatim from in-context evidence documents or generate answers not present in the documents. This is an desired behavior *only* when the gold answer is in the evidence. We first report how frequently a gold answer span is present in evidence text (% Gold in Evi). As expected, full documents contain the answer most frequently, followed by NE and GPT-3 compression. However, having more gold answers in the evidence doesn't equate to better performance, as the model cannot always identify the correct answer from the evidence (84 % for **NE** v.s. 98% for **T5(ours)**).

Table 3.3: Analysis on usage of in-context evidence for NQ. For the last column, we report frequency of model copying from the evidence on a subset where gold answer is present / when it is not.

Evidence	EM	%Gold in Evi.	%Pred in Evi.
Top 1	33.1	36	92 / 51
Top 5	39.3	57	96 / 81
NE	26.0	46	84 / 48
Oracle sent	60.2	34	93 / 16
Contriever	30.2	25	88 / 36
Ours	36.6	28	90 / 33
GPT-3.5	37.1	45	98 / 85
T5	25.9	30	52 / 20
Ours	37.0	34	98 / 39

We also observe that model can be easily distracted by irrelevant contexts, copying from a document span even when it does not contain gold answer, echoing findings from prior work [Shi et al. 2023a]. Prepending top 5 documents has a higher frequency (81%) of copying incorrectly compared to top 1 document (51%), and GPT-3 compression leads to an even higher incorrect copying frequency (85%), potentially as query-focused summarization generates sentences that seemingly contains the answer. Our compressor successfully reduce such erroneous behavior to 39%.

ARE THE GENERATED SUMMARIES FAITHFUL AND COMPREHENSIVE? We (the authors) manually evaluate outputs of the abstractive compressors on two axes [Chen et al. 2023a]: **Faithfulness**: whether the summary can be entailed by the retrieved documents, **Comprehensiveness**: whether the summary contains sufficient information to answer the question, regardless of whether the generated information comes from the retrieved documents. For both, we select one of three labels: **Yes**, **Partially**, **No**, and report the % of **Useful** summaries which are both faithful and comprehensive. Annotation sample can be found in Table C12 in the appendix. We evaluate the summaries generated by GPT-3.5 and our abstractive compressor. We randomly sample 30 non-empty summaries from the test set.

Table 3.4 presents annotation results. GPT-3.5, substantially bigger than our compressor, generates more useful summary across all three datasets. Overall, our abstractive compressors were less faithful compared to the original GPT-3.5, while improving comprehensiveness. The effectiveness of summarization also depends on the datasets – summaries

Table 3.4: Manual analysis on abstractive summaries generated for NQ, TQA and HotpotQA (HQA) dataset.

Dataset	Model	% Faithful			% Compre.			% Use.
		Y	P	N	Y	P	N	
NQ	GPT-3.5	90	0	10	97	0	3	83
	Ours	80	13	7	100	0	0	80
TQA	GPT-3.5	97	0	3	90	0	10	83
	Ours	83	3	14	96	0	4	77
HQA	GPT-3.5	74	0	26	78	0	22	50
	Ours	67	0	33	74	0	26	40

from both models were the most faithful for TQA and the least faithful for HotpotQA dataset. In terms of comprehensiveness, we find both models easily find the information for NQ, but struggle with HotpotQA. These results partially explain why the performance gain was limited for HotpotQA.

3.7 RELATED WORK

EFFICIENT RALM He et al. [2021] improves efficiency of RALMs by reducing retrieval cost, such as data store compression and dimensionality reduction for neural retriever. A line of work proposes to reduce retrieval frequency [; Mallen et al. 2022; Martins et al. 2022]. In this work, we improve efficiency of RALM by compressing retrieved documents into a concise summary or an empty sequence, facilitating selective retrieval augmentation.

DISTILLATION / GOAL ORIENTED SUMMARIZATION Recent work introduces symbolic knowledge distillation [West et al. 2022], which transfers knowledge from a teacher model by training a student model on a training dataset generated with the teacher model. For better performance, they

introduce critic criteria, which filter undesirable examples from generated training dataset. Such distillation technique has been applied for various applications including summarization [Jung et al. 2023], which aims to generate high quality summaries while we optimize for generating effective summary for downstream LMs. Our setting is similar to Hsu and Tan [2021], which trains an extractive summarization model to optimize for prediction accuracy of a sentiment prediction model which uses the summary.

3.8 CONCLUSION

We introduce **RECOMP**, a method which compresses retrieved documents into textual summaries before prepending them to improve in-context retrieval augmented language models. We present two compression models – an extractive compressor and an abstractive compressor. We design a training scheme which leverages end task signals from a blackbox LM to generate useful summaries and enable selective augmentation. Our experiments show that our compressors can improve the efficiency of retrieval augmented LMs significantly with minimal drop in performances.

4 | REFRESHKV: UPDATING SMALL KV CACHE DURING LONG-FORM GENERATION

The previous chapter reduces inference cost by compressing the input before it reaches the language model, building on the assumption that only a subset of tokens is needed to answer a given query. This chapter relaxes that assumption and targets settings where the full input may be useful overall, even if only a small subset is useful at any given decoding step. The key observation is that rather than preemptively selecting relevant tokens before generation begins, we can retain the Key-Value (KV) cache for all tokens and dynamically attend to a subset of it at each step. This preserves access to tokens that prove useful later in generation, addressing a key limitation of eviction-based methods that discard tokens irreversibly. This chapter is based on work with Tanya Goyal and Eunsol Choi [Xu et al. 2025].

4.1 INTRODUCTION

Large language models (LLMs) are capable of ingesting extremely long inputs and generating long outputs [Gemini 2024; Meta 2024]. Yet, deploying such long-context LLMs is very costly. As the context length increases, memory usage for storing the key-value (KV) cache increases linearly, while attention computation scales quadratically. These two factors lead to high latency during inference; Adnan et al. [2024] reports 50x latency increase as context length increased 16x

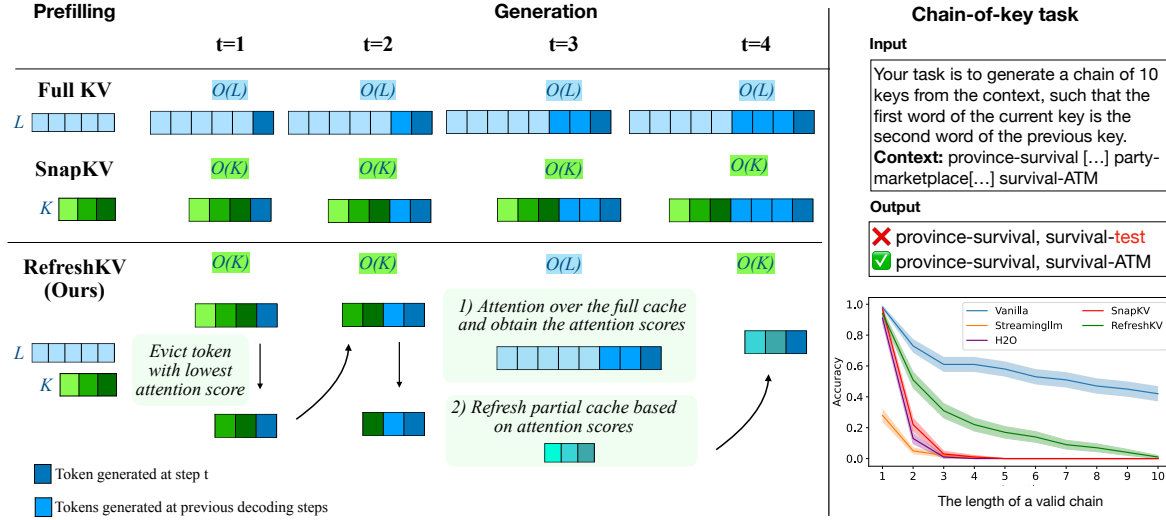


Figure 4.1: Left: Illustration of **RefreshKV** (with $L = 5$, $K = 3$ and a stride $S = 3$) compared to baseline (SnapKV and Full KV) when generating four tokens. The figure shows the computation complexity of attention operation, and the size of the KV cache used at each decoding step for each method. Our approach alternates between inferencing with the partial cache($t=1,2,4$) and the full cache($t=3$). Compared to eviction-based method (e.g. SnapKV) which completely discard the evicted tokens, **RefreshKV** updates the partial cache based on attention scores over the entire context during the full attention steps. Right: An example of the chain-of-key task and performance of **RefreshKV** and the baselines. RefreshKV maintains performances across different length while eviction-based baselines’ performance degrades when generating a chain with more than one key.

for the MPT-7B model [MosaicML 2023].

Prior works [Adnan et al. 2024; Beltagy et al. 2020a; Child et al. 2019; Li et al. 2024; Xiao et al. 2023; Zhang et al. 2024b] propose to maintain a smaller KV cache by evicting a subset of past tokens. These approaches improve both the memory and computation efficiency, as the KV cache of only a subset of tokens will be kept and attention computation is reduced. However, once an input token is eliminated from the KV cache (either based on locality assumption [Xiao et al. 2023] or by eviction during the generation process [Zhang et al. 2024b]), one cannot recover eliminated tokens. We find that while such methods show minor degradation compared to full KV cache in short-form generation tasks, their performance degrades rapidly for long-form generation tasks.

Having observed the limitations of existing approaches, we propose a novel approach, **RefreshKV**, which periodically refreshes the smaller KV cache during the generation process. Our method keeps the full KV cache throughout inference (thus no gain in memory footprint), but

perform attention over a dynamically constructed small KV cache to achieve inference speedups. Our method alternates between two modes of generation: generation that attends over the full KV cache and generation that attends over a smaller KV cache with subset of tokens (see Figure 4.1). To construct the smaller KV, we identify the topK attended tokens from the most recent step that attends over the full KV cache, observing that consecutive tokens have similar attention pattern [Li et al. 2024].

A key component of RefreshKV is deciding when to perform the computationally expensive full attention steps and refresh the small KV cache. Instead of mandating a fixed (and potentially suboptimal) schedule, RefreshKV compares the query embedding similarity of the current and previous full attention step, and dynamically triggers full attention step when the similarity is low. Our approach (no KV eviction, dynamically constructed smaller KV, low latency) establishes a middle ground between full attention (no KV eviction, high latency, high performance) and sparse attention (KV eviction, reduced latency, low performance), particularly useful for long-form generation.

Our method can be applied to any off-the-shelf LLM. We experiment with two long-context LLMs, Llama-3.1-8B [Meta 2024] and Qwen2-7B [Yang et al. 2024a]. We compare against KV eviction baselines StreamingLLM [Xiao et al. 2023], H₂O [Zhang et al. 2024b] and SnapKV [Li et al. 2024] on the long-range language modeling task and a suite of downstream long-context tasks [Bai et al. 2023; Ye et al. 2025; Zhang et al. 2024a] that require long outputs given long inputs.

Our experiments show that **RefreshKV** outperforms eviction-based methods in both these settings, with similar level of speed-up. In particular, we examine two long-form generation tasks that are not evaluated by previously proposed eviction-based methods: (1) when majority of tokens are required to generate the output (e.g. converting information in an HTML page to a TSV file) and (2) when the important tokens required at the current generation step is dependent on the previously generated tokens (a new task, **Chain-of-key**, as depicted in Figure 4.1). While eviction-based methods such as H₂O and SnapKV fail completely in HTML to TSV task [Ye et al. 2025],

achieving 0 F1 score, RefreshKV recovers 52% of the performance. Our analysis shows that the performance gains are attributed to updating the partial cache rather than occasionally attending to the entire output. Lastly, we explore continued pretraining Llama-3.1-8B with RefreshKV, which leads to further improvements. Our contributions are as follows:

- We identify the failures of existing KV eviction methods when LLMs are tasked with challenging long-form generation tasks. To reveal this weakness, we propose a new task (**Chain-of-key**) which requires remembering the input context more comprehensively.
- Motivated by the failures of KV cache eviction methods, we introduce a new inference method, **RefreshKV**, that rebuilds a smaller KV cache periodically during long-form generation.
- We evaluate our method comprehensively on various benchmarks and two LLMs, and conduct ablation studies on our design choices (e.g., dynamic stride vs. fixed stride cache updates).

4.2 REFRESHKV FOR LONG-FORM GENERATION WITH LONG-CONTEXT LLMs

4.2.1 BACKGROUND AND SETTING

Let M be a language model and x be an input sequence of tokens, $x = x_1, \dots, x_L$. At inference time, M generates an output token sequence $\hat{y} = y_1, \dots, y_N$ in two stages: (1) **Pre-filling stage** where M ingests the input and constructs the KV cache for all L tokens, and (2) **Generation stage** where it samples one token y_i at a time from the conditional distribution $P_M(y_i|x, y_1 \dots y_{i-1})$. At each step, the model attends to tokens in the KV cache, and updates the cache to include the current token’s key-value pairs.

Our goal is to reduce the inference latency during the generation stage without severe degradation of model performance. There are two main reasons for latency increase; first, the attention

computation increases quadratically with input length L . Second, a large L necessitates maintaining a large KV cache of the past tokens, incurring latency due to the full KV cache movement from the GPU HBM.¹

Prior approaches, like H₂O [Zhang et al. 2024b] and SnapKV [Li et al. 2024], address this by permanently evicting “unimportant” tokens during the decoding process to maintain a small KV cache. While such methods have shown to be effective for short-form generation task such as “Needle-in-a-Haystack”(NIAH) [Kamradt 2023], it has the potential downside of prematurely removing tokens useful for subsequent generation steps. Instead of this strict strategy, we propose to periodically *update* the small KV cache by performing full attention over all the tokens in the context and constructing the small cache based on the attention pattern. As the cache is only occasionally updated, our method reduces both attention computation and data movement by attending to the small cache.

4.2.2 METHODOLOGY AND IMPLEMENTATION

We present the pseudocode for generating output tokens using **RefreshKV** in Figure 4.2. The algorithm takes as input a language model M and a sequence of input tokens $x_1, \dots, x_L$. As a first step, we prefill M with the input sequence. Then, we alternate between full and partial attention. Our approach maintains two separate KV caches C_f and C_p , corresponding to KV cache used in the full and partial attention steps respectively. These three components of the algorithm are described below:

PREFILLING STAGE (lines 1-2): Given input $x_1, \dots, x_L$, we prefill with full attention M and initialize full KV cache C_f with L tokens. We also obtain the attention scores a_L for the last token x_L . To determine the top K tokens to keep, we employ max pooling over attention scores of surrounding tokens, instead of the raw attention scores to preserve information completeness following prior

¹Adnan et al. [2024] reports up to 40% of the inference latency can be attributed to data movement.

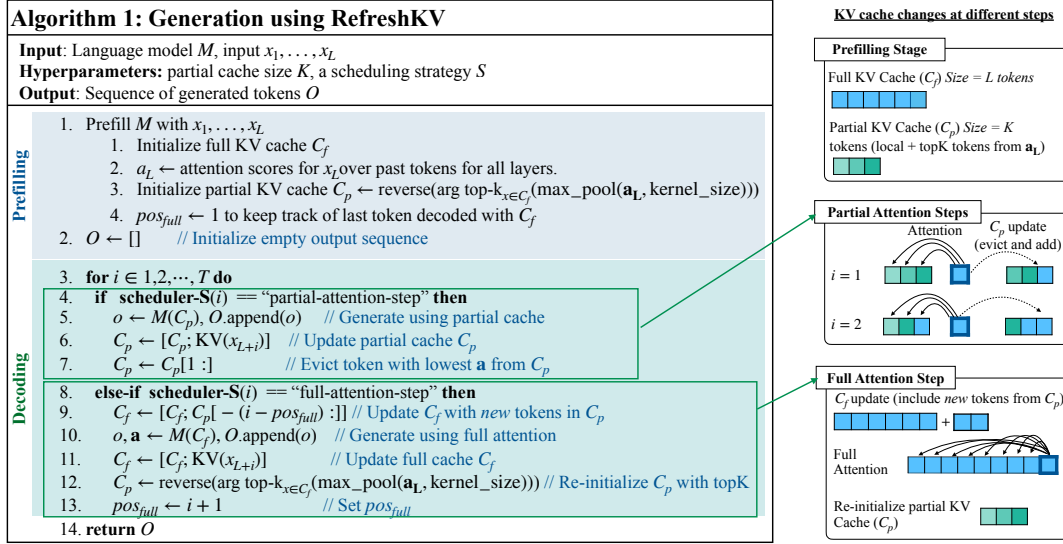


Figure 4.2: Pseudocode for **RefreshKV**. The model prefills the prompt with full attention and initialize the partial cache C_p cache with attention scores of the last token. For each partial attention step, we decode with the partial cache and append the KV pairs of the input token to the partial cache. We evict the token with the lowest attention score to maintain a fixed-sized partial cache. For the full attention step, we first update the full KV cache with the new tokens decoded with the partial cache, then decode with the full cache and refresh the partial cache.

work [Li et al. 2024].²

DECIDING WHEN TO DECODE WITH FULL CACHE (line 4): We need to decide when to alternate between performing attention over all tokens and performing attention over the smaller cache. One straightforward way is to use a fixed schedule, i.e. performing full attention every S steps. However, this enforces the same schedule for *all* the layers and input text. Instead, we propose an adaptive schedule based on the similarity between query vector of the current step and the query vector of the most recent full attention step. Intuitively, if the query vector of a particular layer and head for the current step is similar to the query vector of the most recent full attention step, the attention pattern should be similar. Thus, we only perform the full attention step when this

²For models with Grouped Query Attention [Ainslie et al. 2023], we aggregate attention scores for all query heads in the same group by taking the max to identify the top K tokens. Our ablations (reported in Table D3 in the Appendix) show that taking the max outperforms other aggregation method such as mean, or relying solely on one of the query head in the group.

similarity is lower than a threshold.

Concretely, at every S^{th} decode step, for each layer l , we first determine whether we *need* to perform full attention. We calculate the cosine similarity between the query vectors of the input token t averaged across all query heads in layer l , with the averaged query vector of the most recent full attention step for that layer. If the similarity is higher than a threshold s , we decode with the partial cache C_p , and otherwise decode with C_f for layer l . We describe details for each scenario below. To minimize the computational overhead of the similarity check, we perform this only every S steps; we call this query comparison (QC) stride.

DECODING WITH PARTIAL CACHE (lines 5-7): At each partial attention step, we generate the next token $y_t \sim M(C_p)$ using C_p to compute attention and store the KV cache of the input token. This leads to a reduction in both the attention computation FLOPs and the latency due to KV cache movement (we only need to move the smaller KV cache C_p instead of the larger full KV cache C_f , where $|C_p| \ll |C_f|$). To maintain the size of C_p as we decode each additional token and update the KV cache with this newly generated token, we remove the KV corresponding to the token with the lowest attention score in the full attention step from C_p (line 7). We note that decoding with C_p is equivalent to SnapKV [Li et al. 2024] if the partial cache is never refreshed after prefilling.

DECODING WITH FULL CACHE (lines 9-13): At each full attention step, we first update the full KV cache C_f with the key-value pairs of the tokens decoded with C_p . Next, we generate the next token $y_t \sim M(C_f)$ using the full KV cache C_f and obtain the attention scores $\mathbf{a}_L$. Finally, we refresh the partial cache C_p with the topK tokens based on $\mathbf{a}_L$.

MEMORY AND TIME REQUIREMENTS Our method has memory requirement similar to that of vanilla attention, as we are not permanently evicting any tokens from the KV cache. However, our decoding latency is on par with other KV cache eviction methods, as later shown in our experiments in Section 6.5. We discuss memory and speed considerations in detail in Appendix 8.0.7.

4.3 EXPERIMENT SETUP

MODELS AND EVALUATION TASKS We evaluate our method with two long-context language models Llama-3.1-8B [Meta 2024] and Qwen2-7B [Yang et al. 2024a]. Both models can process inputs of up to 128K tokens. We conduct experiments on language modeling and downstream tasks: We use the base model for the language modelling task and the instruction-tuned model for the downstream tasks.

- **Language modeling** We measure perplexity of the Arxiv and Book split of RedPajama [Together 2023] with context size of 16K. We report results on 100 sequences for each domain. To simulate long-form generation, we report the perplexity of the last 256 tokens.
- **Long-input, short output tasks:** We report the performance of RULER [Hsieh et al. 2024], which consists of a set of 13 tasks with context size of 32K that require short output.
- **Long-input, long output tasks** We evaluate our methods on three sets of downstream tasks which require the model to generate long-form outputs (more than 100 tokens) given long-form inputs (more than 10k tokens).³ (1) **long-context summarization tasks:** QMSum [Zhong et al. 2021], GovReport [Huang et al. 2021] and Novel Summarization [Zhang et al. 2024a] and (2) **HTML to TSV** task from LongProc [Ye et al. 2025] benchmark.⁴ We report results aggregated across three output lengths (0.5K, 2K, and 8K). We report ROUGE-L for summarization tasks and row-level F-1 score for the **HTML to TSV** task.
- **New task: Chain-of-key generation** We propose a synthetic task where model’s previous generation steps, together with its long context input, guides future generation steps. Given a context which consists of a list of two-word keys, the model is tasked with generating a sequence

³For each dataset, we filter examples with input length <10K tokens. We report dataset statistics for each dataset in Table D1 in the Appendix.

⁴We exclude the other tasks from LongProc as they primarily involve short inputs, resulting in minimal speedup in our setting. For completeness, we report the performance of these tasks in Section 8.0.12 in the Appendix, observing a similar trend as the HTML to TSV task in terms of end-task performance.

of T keys, such that the first word of the next key is the last word of the current key. This task requires models to look up information in the context based on what has been previously generated, resembling multi-hop retrieval. An example of the task is illustrated in Figure 4.1. We report accuracy of the output by the relative length of a valid chain (i.e. the length of the valid sub-chain divided by T). More details and examples are in Section 8.0.11 in the Appendix.

COMPARISON SYSTEMS We implement the following baselines: (1) *Vanilla* attention that maintains and performs attention over the full KV cache (2) *StreamingLLM* [Xiao et al. 2023] which consists of “sink tokens” and recent tokens. (3) *H₂O* [Zhang et al. 2024b] which consists of recent tokens and dynamically updated “heavy hitters”, defined by high cumulative attention scores. (4) *SnapKV* [Li et al. 2024] which consists of tokens with high attention scores from the last few tokens in the prompt. We describe the setting for each baseline in Section 8.0.6 in the Appendix.

INFERENCE SETTINGS We prefill the model with the input and report wall clock times for the decoding phase. Our experiments are run on a single A100 80GB GPU using Flash Attention [Dao 2024].⁵ We set K to be 1/8 of the input length. NovelSumm contains the longest input length (100K tokens) and we set K to be 4096, corresponds to $1/25L$. We report results with greedy decoding. For RefreshKV, we report results for two different query comparison strides $\{5, 10\}$ with a similarity threshold s of 0.85 for Llama-3.1-8B and 0.95 for Qwen2-7B. We determine the value of s by experimenting with a range of values on a held-out set of the Book dataset (reported in Section 8.0.9 in the Appendix) and apply the same threshold for all the tasks.

Method	Arxiv/Book PPL ↓	Time ↓
<i>Llama-3.1-8B</i>		
Vanilla	2.22/7.07	7.50
Streaming	2.62/7.94	6.61
H ₂ O	2.48/7.60	10.77
SnapKV	2.54/7.78	6.77
Refresh (QC=5)	2.27/7.31	6.67
Refresh (QC=10)	2.32/7.41	6.33
<i>QWEN-2-7B</i>		
Vanilla	2.33/8.26	9.07
Streaming	2.75/9.10	6.27
H ₂ O	2.68/9.02	11.57
SnapKV	2.80/9.18	6.09
Refresh (QC=5)	2.39/8.55	6.71
Refresh (QC=10)	2.49/8.72	6.33

Table 4.1: Perplexity results and latency on language modeling task for LLama-3.1-8B and QWEN-2-7B. We report results on Arxiv and Book corpora with input context length of 16K tokens. We set $K = 2048$.

4.4 RESULTS

4.4.1 LANGUAGE MODELING

Table 4.1 outlines the performance of the baselines and RefreshKV for perplexity. For both models, RefreshKV achieves better perplexity and comparable inference speeds compared to StreamingLLM and SnapKV for $QC = 10$. Our method also achieves better performance than the best baseline, H₂O, with a much shorter inference time per example, as we do not require accessing attention score at each decoding step. Setting $QC = 5$ increases inference time but also brings performance gain compared to $QC = 10$, allowing a performance-efficiency trade-off.

The key distinction between RefreshKV and SnapKV is that our method *refreshes* the partial cache as generation progresses. We compare the perplexity degradation ratio of both methods relative to vanilla attention over different generation timestamps in Figure 4.3 with Llama-3.1-8B

⁵We describe implementation details in Section 8.0.6.

Input/Output length	32K/<30	10K/ 0.1K	10K/0.7K	128K/1K	30K/2.2K	22K/50
Dataset	RULER	QMSum	GovReport	NovelSumm	HTML to TSV	Chain-of-key*
Method	Acc↑	R-L↑	R-L↑	R-L↑	F-1↑	Acc↑
Vanilla	90 / 79	25.63 / 24.98	34.11 / 33.38	31.29 / 19.91	33 / 24	56 / 83
Streaming	22 / 21	22.27 / 20.30	16.30 / 23.84	24.66 / 22.11	2 / 5	2 / 2
H ₂ O	21 / 21	22.12 / 20.83	27.41 / 26.91	19.31 / 18.51	0 / 0	10 / 11
SnapKV	79 / 58	24.33 / 22.93	28.06 / 28.80	29.23 / 19.09	0 / 0	12 / 13
RefreshKV (QC=5)	86 / 75	24.92 / 24.34	32.56 / 31.40	29.98 / 19.70	17 / 10	25 / 24
RefreshKV (QC=10)	80 / 67	24.73 / 23.98	31.47 / 31.36	29.37 / 18.94	8 / 6	15 / 15

Table 4.2: Downstream task performance. In each cell, the first number represents the performance of Llama-3.1 model and the second number for QWEN-2 model. *We report performance of Llama-3.1-70B and Qwen-2-72B for the chain-of-key task, as the smaller variants cannot perform the task even in vanilla setting.

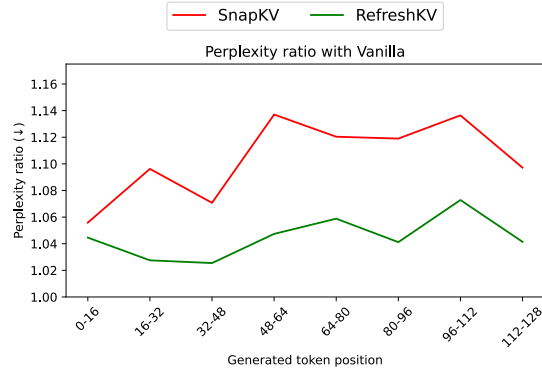


Figure 4.3: We plot the perplexity ratio against the vanilla baseline for RefreshKV (with stride of 10) and SnapKV based on the tokens generated (x axis). While the ratio is similar at the beginning of the sequence, as the generation goes SnapKV’s perplexity diverges from vanilla approach while that of RefreshKV is relatively stable.

on the book dataset. While both methods begin with a similar perplexity ratio compared to vanilla (step 0-16), SnapKV’s performance degrades as generation proceeds, whereas RefreshKV maintains a stable ratio, highlighting the benefit of refreshing the small KV cache during generation.

4.4.2 DOWNSTREAM TASKS

Results for downstream tasks are reported in Table 4.2. We also report the average input and output length for each dataset. For RULER, we report results aggregated over 13 tasks here and report the per-task performance in Table D9 in the Appendix.

EVICTION-BASED METHODS FAIL FOR LONG-FORM GENERATION TASKS. Baseline methods that evict tokens from the KV cache permanently (StreamingLLM, H₂O and SnapKV) show degradation for tasks that require long-form outputs. While SnapKV performs better than the other two baselines on RULER, it shows severe performance degradation on the HTML to TSV task, achieving 0 F-1 scores for the former. For the Chain of key task, eviction-based methods are unable to generate a chain with more than two keys, achieving accuracy < 20.

REFRESHKV CLOSES THE GAP BETWEEN VANILLA AND EVICTION-BASED APPROACH. On HTML to TSV task, RefreshKV with $QC = 5$ recovers 52% and 42% of performance for Llama-3.1-8B and Qwen2-7B respectively. On the **Chain-of-key** task, RefreshKV is the only method that is able to generate a valid key with length longer than two keys, as shown in Figure 4.1. For the long-form summarization tasks, RefreshKV outperforms baselines in all three datasets, except for NovelSumm with Qwen2-7B, where StreamingLLM outperforms the vanilla full attention.⁶ We also observe gains for RULER tasks, particularly the subtasks that require generating longer output (e.g. generating multiple keys), which we discuss in Section 8.0.13 in the Appendix.

⁶We hypothesize that this might be due to the fact that Qwen is pre-trained on 32K context and adopts YARN and Dual Chunk Attention to enable processing of up to 128K tokens. As NovelSumm contains > 100K tokens, it exceeds the pre-training context window, which might cause the performance difference between StreamingLLM (which uses local window) and other methods that leverage attention scores over full context (including vanilla).

Schedule	Book			HTML (0.5K)			GovReport		
	Time ↓	Stride	PPL ↓	Time ↓	Stride	Acc ↑	Time ↓	Stride	R-L ↑
<i>Llama-3.1-8B</i>									
Vanilla	7.50	-	7.07	1.52	-	43	1.43	-	34.11
Fixed	7.20	10	7.40	1.40	10	17	1.37	10	32.30
Dynamic (QC=5, s=0.85)	7.17	12	7.31	1.40	14	30	1.38	14	32.56
Fixed	6.99	15	7.45	1.37	15	8	1.34	15	30.67
Dynamic (QC=10, s=0.85)	6.89	17	7.41	1.33	19	16	1.34	19	31.47
<i>Qwen-2-7B</i>									
Vanilla	9.07	-	8.26	1.96	-	35	1.73	-	33.38
Fixed	6.59	10	8.74	1.38	10	8	1.29	10	31.18
Dynamic (QC=5, s=0.95)	6.71	7	8.55	1.29	7	20	1.34	7	31.40
Fixed	6.43	15	8.81	1.31	15	9	1.27	15	30.73
Dynamic (QC=10, s=0.95)	6.33	11	8.72	1.23	12	14	1.28	12	31.36

Table 4.3: Results comparing fixed stride and dynamic stride based on query similarity. In all tasks, dynamic stride shows better task performance while performing full attention step fewer times.

Method	Stride	Arxiv	HTML (0.5K)
Vanilla	-	2.22	43
SnapKV	-	2.54	0
RefreshKV	10	2.32	16
- w/o refresh	10	2.50	0
- w/o full attention	10	2.32	16

Table 4.4: Ablation study on LLama-3.1-8B. We report perplexity for Arxiv and F-1 score for the HTML to TSV task.

4.5 ABLATION STUDIES

4.5.1 ADAPTIVE STRIDE VS. FIXED STRIDE

We trigger full attention step when the query vector of the input token is substantially different from the query vector of the most recent full attention step. Can we use a simpler strategy to decide when to perform full attention? In this section, we explore refreshing at a fixed stride, performing full attention every N -th step across all the layers.

SETTING We compare the results of (1) employing a dynamic stride with the set-up in Section 6.5, i.e. QC stride of $\{5, 10\}$ and similarity threshold $s = 0.85$ (Llama-3.1-8B) and $s = 0.95$ (Qwen2-7B) and (2) employing a fixed stride S of $\{10, 15\}$ for comparable decoding time. We report results on the language modeling task on the Book dataset and two downstream tasks. We report the decoding time measured on one A100 machine. For the language modeling task, we report the time for generating 256 tokens. For the downstream tasks, we measure the time of generating the first 50 tokens. We also report the *effective* stride averaged across all the layers, i.e. how often is full attention performed when employing dynamic strides.

RESULTS Table 4.3 presents the results. For Llama-3.1-8B, comparing $QC = 5$ and $S = 10$, employing dynamic stride consistently achieves better performance with similar or less decoding time for all three tasks. We see a similar trend comparing $QC = 10$ and $S = 15$. For Qwen2-7B, dynamic stride achieves better performance across all three tasks, with slightly more decoding time on Govreport. We also observe slightly different effective stride for different tasks when employing the same QC and s , showing that dynamic stride enable flexible scheduling based on the context. We report per-layer stride in Section 8.0.10 in the Appendix.

4.5.2 IMPACT OF FULL ATTENTION STEPS

Compared to other baseline methods which never perform full attention during the generation, RefreshKV involves extra attention calculation (i.e. attending over the entire output). To tease apart the performance gains from occasional full attention step and updating the small KV, we present two ablation setting for RefreshKV: (1) *w/o refresh* which performs attention over the full KV cache at the fixed stride of S but without refreshing the partial cache. This is equivalently using the partial cache obtained with SnapKV and occasionally performing full attention. (2) *w/o full attention* which calculates the attention scores over the entire KV cache and updates the partial cache, then attends to the updated partial cache, instead of attending to the full KV cache,

at stride S .

Results are in Table 4.4. While performing occasional full attention (**w/o refresh**) improve perplexity slightly compared to SnapKV, the performance lags behind RefreshKV. In contrast, the ablation setting where partial cache is refreshed (**w/o full attention**) achieves the same performance of RefreshKV for both tasks. This shows that the gain of RefreshKV mostly comes from refreshing the partial KV cache, instead of performing occasional full attention over the entire cache.

4.6 CONTINUED PRE-TRAINING WITH REFRESHKV

We have demonstrated **RefreshKV** can be used as an inference-time method. However, since the LLMs we study are trained with full attention, applying RefreshKV during inference introduces a discrepancy between training and inference. Specifically, it involves attending to a non-contiguous sequence of tokens in the partial cache. Here, we explore continued pretraining with RefreshKV to adapt models to this new attention pattern.

To make training setting simpler, we do not fully implement RefreshKV during training. We use a fixed stride of 50 and never refresh the partial cache. We assume a length $L + S$ for all sequences, where L is the pre-fill length. We perform standard attention over all past tokens for the first L tokens. We emulate the partial attention pattern for the last S tokens in the sequence during training. For the next S tokens, we perform attention over the top K tokens identified as well as local tokens (i.e. tokens $L + 1$ onwards). We train the model with next token prediction loss for all the tokens in the sequence.

SETUP We set $L = 8092$, $S = 50$ and $K = 2048$ for this experiment. We randomly sample a subset of 200k sequences from the Arxiv split of RedPajama dataset. We split the data into 80%, 10% and 10% train/dev/test splits, resulting in 120k training data samples. We perform continued

Method	Stride	Test PPL (8K)	Test PPL (16K)
Vanilla	-	2.70 $\rightarrow$ 2.70	2.50 $\rightarrow$ 2.50
Streaming	-	3.40 $\rightarrow$ 3.38	3.50 $\rightarrow$ 3.49
H ₂ O	-	3.95 $\rightarrow$ 3.90	3.52 $\rightarrow$ 3.49
SnapKV	-	3.21 $\rightarrow$ 3.15	2.98 $\rightarrow$ 2.92
RefreshKV	10	2.83 $\rightarrow$ 2.79	2.57 $\rightarrow$ 2.56
RefreshKV	25	2.97 $\rightarrow$ 2.93	2.67 $\rightarrow$ 2.63
RefreshKV	50	3.13 $\rightarrow$ 3.05	2.79 $\rightarrow$ 2.72

Table 4.5: Results on continued pre-training with RefreshKV for LLaMA-3.1. The context size is 8k and we report perplexity on the last 50 tokens. We report the performance for each setting before (the number on the left) and after (the number on the right) CPT.

pre-training on Llama-3.1-8B and describe implementation details in Section 8.0.6 in the Appendix.

EVALUATION As our continued pretraining is relatively small scale on the base model, we focus on evaluating on the language modeling task for two settings: (1) input size $L=8K$ consistent with the training set-up and (2) $L=16K$. We set $K=1/8L$. For each method, we report the performance from the pre-trained checkpoint and the performance after continued pre-training.

RESULTS We report the results in Table 4.5, each row represents a different inference strategy on the same model. Despite the mismatch in how partial KV was constructed, continued pre-training benefits other methods (Streaming, H₂O) slightly. We see larger gain for RefreshKV from continued pre-training across all settings. Our training assumes a fixed stride of 50, but we see performance gain for different strides ($S=10, 25$). Training on shorter context (8K) also translates to gains when inferencing on longer context (16K), showing promise for improving the performance of RefreshKV with continued pre-training.

4.7 CONCLUSION

We propose **RefreshKV**, an inference-time method which accelerates long-form generation for long-context input by decoding from a small, dynamic KV cache that is updated based on attention

patterns of neighboring tokens. Compared to previous work which permanently evict tokens from the context, **RefreshKV** maintains the full KV cache and alternates between inferencing over the full and small KV cache. We apply our method to two off-the-shelf long-context models and show that our method reduces inference wall-clock time while better preserving performance compared to eviction-based methods on long-form generation tasks. Finally, we show that continued pre-training the model with **RefreshKV** can further improve the performance-efficiency trade-off.

LIMITATIONS

PROPOSED METHOD While we focus on accelerating inference speed, our method does not reduce memory requirement for using long-context LLMs, which can be a bottleneck for certain use cases. Our objective is to accelerate decoding for long-context models. While our method outperforms eviction-based approaches, it still involves a trade-off between performance and efficiency. In this study, we employ query similarity based dynamic scheduling to decide when to perform full attention and refresh the small KV cache. Future work can explore other strategy, such as more exhaustively tuning the similarity threshold, or setting a different threshold per layer.

EXPERIMENTAL SETTINGS We have conducted experiment with two open-sourced long-context models and two evaluation settings. We did not test out more language models and other long-context benchmarks [An et al. 2023; Karpinska et al. 2024] given our limited compute resources. For the same reason, our experiment on continued pre-training is relatively small scale on a limited domain. We have demonstrated the effectiveness of refreshing a small KV cache constructed with attention scores and use the same size across different layers. Future work can extend our method to refresh smaller cache constructed with different strategy, e.g. layer-specific strategies [Cai et al. 2024; Yang et al. 2024b]. Finally, our method is not limited to the language domain. Future work can explore applying RefreshKV to other modalities, for example, vision transformers.

5 | A CRITICAL EVALUATION OF EVALUATIONS FOR LONG-FORM QUESTION ANSWERING

The previous two chapters address the first challenge raised in the introduction: making inference more efficient as context lengths grow. We now turn to the second challenge, measuring whether models are actually doing well. As argued in [Chapter 1](#), the assumptions underlying reference-based evaluation hold for short-form tasks with enumerable answers but break down for open-ended generation, where many distinct outputs may be equally valid. This chapter examines one representative case: long-form question answering, where models produce paragraph-length responses to questions whose space of valid answers is large and ill-defined. Through a detailed analysis of both automatic and human evaluation protocols used in prior work, we identify substantial flaws and find that even expert annotators frequently disagree about which of two answers is better, in ways that reflect genuine differences in what they value rather than annotation error. This chapter is based on work with Yixiao Song, Mohit Iyyer, and Eunsol Choi [[Xu et al. 2023](#)].

5.1 INTRODUCTION

Long-form question answering [Fan et al. 2019b; Krishna et al. 2021; Nakano et al. 2021; Su et al. 2022, henceforth LFQA], an emerging research area within QA, requires systems to *generate* long and complex answers to questions by leveraging large language models and evidence document retrievers. While remarkable strides have been made in LFQA model development, the current state of LFQA *evaluation* is dire: most prior papers use a combination of crowdsourced human annotations and simple string-matching metrics (e.g., ROUGE). We present the first study of the evaluation of long-form answers, exploring both human and automatic evaluation protocols to better understand how we should evaluate LFQA moving forward.

Human evaluation: In most prior human LFQA evaluations [Krishna et al. 2021; Nakano et al. 2021], crowd annotators are given a question, two candidate answers, and (optionally) evidence documents, and they are asked to identify the better answer. However, crowdworkers do not necessarily have the expertise or background knowledge to reliably judge properties such as factuality [Gillick and Liu 2010; Iskender et al. 2020]. Thus, we hire *domain experts* in seven different fields (e.g., biology, economics) to perform the same answer preference task and additionally provide detailed justifications as to why they chose a particular answer. Analyzing their justifications reveals that experts consider properties such as completeness and factuality to be more decisive than surface-level aspects (e.g., conciseness and level of detail) on which crowdworkers tend to fixate. Additionally, even experts often disagree with each other about which answer is better; this disagreement stems from valuing fine-grained answer properties differently.

Automatic evaluation: As human evaluation is slow and expensive, developing a reliable automatic LFQA evaluation metric is crucial for speeding up model development. While ROUGE [Lin 2004] has been shown to be misleading for LFQA [Krishna et al. 2021; Wang et al. 2022], do any other existing text generation metrics correlate to human judgments of answer quality? Can

Aspect	Question	Answer A	Answer B	Expert Justification
Factuality	Does your body absorb all blood clots? How does it do it? Does location affect it?	(human) Kind of. Your blood will dissolve clots over time. I've got a clotting disorder and need to take thinners to make that happen faster , but for most they break down clots naturally [...]	(model) No, your body does not absorb all blood clots. Clots are usually broken down by the body's natural clotting mechanism, but some clots may persist and cause problems. Location can affect how quickly a clot is broken down [...]	(expert) [...] Answer A mentions some inaccurate and irrelevant information, like role of blood thinner which is not correct as they cannot break down the clots. [...]
Completeness	Why do people go bald on the top of their head and still have some on the sides and not vice-versa?	(model) One reason is that the hair follicles on the top of the head are more sensitive to a hormone called dihydrotestosterone (DHT). DHT is [...] found in both men and women. [...] Another reason is that the hair on the sides and back of the head is not as exposed to the sun and other environmental factors, [...]	(human) Seems unsettled but here's a theory: The most recent hypothesis suggests that the hair loss process begins during puberty, when growth of the skull and the muscles in the forehead and neck increases the tension in a tight band of tissue stretching over the top of the head. The more DHT (a type of testosterone) there is...	(expert) Answer A is the better choice as it describes both the hormonal and environmental causes and Answer B only focuses on one theory which might not be 100 percent accurate. [...] According to research, baldness is due to genes. In 95 percent cases, balding is due to androgenetic alopecia [...]

Table 5.1: Examples of two fine-grained aspects, **factuality** (top) and **completeness** (bottom), that were decisive factors in our expert annotators’ preference of one answer over another. The *human* answers are from the [r/explainlikeimfive](#) subreddit and the *model* answers are generated zero-shot by text-davinci-002. See Table A5 for more examples.

we train a metric to mimic human preference judgments? To answer these questions, we curate a suite of 12 automatic metrics and measure how they correlate to human judgments of both “overall quality” and two fine-grained aspects (coherence and faithfulness). None of these metrics reliably matches human judgments of overall answer quality. However, automatic metrics such as QAFactEval [Fabbri et al. 2022] and RankGen [Krishna et al. 2022] show potential at modeling fine-grained aspects of LFQA answers, which can spur research on a new generation of automatic LFQA metrics.

Overall, we provide the first thorough study of LFQA evaluation and shed light on the components of good long-form answers. As part of our exploration, we collected and will release a small-scale dataset of expert evaluation of long-form answers (260 ratings and justifications over 140 answer pairs). We conclude by providing recommendations for the future of human and automatic LFQA evaluation, encouraging the community to hire expert evaluators and move from poorly-defined judgments of “overall preference” to a multi-faceted evaluation modeling

attributes such as answer completeness, factuality, and ease of understanding.

5.2 BACKGROUND AND RELATED WORK

We begin by reviewing the evaluation protocols used by prior work in LFQA, which has centered around a dataset scraped from the “Explain Like I’m Five” subreddit [Fan et al. 2019b, ELI5].¹

PRIOR AUTOMATIC EVALUATIONS: Early work on LFQA [b] uses ROUGE [Lin 2004] to measure the similarity of human reference answers to model-generated answers. Krishna et al. [2021] find that ROUGE is not a meaningful metric due to the open-ended nature of long-form answers, but they do not examine other automatic metrics. Given the difficulty of evaluation, recent works re-scoped the task to allow more reliable evaluation: Wang et al. [2022] focus on exemplification in long-form answers by treating this sub-task as a retrieval problem, while Stelmakh et al. [2022] aim to evaluate long form answers limited to ambiguous factoid questions that cover the different disambiguated questions and their corresponding answers. However, these evaluation protocols cannot be easily adapted to the general LFQA task: the metric in Stelmakh et al. [], for example, requires a list of disambiguated questions and their answers, which is not available for many questions.

PRIOR HUMAN EVALUATIONS: We summarize the human evaluation studies conducted by two previous studies, HURDLES [Krishna et al. 2021] and WEBGPT [Nakano et al. 2021]. Both works evaluate via A/B testing (i.e., choose which of two candidate answers is better), and they collected judgments of overall answer quality, factuality, and coherence. While both works recruited non-expert annotators and collect only one-way annotations, WEBGPT’s evaluation allows annotators to look at a set of evidence documents when judging the answer, and they also collect optional

¹<https://www.reddit.com/r/explainlikeimfive>

Category (# of experts)	Preference		Fleiss' κ
	Upvote $\uparrow$ (H/H)	Model (H/M)	
Biology (3)	76.7%	53.3%	0.52
Physics (2)	50%	65%	0.50
Chemistry (1)	70%	50%	–
Economics (2)	60%	90%	0.40
Law (1)	60%	90%	–
Tech/CS (1)	40%	60%	–
History (3)	80%	24.4%	0.65
Average	62.4%	61.8%	–

Table 5.2: Results of our expert annotation of seven domains, where the two candidate answers are either both human-written (H/H) or human-written vs. model-generated (H/M). We report how often the highly-upvoted answer was preferred in H/H, and how often the model-generated answers are preferred in H/M.

free-form justifications from the annotators to justify their choice. While fine-grained aspects such as coherence [Goyal et al. 2022b; Jiang et al. 2022] and factuality [Goyal and Durrett 2020; Laban et al. 2022] have been studied before for other tasks such as summarization, ours is among the first works to study LFQA-centric properties such as completeness or ease of understanding.

5.3 HOW DO DOMAIN EXPERTS EVALUATE LONG-FORM ANSWERS?

Prior LFQA human evaluations use non-expert crowdworkers to evaluate highly domain-specific answers, either with no access to external information [Krishna et al. 2021] or access to only model-retrieved evidence documents [Nakano et al. 2021]. Both settings are problematic: non-experts cannot be relied on to judge the correctness of answers in isolation, and they also cannot be expected to thoroughly comprehend evidence documents and judge their validity or relevance to the answer [Gao et al. 2022a]. While Nakano et al. [2021] solicit optional free-form justifications from their workers to explain their preference judgments, it remains unclear how well these workers can judge *correctness* in fields that are not their expertise. Our first contribution is to hire *domain experts* in seven fields (see Table 5.2) and have them evaluate both human-written and

model-generated answers via A/B judgments as well as paragraph-length free-form justifications. An analysis of the expert annotations reveals a complex and subjective interplay between many different fine-grained aspects of LFQA answers (e.g., completeness, factuality) that pose challenges for future LFQA evaluation.

5.3.1 COLLECTING EXPERT JUDGMENTS

HIRING EXPERTS: We recruit domain experts on the freelancing platform Upwork for seven domains shown in Table 5.2. Each expert has earned at least a bachelor’s degree in the target domain and has expertise performing tasks in that domain (e.g., summarizing scientific articles or being a teacher of the domain). As shown in Table 5.2, we hire 1-3 experts per domain. Given a question and two candidate answers, the experts were asked to choose which of the answers is better (*overall preference*), indicate whether the decision was difficult to make (e.g., because both answers were of similar quality), and lastly to justify their choice in a free-form paragraph. The evaluation tasks are hosted on Label Studio.² The experts reported that they spent 15 to 30 minutes per question, which shows the demanding nature of the annotation task. We accordingly paid \$3.25 per question, which resulted in a total cost of \$845 to collect 260 expert judgements.³

SETTING UP THE A/B TASK: Following prior work, we conduct A/B preference testing on two answers to the same question. We include two settings: (1) H/M: comparing a model-generated answer with a highly-upvoted human-written answer, and (2) H/H: comparing a highly-upvoted human-written answer to an answer with fewer upvotes (where upvotes are a noisy proxy to answer quality).⁴ The first setting is intended to identify common classes of errors made by state-of-the-art LFQA systems, while the second setting is more of a sanity check exploring whether low-effort human answers make similar errors to models.

²Figure B2 contains a screenshot of our annotation interface. <https://labelstud.io/>

³We explain in Appendix 8.0.14 why the numbers of experts in each domain differ.

⁴Reddit users give upvotes to content to show their support or approval of the content.

We chose GPT-3 text-davinci-002 model (175B) [Brown et al. 2020b] as the LFQA model to evaluate. A small-scale qualitative analysis found that zero-shot GPT-3 possesses more advanced LFQA capabilities than fine-tuned LFQA systems built on smaller language models. Since this model may have already seen the entire ELI5 dataset released by Fan et al. [2019b] during its pretraining, we scrape more recent questions from the [r/explainlikeimfive](#) and [r/AskHistorians](#) subreddits posted between July to December 2021.⁵ Question askers on the ELI5 subreddit often categorize their questions into domains via the `flair` label, which enables us to perform a domain-specific analysis.⁶ We randomly sample 20 questions per domain except for the history domain, which has 15 questions in the H/M setting and 5 in H/H. This discrepancy is due to the difficulty of finding history questions with a moderate answer length. As shown in Figure 5.1 and Table A1, human-written answers to history questions are much longer than the answers in the other domains, even after careful screening.

To obtain model-generated answers, we prompt the model in a zero-shot manner with the following prompt: “Generate a long answer to the following question with examples and references when necessary.” For decoding, we used the default decoding setup in the API (i.e., top $p = 1$ and temperature= 0.7).

EXPERT (DIS)AGREEMENT: We report Fleiss’ κ [Fleiss 1971; Fleiss et al. 2013; Landis and Koch 1977] as a measure of agreement in Table 5.2. Our expert A/B testers achieved fair agreement in economics, moderate agreement in biology and physics, and a substantial agreement in history. We observe that agreement increases when comparing a high and low-upvoted human answer together, as opposed to comparing model-generated answers with human answers. We emphasize that disagreement is *not* a failure of one of the experts to properly evaluate the answers. In fact, disagreement within experts highlights the challenges (and futility) of judging “overall answer quality” in this way. There are many salient properties of long-form answers, which we discuss

⁵text-davinci-002 was trained on data up to June 2021.

⁶The details on domain identification are in Appendix 8.0.14.

next, and deciding how to value each property when coming up with an overall preference is highly subjective (see Appendix Table A4 for several examples).

5.3.2 QUANTITATIVE RESULTS

As shown in Table 5.2, experts surprisingly display a slight preference (61.8%) for *model-generated* answers from GPT-3 compared to human answers; as a sanity check, they exhibit preference (62.4%) for highly-upvoted human answers over those with fewer upvotes. The preference of our annotators for model-generated answers is corroborated by similar findings for summarization by Liu et al. [2022], who show that GPT-3 generated summaries score higher than reference summaries.

Comparing different domains, we observe that model-generated answers are strongly preferred in economics (90%) and law (also 90%), while human answers are preferred in the history domain (75.6%). To understand the divergence in preferences for different domains, we report the answer length distribution of both answer types in the H/M setting in our expert-annotated dataset in Figure 5.1. The model’s struggles in history domain are likely because this domain contains the longest and most complex questions as well as human answers (averaging 356 words long in the H/M setting) out of all domains. Table A1 in the appendix report the length of questions, model-generated, and human-written answers of the whole expert-annotated dataset.

5.3.3 WHAT MAKES ONE ANSWER BETTER THAN ANOTHER?

To better understand the various components of a good long-form answer, we perform an analysis on the free-form justifications collected from both our expert annotators as well as WEBGPT crowd annotators from Nakano et al. [2021]. WEBGPT allowed *optional* justifications, and many of them are not very long or detailed. Our justification is about three times longer on average (statistics can be found in Table A2 in the Appendix). Our analysis focuses on the

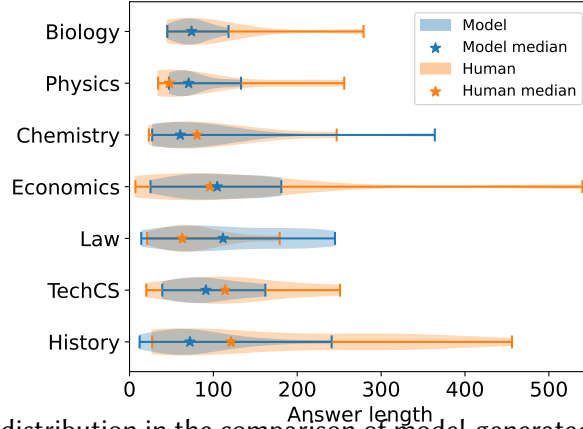


Figure 5.1: Answer length distribution in the comparison of model-generated and human-written answers (H/M) in our expert-annotated dataset. History is the hardest domain for models and also has the largest discrepancy between model and human answer length. There are 75 questions and 75 human-written and model-generated answers.

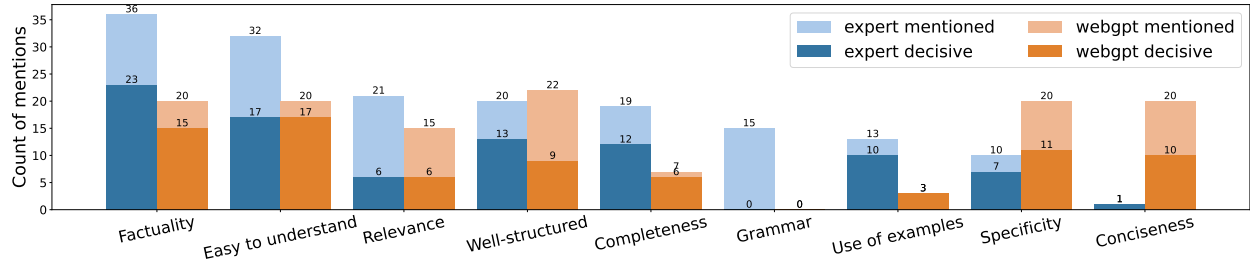


Figure 5.2: We manually analyzed 50 justifications each from both experts and WEBGPT crowd annotators. We report nine frequently-mentioned fine-grained aspects here. The plot shows that experts and crowdworkers disagree on which aspects are more decisive, and that experts are more sensitive to factuality and completeness.

model-generated vs. human-written answer setting, where the model is either zero-shot GPT-3 (our work) or the 175B WEBGPT model. Concretely, we analyze 50 randomly sampled justifications from each population. Our analysis is limited in that these two comparisons do not consider the same set of questions. We identify and code nine fine-grained aspects that are mentioned in them, and mark whether these aspects are decisive factors for making the preference judgment. The results are summarized in Figure 5.2, and we highlight takeaways below.

EXPERTS ARE BETTER JUDGES OF FACTUALITY: Perhaps unsurprisingly, our experts mention **factuality** in their justifications almost twice as frequently as crowdworkers (36 to 20), and it is

the most common aspect referenced by experts. As an example, in the first row of Table 5.1, the expert accurately points out incorrect information in Answer A about blood thinners breaking up clots. Since WEBGPT annotators lack domain expertise, they generally judge factuality by checking if a statement is supported in evidence documents, which gives them only limited coverage over the full answer.

EXPERTS VALUE ANSWER COMPLETENESS: We observe that experts mention **completeness** as a decisive criteria twice as often than WEBGPT annotators (12 vs. 6). Completeness refers to whether the answer adequately addresses all aspects of the question or provides all necessary information to clarify the question. Judging completeness requires deeper domain expertise than a handful of retrieved articles offer. As an example, in the second row of Table 5.1, the expert states that Answer B mentions only one reason why people go bald (hormonal), while Answer A mentions hormonal and environmental factors and is thus superior.⁷

ALL ANNOTATORS VALUE EASE OF UNDERSTANDING. Both experts and crowdworkers mention **easiness to follow** as a decisive criterion at the same frequency; in fact, this is the most decisive aspect for both populations. One of the main goals of LFQA is to convey the answer of a question to a non-expert; as such, it makes sense that this property is so critical. We emphasize that this has *never* been evaluated in prior LFQA research and encourage future work to embrace it as a major component.

NON-EXPERTS FOCUS ON SURFACE-LEVEL PROPERTIES: WEBGPT annotators are far more likely to mark **conciseness** and **specificity** as decisive factors for their preferences than experts. They prefer shorter to-the-point answers, despite the fact that such answers might be incomplete, and they also prefer answers that include specific details instead of generalities. We note that these properties are much more feasible to judge for crowdworkers than factuality and completeness,

⁷The expert further points out that both answers miss a third major cause of baldness: genetics.

Data	Model	Token level EM		
		O $\uparrow$	F $\downarrow$	R
Expert	T5-base	0.36	0.37	0.33
	T5-large	0.51	0.44	0.41
	T5-3B	0.66	0.36	0.48
	T5-11B	0.76	0.28	0.47
WEBGPT	T5-base	0.40	0.38	0.37
	T5-large	0.50	0.49	0.50
	T5-3B	0.60	0.46	0.53
	T5-11B	0.65	0.40	0.54

Table 5.3: Results on masked justification reference prediction: **O**riginal comments, **F**lipped comments and **R**andom comments. The larger LMs can identify references in justifications better.

which is likely a reason why they are mentioned so frequently (Table A5 in the appendix for examples).

5.3.3.1 DO MODELS UNDERSTAND JUSTIFICATIONS OF HUMAN PREFERENCES?

Our manual analysis of the justifications shows that experts consider a wide range of aspects when forming their decision. Detailed justifications of generated answers are useful in understanding why an answer was preferred, but they are costly to obtain. Generating these justifications automatically and evaluating them is outside the scope of this paper. Instead, we perform a simpler evaluation via a proxy task: given a justification with masked references to both candidate answers, can a model disambiguate the missing references? An example of the task is below:

Input: Question: q Answer A: a_1 Answer B: a_2 Comment: Both answers are coherent, but Answer <extra_id_0> is completely irrelevant to the question since it is about a bionic ear instead of a person learning speech when they get a hearing implant. Answer <extra_id_1> is relevant and a complete, concise answer.

Expected Output: <extra_id_0> B <extra_id_1> A

We experiment with pretrained T5 checkpoints [Raffel et al. 2020b] of different sizes (220M,

770M, 3B, and 11B parameters) on our task zero-shot.⁸ For each (question q , answer pairs (a_1, a_2) , justification j), we construct three types of inputs: **Original**: The original justification j with (q, a_1, a_2) , **Flipped**: The original justification j with flipped answer identity (q, a_2, a_1) , **Random**: j with randomly paired q', a'_1, a'_2 , as a baseline. We evaluate using token-level exact match, which gives the model credit only when its output exactly matches that of the target. We expect better than random performance on **Original** and worse than random performance on **Flipped** if the model comprehends the justifications.

Results are shown in Table 5.3. We see that T5-3B and T5-11B are able to comprehend the justifications, as they show different results for original and perturbed comments. This suggests adapting LMs for multi-faceted automatic evaluations of long-form answers is promising. Preprocessing details on this study are described in Appendix 8.0.14.1

5.4 DO AUTOMATIC METRICS CORRELATE WITH HUMAN JUDGMENTS?

The experiments in the previous section establish that LFQA is very difficult for humans to converge on in terms of an “overall” score, as even domain experts disagree with each other when choosing a “better” LFQA answer. Furthermore, several properties of these answers are important to evaluate, including factuality, relevance, and coherence, among others. Do existing automatic text generation metrics correlate with human judgments of these fine-grained aspects, or “overall” answer preference? We now explore this question with a wide range of text generation evaluation metrics.

5.4.1 TEXT GENERATION METRICS

We experiment with existing text generation metrics and metrics that we train directly on the human preference judgments.

⁸We experimented with two-shot prompting with GPT-3 but observed worse results compared to the outputs from T5-3B and T5-11B, potentially because the task resembles the pretraining setup of T5.

5.4.1.1 GENERAL-PURPOSE GENERATION METRICS

Prior work used existing text generation metrics (e.g., ROUGE) to evaluate LFQA. The metrics were initially designed for other text generation tasks (e.g., translation or summarization), and their usage has not been validated for LFQA.

REFERENCE-BASED METRICS: Many generation metrics assume access to human-written references (in our case, gold answers), which are used to compute similarity scores to model-generated text. Of these, we evaluate **ROUGE** [Lin 2004], which is the only reference-based evaluation metrics employed by prior work for LFQA, as well as **BERTScore** [Zhang et al. 2019] and **BLEURT** [Sellam et al. 2020], which leverage pretrained language models and have shown to be effective in evaluating many generation tasks [Kasai et al. 2022]. A major limitation of reference-based metrics for LFQA is the huge space of valid output answers for any given question, which has been noted in prior work [Wang et al. 2022].

ANSWER-ONLY METRICS: Some aspects, such as fluency and coherence, can be determined by looking at just the answers alone. Thus, we also examine a set of answer-only automatic metrics: (1) **Self-BLEU** [Zhu et al. 2018], which measures the diversity of generated text (higher scores mean lower diversity) and has been previously used in open-ended generation [Holtzman et al. 2019]; and (2) **GPT-2 perplexity**, which prior work on constrained generation [Qin et al. 2022; Zhang et al. 2020] has used to evaluate fluency.

(QUESTION, ANSWER) METRICS: Good answers should be *relevant* to the question asked, so we can model $p(q|a)$ to rank answers using the following methods: (1) **Zero-shot question likelihood**, which uses the instruction-tuned T0 model [Sanh et al. 2022] to calculate the likelihood of the question given the long-form answer; (2) **BARTScore** [Yuan et al. 2021], which is an encoder-decoder model fine-tuned on text summarization; and (3) **RankGen** [Krishna et al. 2022], which is

an encoder model trained contrastively to score model-generated sequences (in our case, answers) given a prefix (the question).

(ANSWER, EVIDENCE) METRICS: Arguably the most challenging aspect of LFQA evaluation is to measure the correctness of the answer. While there are no existing factuality metrics for LFQA, the task is related to faithfulness in summarization. Metrics for faithfulness assume access to a set of evidence documents and evaluate whether a text is supported by the evidence [Barrantes et al. 2020; Goyal and Durrett 2020; Kryscinski et al. 2020; Laban et al. 2022]. We experiment with the **QAFactEval** metric [Fabbri et al. 2022], which evaluates faithfulness by comparing answers from the summary (in our case, the answer) and the evidence document (retrievals from the WEBGPT LFQA system).

5.4.1.2 TRAINED LFQA METRICS

The metrics discussed so far are not trained on long-form answers. We now shift to training an LFQA evaluation metric directly on human-annotated preference judgments of pairs of long-form answers. Prior work from OpenAI [Nakano et al. 2021] experimented with learning an evaluation metric by fine-tuning WEBGPT to rank pairs of answers. As this model is not publicly available, we fine-tune a smaller-scale pretrained language model (176M Longformer-Base model) and rely on OpenAI’s API to fine-tune bigger pretrained language model (6B GPT3 text-curie-001 model⁹). Details of fine-tuning setup are in Appendix 8.0.15.1.

DATA We use comparison data collected by Nakano et al. [] for fine-tuning, which contains 17,598 preference annotations. We remove ties and randomly split the data into train, validation and test sets with a 70%, 15%, 15% ratio. More details are provided in Appendix Table A7.

⁹To the best of our knowledge, OpenAI has not clarified the exact size of each of the models in the API. We use this estimation: <https://blog.eleuther.ai/gpt3-model-sizes/>.

FINE-TUNING LONGFORMER Our learned metric f takes in question q , answer a , and optionally evidence documents d to produce a scalar score. We encode $[q, a]$ and $[a, d]$ separately with an encoder model and concatenate respective [CLS] representation then pass it to a linear layer to obtain a scalar score s . As our input text is relatively long, we fine-tune a Longformer encoder [Beltagy et al. 2020b].

Following Nakano et al. [2021], we train the model with cross-entropy loss such that the scores produced by f rank a pair of answers (a_1, a_2) in the same order as the human preference. We estimate the likelihood that a_1 is preferred over a_2 as $\frac{\exp(s_1)}{\exp(s_1) + \exp(s_2)}$ where $s_1 = f(q, a_1)$, $s_2 = f(q, a_2)$. Given a set of answer pairs with gold preference $\hat{p}$, the loss is,

$$L = -(\mathbb{1}[\hat{p} = a_1] \log P(p = a_1) + \mathbb{1}[\hat{p} = a_2] \log P(p = a_2)),$$

where $\mathbb{1}$ is the indicator function. We consider two inference settings, **longformer(D)**, which considers evidence documents, and **longformer** which takes the concatenation of $[q, a]$ and $[a]$, as evidence documents are not always available.

FINE-TUNING GPT-3 To leverage the advanced capabilities of larger-scale language models, we use OpenAI API to finetune GPT-3 text-curie-001 with the same comparison data split we used for the Longformer. Given a prompt consisting of question q , answer a_1 and answer a_2 , the model is fine-tuned to output the label Answer1 or Answer2. This metric takes a *pair* of answers as input and outputs a preference, unlike the Longformer model which produces a score given a single answer.

5.4.2 EVALUATING AUTOMATIC METRICS

TASK Each evaluation example consists of $\{(q, a_1, a_2, \hat{p})\}$, where q is question, a pair of long-form answers a_1 and a_2 , and $\hat{p} \in \{a_1, a_2\}$ denotes the human preference of choosing answer a_1 or a_2 . We report the accuracy of the metric preference p_i against the gold human preference $\hat{p}_i$. We omit the

Data source	Expert	Overall				Coherence			Factuality		
		WEBGPT		HURDLES		WEBGPT		HURDLES		WEBGPT	
Setting		h/m	m/m	h/m	m/m	h/m	h/m	m/m	h/m	h/m	m/m
# pairs	129	637	1,923	419	370	496	164	194	149	151	169
Baselines											
Random	0.50	0.50	0.49	0.50	0.48	0.50	0.51	0.50	0.50	0.50	0.49
Always Human	-	0.61	-	0.81	-	0.70	0.87	-	0.52	0.95	-
Length	0.68	0.52	0.57	0.61	0.48	0.38	0.62	0.49	0.57	0.68	0.57
Reference-based metrics											
ROUGE	0.58 [†]	0.53	0.53	0.43	0.52	0.54	0.46	0.48	0.46	0.40	0.51
BERTScore	0.57 [†]	0.57	0.51	0.46	0.61	0.62	0.39	0.69	0.48	0.39	0.61
BLEURT	0.62 [†]	0.52	0.54	0.42	0.56	0.55	0.32	0.45	0.52	0.33	0.53
Answer-only metrics											
Self-bleu	0.36	0.50	0.45	0.57	0.48	0.59	0.64	0.61	0.49	0.62	0.47
GPT2-PPL	0.60	0.48	0.51	0.28	0.52	0.46	0.21	0.34	0.47	0.19	0.44
(Question, answer) metrics											
QG	0.63	0.58	0.51	0.60	0.61	0.56	0.59	0.50	0.56	0.64	0.48
RankGen	0.60	0.58	0.52	0.63	0.54	0.59	0.66	0.55	0.58	0.66	0.53
BARTScore	0.60	0.57	0.49	0.58	0.55	0.55	0.55	0.48	0.58	0.58	0.53
(Answer, evidence docs) metrics											
QAFactEval	-	0.50	0.54	-	-	0.48	-	-	0.69	-	-
Learned metrics											
longformer	0.67	0.62	0.59	0.60	0.62	0.56	0.62	0.65	0.63	0.63	0.63
longformer (D)	-	0.60	0.61	-	-	0.54	-	-	0.65	-	-
GPT3 curie	0.69	0.55	0.59	0.60	0.51	0.45	0.53	0.55	0.58	0.56	0.51
Human	0.80 [◇]	0.73 [♣]	-	-	-	-	-	-	-	-	-

Table 5.4: Accuracy of automatic metrics for imitating human judgments of overall answer preference, coherence, and factuality. **h/m** denotes comparisons between human-written answers and model-generated answers, while **m/m** denotes comparisons between pairs of model-generated answers. [†]These metrics are calculated on 109 pairs of comparisons, where comparisons of History are removed because there are only one answer available on the subreddit and hence no reference answer to compare. [◇] We estimate the human performance with a pairwise agreement for two-way and three-way expert annotations. [♣] This pairwise agreement is reported by WEBGPT [], estimated on a subset of the data.

evidence documents d_1, d_2 here for simplicity, but QAFactEval and longformer (D) metric take the evidence documents as additional input.

HUMAN PREFERENCE DATA We compile human evaluations from previous studies [Krishna et al. 2021; Nakano et al. 2021] and our expert annotations from Section 5.3. See appendix 8.0.16 for

descriptions of the models evaluated in these datasets as well as data statistics on the answers. Both prior studies present large-scale preference judgments of **overall** answer quality and smaller-scale judgments for two targeted aspects, **coherence** and **factuality**. In total, we look at 3,478 comparisons on overall answer quality, 854 comparisons on coherence, and 469 comparisons on factuality. As shown by our analysis of expert annotations (Section 5.3), annotators can frequently disagree with each other.

5.4.3 RESULTS

Table 5.4 reports the accuracy of each metric at imitating human preference data. We report three baselines: **Random**, which randomly chooses one of the answers; **Always Human**, which prefers the human-written answer when available; and **Length**, which prefers the longer answer.¹⁰ **All metrics exhibit relatively low accuracies, falling substantially below estimated human agreement.** None of the metrics are robust across different types of input answer pairs. For instance, pretrained reference-based metrics such as BERTScore and BLEURT have low accuracy on HURDLES human vs. model data, which adds further evidence to the issues with ROUGE noted by Krishna et al. [2021]. Supervised metrics (Longformer and GPT-3) also struggle in this setting, despite outperforming all other metrics on overall rating in the other three data settings. While trained to imitate only overall rating, they achieve relatively strong accuracies on fine-grained ratings too, suggesting that they are correlated.

We observe spurious correlations with length for long-form answer evaluation. Choosing the longer answer achieves higher accuracy than all unsupervised metrics for the WEBGPT model vs. model comparison; the best performance on factuality for HURDLES human vs. model answer; and the second-highest accuracy on our expert data. On the other hand, when comparing WEBGPT human vs. model answers, choosing a shorter answer would have been more beneficial

¹⁰The **Length** baseline is inspired by prior findings in summarization [Liu et al. 2022; Sun et al. 2019] that **length** has a non-trivial impact in human preferences.

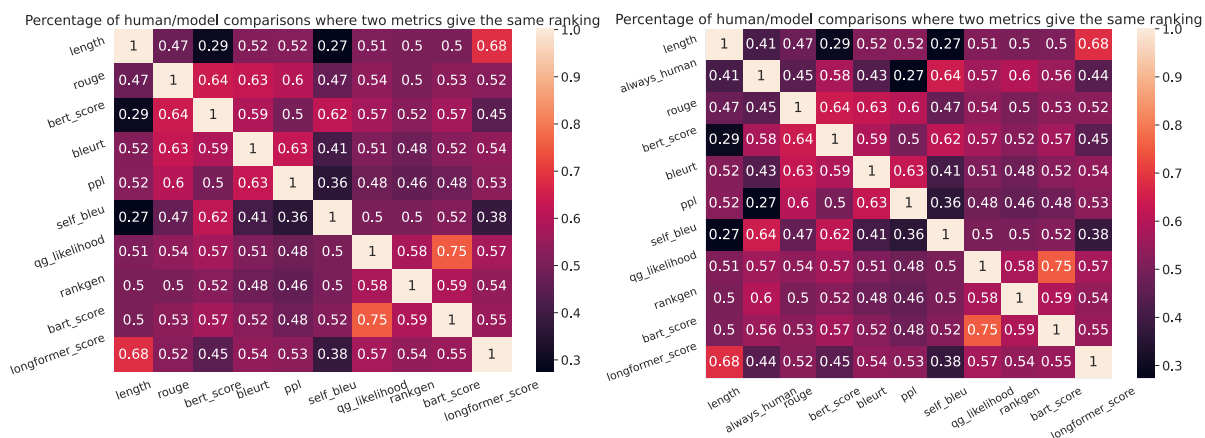


Figure 5.3: Pairwise automatic metric correlation.

for coherence evaluation (62% of the time). The “strong” performance of the length baseline displays the brittleness of all existing automatic metrics for LFQA.

It is more feasible to model fine-grained answer aspects than overall answer quality. The QAFactEval metric, designed for factuality, does indeed outperform all other metrics on factuality. However, the metric is limited in that it requires a set of input evidence documents, which may not always be available or reliable. For coherence, simpler metrics such as self-BLEU perform competitively, and we also find that our upper bound of always choosing the human answer performs strongly on coherence, suggesting that models struggle to generate coherent long-form answers.

Correlation of Automatic Metrics Given pairs of long-form answers of the comparison data, we measure how frequently two automatic metrics prefer the same answer (Figure 5.3). We see a positive correlation among reference-based metrics (e.g., rouge and bertscore gives the same ranking for 63% of the pairs), as well as the (question, answer) metrics (e.g. qq likelihood and bartscore).

5.5 CONCLUSION & FUTURE WORK

Our study provides a unified evaluation benchmark for long-form answers, including new annotations from domain experts. We present a new set of expert LFQA evaluations along with detailed justifications, and we also compile existing human annotations across different properties (overall preference, factuality, coherence) to facilitate future development of automatic LFQA metrics.

Evaluation of long-form answers is a multi-faceted problem and thus should be more targeted. Our expert justifications suggest that many aspects are considered when deciding which answer is better, some of which may be at odds with others (e.g. completeness vs. conciseness). This suggests that computing an “overall” score for answer quality is not meaningful, which is further supported by the limitations of metrics trained directly from overall preference judgments. Future work should look deeper into modelling frequent aspects mentioned by expert annotators, such as completeness and ease of understanding, perhaps by taking inspiration from evaluation methods that explicitly localize and categorize errors [Freitag et al. 2021; Goyal et al. 2022b].

LIMITATIONS

We study a limited scope of long-form answers. The questions are either drawn from search queries or from community forums. In the real world, we will encounter many more diverse forms of long form question answering, such as answering questions in education or commercial settings. We only cover the English language, and thus our questions are topically limited to English-speaking culture.

Our evaluation of long-form answers is stationary. Annotators are provided a pre-generated output from the model without being able to interact with the model over multiple rounds. A more interactive evaluation [Lee et al. 2022] of models is a great direction for future work.

6 | KIWI: A DATASET OF KNOWLEDGE-INTENSIVE WRITING INSTRUCTIONS FOR ANSWERING RESEARCH QUESTIONS

The previous chapter examines evaluation in open-ended settings, focusing on a single output from the model. But real-world interactions with language models often unfold over multiple turns: users refine their requests, ask follow-up questions, and incorporate model outputs into ongoing tasks. Standard benchmarks consist almost entirely of single-turn queries, leaving an entire class of failure modes that arise specifically in multi-turn settings invisible to evaluation. This chapter studies one representative multi-turn use case: language models as writing assistants on knowledge-intensive tasks. We design an interactive interface and collect KIWI, a dataset of interaction traces between expert annotators and language models. Our analysis on the interaction traces surfaces error patterns specific to multi-turn settings, including inconsistency across turns and failures to follow precise instructions. This chapter is based on work with Kyle Lo, Luca Soldaini, Bailey Kuehl, Eunsol Choi, and David Wadden [Xu et al. 2024a].



Figure 6.1: An example interaction session between a user and our system. Given a question q and a set of documents $\mathcal{D}$, the model first generates an initial answer a_0 . At each turn t , the user issues an instruction i_t , for which an updated answer a_t is generated. The user provides a rating r_t and a comment c_t for the generated answer, and optionally edits the answer (such as a'_3 in turn 3) such that the edited answer completely follows the instruction.

6.1 INTRODUCTION

As LLM-powered conversational agents [Google 2023; OpenAI 2023; Touvron et al. 2023] have gained widespread adoption, users have turned to these systems for assistance on a wide range of real-world tasks. In particular, recent works studying LLM interactions “in the wild” [Ouyang et al. 2023; Zheng et al. 2023a] have discovered that 10-20% of user queries contain requests for writing assistance, i.e. using an LLM to create, revise, or organize a piece of written text. However, we currently lack an understanding of LLMs’ capabilities as writing assistants,

especially in knowledge-intensive settings. What types of writing instructions do users issue to LLMs, and how well can LLMs follow different types of instructions?

We present **KIWI**, a dataset of expert-written **K**nowledge-**I**ntensive **W**riting **I**nstructions to better understand LLMs’ instruction-following abilities as writing assistants. To collect **KIWI**, we set up an interactive interface between a researcher and an LLM (Figure 6.1). We first prompt an LLM to generate a long-form answer [Fan et al. 2019a] to an NLP research question, based on a set of passages from relevant research papers. A researcher then iteratively issues instructions for the model to revise the answer and evaluates the model-generated revision. The interaction session continues until the user is satisfied with the final answer or a maximum number of turns is reached. **KIWI** contains instructions collected from 234 interaction sessions with three state-of-the-art language models (GPT-4, GPT-3.5-turbo and LLaMA-2-70b-chat), consisting of a total of 1,260 unique instruction instances. Each instance contains (1) a user instruction, (2) the model’s previous and (3) revised answers, and (4) a human annotator’s judgment (both categorical rating and free-form explanation) on whether the revised answer followed the instruction.

Using **KIWI**, we conduct an in-depth analysis to characterize the types of instructions issued by researchers, and to measure how well models can follow different types of instructions. We find that LLMs *do not* excel at this task yet, with the best model (GPT-4) achieving success for only 59% of the instructions. Specifically, LLMs fail to precisely follow user’s instructions (such as satisfying location and length constraints), often cause answer quality to degrade when integrating new information into a previous answer, and struggle to avoid making changes to answers that are not requested.

Finally, we examine the ability of the strongest LLM (GPT-4) to evaluate whether a response follows an instruction, comparing its judgement to the user ratings in **KIWI**. Unlike prior works which found that LLMs can judge outputs as reliably as humans for open-ended instruction following [Li et al. 2023a; Zheng et al. 2023b], our experiments show that GPT-4 cannot reliably evaluate responses for instructions in **KIWI**, which are often specific and precise. While the

best performing model which retrieves and prepends 10-shot examples improves upon zero-shot performance by 5% accuracy, it still lags behind human agreement by 12% accuracy. Given these findings, we believe that **KIWI** will be a valuable resource for future research in instruction following and evaluation.

6.2 INTERACTION DESIGN

6.2.1 DESIGN GOALS

While prior works [Kopf et al. 2023; Zhao et al. 2023a; Zheng et al. 2023a] have deployed an LLM in the wild to collect diverse types of user interaction data, we focus on the use case of seeking writing assistance. We design a knowledge-intensive interaction setting with two key considerations. First, the information in the answer should be grounded to a set of relevant documents. This allows us to examine model’s ability to leverage information from *multiple* documents, motivated by writing tasks such as literature review [Shen et al. 2023]. Second, the user interacts with the model for *multiple* turns to iteratively revise the answer. Compared to collecting a single editing instruction for (question, answer) pairs, this setting adheres to the iterative nature of the text editing process [Collins and Gentner 1980; Du et al. 2022], and allows us to collect diverse and fine-grained instructions across different editing stages, as we will see in §6.4.1.

6.2.2 INTERACTION PROCESS

We provide an overview of the interaction process (Figure 6.1) and describe the data collection procedure in §6.3. The system inputs are an NLP research question q and a collection $\mathbf{D}$ of research papers, which collectively contain information sufficient to write a high-quality, paragraph-length answer to q . To begin the interaction process, an LLM generates a “first draft” answer a_0 to q based

on **D**. Each interaction turn t involves four steps with actions taken by the user () or the model (): (1) The user reads the LLM-generated answer and issues an instruction i_t to the model to revise the answer. (2) The model then generates an updated answer a_t given q , a_{t-1} and i_t . (3) The user provides a rating r_t and a comment c_t evaluating how well a_t follows i_t . (4) The user can choose to edit the answer a_t if it did not follow their instruction, producing a'_t . Afterwards, the user can start a new turn by issuing another instruction for a_t (or a'_t). The interaction continues until either the user is satisfied with the answer, or a total of 10 turns is reached. We describe each step below with additional details (prompts, hyperparameters) in Appendix 8.0.17.

 **GENERATING THE INITIAL ANSWER** Given the question q , we first retrieve a set of five passages from **D** with a retrieval model R , denoted as $R(q, \mathbf{D})$. We prepend the retrieved passages to the question q to generate the initial answer a_0 with a language model M , that is, $a_0 = M(q, R(q, \mathbf{D}))$.¹

 **ISSUING INSTRUCTIONS** To start a turn t , the user issues an instruction i_t , specifying how they would like the LLM to revise its answer in the previous turn a_{t-1} , and categorizes its type I_t . We identify two types of instructions. **Information-seeking** instructions require the model to incorporate new information into the answer; e.g. Turn 3 of Figure 6.1 asks the model to acquire additional information from the source documents **D**. **Stylistic** instructions ask the model to transform content that is already present; e.g. Turn 4 in Figure 6.1 asks the model to compress its answer. For each interaction turn, the user indicates the type of the instruction I_t by selecting a checkbox in the annotation interface.

 **ANSWER REVISIONS** Next, the model M is prompted to revise its previous answer a_{t-1} to satisfy user instruction i_t . We use one prompt for information-seeking instructions (which require passage retrieval from **D**), and another for stylistic questions (which do not).

¹We perform retrieval because prepending the full text of multiple papers (each with around 6,000 tokens) exceeds the context window of the LLMs (with at most 8,000 tokens).

For information-seeking instructions, we first leverage a retrieval model R to retrieve five passages p_t from $\mathbf{D}$, using the instruction i_t as the query. Next, we prompt M to summarize the information in p_t that is likely to be relevant given (a_{t-1}, i_t) ; we denote this summary as s_t . Finally, we generate an updated answer by prompting M with (q, a_{t-1}, i_t, s_t) .² For stylistic instructions, we generate the updated answer directly by providing (q, a_{t-1}, i_t) as input. We only include a single previous answer in the prompt to generate the updated answer; in the pilot study, we found that this approach yielded comparable performance to including the entire interaction history.

 **MEASURING ANSWER QUALITY** After a revised answer a_t is generated, the user provides a rating indicating whether M successfully followed their instruction: $r_t = \mathbf{r}(i_t, q, a_{t-1}, a_t)$. We instruct the annotators to rate a_t as one of the following:

- good: Successfully followed the instruction.
- neutral: Partially followed the instruction.
- bad: Didn't follow the instruction at all, or modified the answer to an undesirable state.

Since our goal is to measure the helpfulness of the model response as determined by the annotator, we ask annotators to use their own judgement in selecting a rating rather than providing a detailed guideline.³ Nonetheless, we find in §6.5 that annotators have high agreement on this categorization. We also collect a mandatory free-form comment c_t from the annotator explaining their rating r_t . Example turns are in Table B4 in the Appendix 8.0.18.

 **HANDLING INCORRECT INSTRUCTION-FOLLOWING** When $r_t \neq \text{good}$, answer a_t does not fully follow instruction i_t and needs to be corrected. While the user might issue instructions in the subsequent turns to correct what has gone wrong with a_t , the success of the user's correction

²We found that this “summarize-then-answer” workflow yielded better results than directly prompting M with the retrieved passages p_t to produce a_t in preliminary experiments.

³To alleviate annotator burden, annotators are not required, but welcomed, to check the factual correctness of model-generated answers. Assessing faithfulness of LLM generations represents an important but orthogonal research direction.

attempt depends heavily on the model’s instruction following ability. The user might fall into a loop of continuing to prompt a model which is not able to perform the action requested in the instruction. To avoid such scenarios, we allow the user to edit a_t when $r_t \neq \text{good}$, which induces a'_t , such that $\mathbf{r}(i_t, q, a_{t-1}, a'_t) = \text{good}$. If the user chooses to not edit the answer and $r_t = \text{bad}$, we remove the answer from the answer history when generating answers in the subsequent turns.

6.3 DATASET COLLECTION

We first describe our approach to curate high-quality $(q, \mathbf{D})$ pairs (§6.3.1), followed by our annotation protocol for interaction collection (§6.3.2).

6.3.1 (QUESTION, DOCUMENT SET) CREATION

The process proposed in §6.2.2 requires high-quality $(q, \mathbf{D})$ pairs to serve as a starting point for interaction collection. We aim to create $(q, \mathbf{D})$ pairs which satisfy the following desiderata: (1) the questions should be realistic and likely to be asked by a researcher, (2) the questions should be challenging, requiring information from multiple documents and (3) each document set $\mathbf{D}$ should contain sufficient information to answer its corresponding question. We observe that the related work section of a research paper often answers a set of implicit research questions, and supplies the documents answering each question as citations. Thus, we collect high-quality $(q, \mathbf{D})$ pairs by writing questions implied by related work sections, and pairing each question with its cited papers.

SOURCE ARTICLE SELECTION We select a set of roughly 100 papers published in ACL 2023 across 11 different tracks listed in the ACL handbook.⁴ This set of papers covers a wide range of NLP research topics and was published after the training data cutoff time of the language models used

⁴We choose to focus on NLP research questions, since the authors of this paper have expertise in this domain.

for interaction collection in this work. The model thus has to answer the question based on the documents, instead of memorizing from its training data.

QUESTION CREATION AND FILTERING Three of the authors with prior NLP research experience manually annotate 88 questions from 75 papers. Given a source paper, the annotator first reads through its related work section and decides if a good question can be derived. A good question should fulfill the following criteria: (1) it should “stand alone”, i.e. it is not anchored in the source paper; and (2) there should be at least four articles cited in the related work which contain relevant information for answering the question. If a good question can be derived, the annotator writes a question q , together with the list of evidence papers D and the section of the related work r , from which the question is derived. We query the S2ORC [Lo et al. 2020] corpus to retrieve the full text of each paper and extract paragraphs. To ensure sufficient coverage, we filter out questions with either (1) more than one evidence paper missing in S2ORC or (2) fewer than 4 evidence papers in total. We obtain a total of 78 questions with an average of 6 cited papers per question.

ANALYSIS OF COLLECTED QUESTIONS Our annotated questions consist of 12 words on average, with the majority of questions starting with "How" (47%) and "What" (46%). Upon manual examination, all models are able to generate a reasonable initial answer given the question and retrieved passages. This shows that our pipeline creates high quality data: the document set indeed contains sufficient information to answer the question.⁵ See example questions in Table B3 in the Appendix 8.0.18.

6.3.2 INTERACTION DATA COLLECTION

At the start of an interaction session, the annotator is shown the question, initial answer, and the titles of relevant papers for the question. During the interaction, the annotator is shown

⁵In the initial phase of the project, we explored using GPT-4 to generate questions given related work paragraphs, but found that the generated questions were often not answerable by the cited papers. We include analysis on question generation in the Appendix 8.0.22.

Source Paper: Small pre-trained language models can be fine-tuned as large models via over-parameterization [Gao et al. 2023]

Related work paragraph: Over-parameterization in Neural Network. Over-parameterization has shown the superiority on providing better model initialization (Arpit and Bengio, 2019), improving model convergence (Du et al., 2019; Allen-Zhu et al., 2019b; Gao et al., 2022a) and generalization (Allen-Zhu et al., 2019a). [...]

Question: Why does over-parameterization lead to improved initialization and convergence of deep neural networks?

Evidence papers: Allen-Zhu et al. [2019]; Du et al. [2019]; Gao et al. [2022b]; Li and Liu [2016]

Table 6.1: An example (q , D) annotation (§6.3.1). Given a source paper, the annotator first selects the related work paragraph(s) that imply the question, then writes the question and extracts evidence papers cited.

the full interaction history, and the retrieved passages from the last info-seeking instruction. We release annotation instruction⁶ and provide annotation interface screenshot as Figure B2 in Appendix 8.0.17.

LANGUAGE MODELS We conduct annotations with three models M : GPT-3.5-turbo, GPT-4 and LLaMA-2-chat(70B) [Touvron et al. 2023], competitive commercial and public LLMs available at the time of data collection, respectively. For all models, we decode with temperature 0.7. During data collection, the identity of model M is not revealed to the annotator. For each question, we collect one interaction session with each model. We ensure that annotators do not answer the same question more than once.

RETRIEVAL MODELS We use the state-of-the-art dense retriever Contriever [Izacard et al. 2021] finetuned on MSMARCO [Campos et al. 2016] as our retriever R . In preliminary experiments, we found that Contriever outperformed sparse methods like BM25 [Robertson and Zaragoza 2009],

⁶<https://docs.google.com/document/u/1/d/e/2PACX-1vTrsM5r-p5kfy6Ue7AkbLqwOmJ-GoK9kwToBenW-Nwk00yd4tKSfWu9p63j7rH-PvNwwNXLzTyRWhi/pub>

which struggle at handling synonyms.

ANNOTATORS To ensure our annotators have expertise in answering research questions, we recruit NLP researchers through Upwork⁷ and professional networks. We pay the annotators USD \$25 per hour on average. The annotators first participate in a paid pilot study, which involves reading the instructions and completing an annotation example. A total of 22 annotators participated in our pilot annotations and 15 of them performed final annotations for **KIWI**. The final group of annotators consists of four people with a Ph.D. degree, seven NLP Ph.D. students, one person with a Master’s degree and three undergraduate students. All annotators have at least one year of NLP research experience. On average, each interaction session required 15 minutes to complete. To ensure that the annotators are familiar with the subject matter of the question, we first collect their preferences on a set of topics derived from the track information of each question’s source paper. We assign questions to annotators based on their preferences. We randomly assign models to each annotator and hide the model information from the annotator. The annotations were collected from July to December 2023. In total, our annotations cost \$2,240 USD.

6.4 ANALYSIS OF **KIWI**

We first present results on overall model performance (§6.4.1). Then we conduct a fine-grained analysis to understand the different types of instructions (§6.4.2) and errors made by LLMs (§6.4.3).

6.4.1 DATASET OVERVIEW

We present statistics of the interaction sessions in Table 6.2. We observe that the number of turns required to reach a satisfactory answer negatively correlates with model capability, with

⁷<https://www.upwork.com/>

Model	# turn (edited)	avg. # t/s	% info/style
gpt-4	370 (55)	4.7	59/41
gpt-3.5	402 (111)	5.2	61/39
llama2	488 (149)	6.3	44/56
Total	1,260 (315)	5.4	54/46

Table 6.2: Data statistics of the **KIWI**. We collect a total of 234 sessions (78 sessions per model). We report the total number of turns, number of turns with edited answers, number of turns per session, and the distribution of instruction types for each model.

Type	% info/style	% good	Definition	Example Instruction
More in- formation (43%)	88/12	gpt-4 : 57 gpt-3.5 : 45 llama2 : 34	Asking for auxiliary information related to the question or the answer (e.g. example, results, limitation).	How well do state-of-the-art models perform on these datasets?
Expand coverage (9%)	86/14	gpt-4 : 40 gpt-3.5 : 31 llama2 : 12	Asking for more information directly answering the question, usually from multiple papers.	Keep the text content as it is, but extract examples of applications of contrastive explanations to NLP from papers 0,1, and 2.
Clarification (5%)	69/31	gpt-4 : 52 gpt-3.5 : 50 llama2 : 50	Asking for clarification of a concept already in the answer.	Do all of the quantization methods you mention require additional training?
Remove (13%)	4/96	gpt-4 : 77 gpt-3.5 : 28 llama2 : 16	Requests to remove certain part of the answer.	Remove the repeated techniques listed in the last paragraph.
Specific edit (6%)	23/77	gpt-4 : 59 gpt-3.5 : 35 llama2 : 19	Requests to directly add information that are provided in the instruction verbatim to the answer.	At the end of the final paragraph, add the sentence "Evaluation of factuality for natural text remain active research lines."
Condense (10%)	4/96	gpt-4 : 63 gpt-3.5 : 27 llama2 : 23	Requests to shorten or summarize the answer.	Please shorten the answer to a maximum of 6 paragraphs.
Reorganize (11%)	13/87	gpt-4 : 74 gpt-3.5 : 27 llama2 : 34	Requests to reorganize the answer (e.g. move things around)	Reorganize the answer so that positive results are described first, followed by challenges and open problems.
Others (3%)	53/47	gpt-4 : 62 gpt-3.5 : 23 llama2 : 29	Edits that don't belong to any of the category above.	Don't change anything further about this answer but copy it as-is.

Table 6.3: Definitions and example instructions for the fine-grained instruction types (§6.4.2). We report the distribution classified by GPT-4 and % good rating (color-coded based on the % good) across the three models.

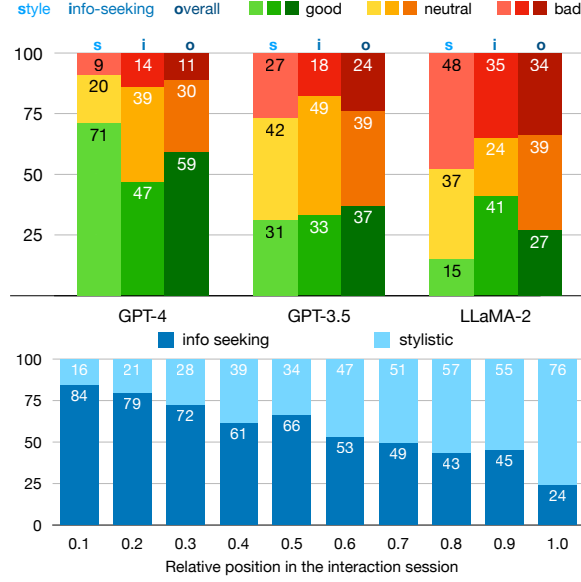


Figure 6.2: Top: distribution of annotator ratings. The left columns represent ratings for stylistic instructions (s), the middle columns for info-seeking instructions (i) and the right columns for overall (o) ratings. **Bottom:** distribution of instruction types across the session.

gpt-4 requiring the fewest turns and llama2 requiring the most. Comparing instruction types, around 60% of instructions are information-seeking for gpt-4, and 45% for llama2; this suggests that the gpt-4 is able to more quickly generate stylistically acceptable text, allowing users to focus on information content. We also observe that users tend to issue information-seeking instructions early in the interaction session followed by stylistic instructions (Figure 6.2). This confirms that collecting instructions over multiple turns allows us to cover diverse instructions occurring in different writing stages.

KIWI PRESENTS A CHALLENGE FOR EXISTING LLMs: The distribution over user ratings of model responses is shown in Figure 6.2. gpt-4 exhibits the strongest instruction-following performance (59% good responses), while llama2 exhibits the weakest (27% good responses). Interestingly, trends for the two types of instruction differ—while gpt-4 provides good responses to stylistic instructions around 70% of the time, it struggles at following information seeking instructions (i.e. adding new information to an answer). In contrast, llama2 performs extremely poorly

at following stylistic feedback, with only 15% of responses rated as good. All models fail to follow user instructions in more than 40% of responses, leaving significant headroom for LLMs’ instruction-following ability.

6.4.2 FINE-GRAINED INSTRUCTION ANALYSIS

Based on our findings that LLMs struggle at following the instructions in **KIWI**, we conduct a fine-grained analysis to categorize the different types of instructions found in the dataset, and measure model performance on each. We manually label a sampled set of instructions, finding eight categories, and then scale up the analysis using GPT-4. We construct a prompt containing an example of [(original question, instruction) → fine-grained instruction type] for each instruction type, and prompt GPT-4 to label the entire dataset. Details about the prompt and human analysis can be found in the Appendix 8.0.19. We validate our approach by confirming that GPT-4 achieves an agreement of 0.96 Cohen’s kappa [Cohen 1960] with humans on a held-out set of 40 instructions.

KIWI CONTAINS DIVERSE INSTRUCTION TYPES: The results of our analysis are shown in Table 6.3. Although we did not reference coarse-grained category labels when defining the fine-grained categories, we observe that our resulting fine-grained categories can be divided based on our distinction between information-seeking and stylistic instructions. Information-seeking instructions include requests to provide auxiliary information, expand answer coverage, or provide clarification. Stylistic instructions range from lower-level requests such as performing a specific edit, to more complicated operations such as condensing and reorganizing the answer. We report additional statistics for each category in Table B1 in the Appendix 8.0.19.

MODELS STRUGGLE AT INTEGRATING NEW INFORMATION AND PRECISE EDITING: Among the information seeking instructions, “expand coverage” is the most difficult, as this type of instruction requires both successfully retrieving relevant passages from *multiple* papers and coherently inte-

grating these new information into the previous answer. For stylistic instructions, gpt-4 performs well at removal and reorganization, achieving more than 70% good ratings, while gpt-3.5 and llama2 perform poorly. Interestingly, all three models struggle to make specific edits (e.g. add a requested sentence verbatim), which are usually trivial for humans. While prior work has found that LLMs excel at single document text summarization [Goyal et al. 2022a], we observe poor performance for the “condense” instructions (which request to shorten previous answer), with gpt-4’s response rated as good for only 63% of the time. We hypothesize that this is because answers in our task involve information from *multiple* papers, and multi-document summarization remains a challenge for current LLMs [Martin-Boyle et al. 2024; Shaib et al. 2023].

6.4.3 ERROR ANALYSIS

To understand *how* models fail to follow instructions, we analyze the free-form comments c_t written by the annotators. As in §6.4.2, we manually label a small set of comments and scale up with GPT-4 to all 760 turns which received a neutral or bad rating. We construct a prompt containing [(instruction, comment $\rightarrow$ target category)] demonstrations for each fine-grained error category; see Appendix 8.0.20 for details of human analysis and the exact prompt. On a validation set with 22 examples, GPT-4 achieves an agreement of 0.72 Cohen’s kappa with humans.

We find five major categories (Table 6.4). The most common error type is **Unrequested change**, indicating that models fail to maintain answer consistency across turns. Models also fail to satisfy hard constraints, such as the location (“At the beginning of the answer..”) and length of the information (“Add one sentence about..”) specified in the instruction. This suggests that current models struggle to perform *precise* actions, in agreement with findings from recent work [Sun et al. 2023; Yao et al. 2024] on controlled generation. Finally, models struggle to coherently integrate new information into the existing answer text, often leading to imbalanced structure or awkward answer flow.

Error type	Example
Unrequested change (31%) The update introduces changes not requested.	Instruction: Limit the number of sentences describing advantages and disadvantages for each method to 3. Comment: The model shortened the desired paragraphs but added another and removed information from the previous answer.
Ignored (21%) The requested change was not made.	Instruction: Reduce the discussion about task-specific pretraining objectives Comment: Model does not seem to have reduced any information at all.
Constraint failure (11%) The update in the answer does not follow some constraint(s).	Instruction: As first sentence of the text provide a definition of "clarification question". Comment: It gave the definition but not as first sentence in the text.
Poor integration (9%) The answer is less coherent after the update.	Instruction: Please start the answer by describing what "leveraging future utterance for dialogue generation" is. Comment: The requested info was added. However, the quality of the answer degraded somewhat and now has a odd ordering.
Others (28%) The update is not satisfactory for other reason.	Instruction: Explain some examples of methods in detail. Comment: only one example of an actual technique is given here

Table 6.4: Fine-grained error categories alongside example instruction and corresponding annotator comment pairs. We report the distribution classified by GPT-4 on interaction turns that are rated as neutral or bad.

RETRIEVAL ANALYSIS The updated answer could fail to follow the instruction due to failure of the retriever instead of of the LM. We randomly sample 20 turns with info-seeking instructions that are rated as neutral or bad and manually examine retrieval performance. We find that for 50% of the turns, relevant passages are retrieved which contain sufficient information to follow the instruction, yet the language models fail to integrate the information into the previous answer. For 20% of turns, some (but not all) relevant passages are retrieved. For the remaining 30%, the retrieval system fails to retrieve any relevant information. This demonstrates room for improvement in *both* components.

6.5 EXPERIMENTS: AUTOMATICALLY EVALUATING INSTRUCTION FOLLOWING

Our analysis so far demonstrates that current LLMs often fail to follow users’ instructions to revise long-form answers. Next, we examine whether they can serve as an *evaluator* to assess

whether a model-generated answer followed the instructions [Chiang and Lee 2023; Dubois et al. 2023; Liu et al. 2023b; Zeng et al. 2023; Zheng et al. 2023b].

SETTING AND DATA Formally, the task is to predict the user rating $r_t = \mathbf{r}(q, i_t, a_{t-1}, a_t)$, introduced in §6.2.2. To simplify the task, we collapse {neutral, bad} into a single label.⁸ We randomly split the 1,260 turns into train/dev/test set in a 70%, 15%, 15% ratio. We report aggregated results on three different random splits.

METRICS AND BASELINES We evaluate using **Accuracy**, **Precision**, **Recall** and **F1** against the collected human labels r_t , with the good label as the positive class. We report two baselines: a **Random** baseline which randomly assigns a label according to the training data distribution, and a **Majority** baseline which always chooses the majority class in the training data. We measure **Human** agreement by collecting a second set of human ratings for 65 randomly-sampled instructions, which reaches a moderate to substantial agreement (Cohen’s Kappa of 0.59) with the original rating.⁹ We report the average and standard deviation for human performance obtained through bootstrapping.

MODEL We experiment with **zero-shot** and **few-shot** prompting with gpt-4 and a **fine-tuned** T5-large (770M) [Raffel et al. 2019] model. For zero-shot prompting, we construct an instruction which specifies the criteria for the two ratings. Inspired by prior work on retrieving in-context examples [Rubin et al. 2022], we retrieve the k turns from the training data whose instructions have the highest BM25 similarity with the test instruction. See Appendix 8.0.21 for details.

RESULTS We report results on the test set in Table 6.5. GPT-4 zero-shot performs slightly worse than choosing the majority label; we found that it is biased heavily towards judging answers

⁸We report performance on three-way prediction in §8.0.21.3; the same conclusions hold as for two-way prediction.

⁹Two authors and two annotators from the interaction annotation performed the annotation. We ensure the interaction annotators do not re-annotate their own previous interactions.

Model	Acc	P	R	F1	% g/b
Majority	53	0	0	0	0/100
Random	47	42	31	36	38/62
gpt-4 zero-shot	63	57	91	70	75/25
gpt-4 one-shot	62	50	86	65	65/35
gpt-4 ten-shot	66	67	59	63	40/60
T5(finetuned)	64	50	73	37	24/76
Human*	80 ₅	63 ₉	90 ₇	74 ₇	46 ₆ /54 ₆

Table 6.5: Test set results on predicting ratings for instruction following edits (§6.5). We report the % of good and bad ratings in the model predictions in the last column.

as good. Adding 10 in-context examples improves performance, while leaving a substantial gap relative to human agreement. Finetuned T5 only achieves similar accuracy with zero-shot gpt-4. While recent work found that LLMs can rate responses as reliably as human [Chiang and Lee 2023; Zheng et al. 2023b], our experiments show that they are not reliable for judging responses for specific instructions [Liu et al. 2023b; Zeng et al. 2023].

6.6 RELATED WORK

INSTRUCTION FOLLOWING A number of recent efforts have collected instruction following datasets to train and evaluate LLMs. Some datasets contain crowd-sourced interactions in the wild [Conover et al. 2023; Kopf et al. 2023; Taori et al. 2023; Zhao et al. 2023a; Zheng et al. 2023a], while others target specific tasks such as summarization [Liu et al. 2023b] and controllable generation [Zhou et al. 2023]. Our work focuses specifically on instruction-following for writing assistance and text editing. A number of prior works have curated instruction-based editing benchmarks [Dwivedi-Yu et al. 2022; Raheja et al. 2023; Shu et al. 2023; Zhang et al. 2023]. However, these efforts consist of post-hoc synthetic instructions derived from previous datasets, consisting of a limited set of edit instructions. Our data consists of diverse instructions issued by expert users interacting with LLMs.

LLM-BASED EVALUATION Recent works have investigated using an LLM as an evaluator in place of human evaluation [Chiang and Lee 2023; Li et al. 2023a; Zheng et al. 2023b] for instruction following. A line of work shows that LLMs can judge responses as reliably as human annotators [Li et al. 2023a; Zheng et al. 2023b]. Recent work [Zeng et al. 2023] points out that evaluating open-ended instruction following exhibits high subjectivity and thus low human performance. The authors curate a set of instructions that are objective and craft adversarial responses to stress test LLMs, finding that the models struggle at evaluating responses for such instructions while humans exhibit high agreement. Our study also shows that LLMs struggle at evaluating responses for precise instructions collected in a *realistic* setting, complementing prior work.

DOMAIN-SPECIFIC DATASETS Prior work has introduced datasets to evaluate language models in specific domains, including law [Guha et al. 2023; Niklaus et al. 2023; Shen et al. 2022], medicine [Fleming et al. 2023; Jin et al. 2019], and computer science research papers [Dasigi et al. 2021; Lee et al. 2023]. While most prior work focuses on addressing a specific information need (e.g. through task formulations such as question answering or summarization), our setup more generally evaluates a model’s ability to closely follow user instructions, which can encompass many of these information needs.

6.7 CONCLUSION

We present **KIWI**, a dataset with expert-issued writing instructions to revise a long-form answer to a research question using relevant documents. Our analysis and experiments show that current LLMs (including GPT-4) cannot perform this task reliably yet and identify common failure patterns. We are optimistic that **KIWI** will be a useful resource for several research directions. First, the model revisions that were judged as successful by humans, and answers edited manually by humans when models failed, can be used as training data to improve models’ instruction following

abilities. Second, human judgments of model revision quality can be used to develop more accurate reward models for writing assistance. Finally, the instructions in **KIWI** can be used as inputs to evaluate the performance of future models; either through human judgments, or by future reward models.

7 | CONCLUSION AND FUTURE WORK

7.1 SUMMARY OF THE THESIS

This thesis addresses two challenges in deploying large language models (LLMs) for real-world information-seeking at scale: improving the **inference-time efficiency** of long-context LLMs, and developing **evaluation methods** that reflect how these models are actually used.

EFFICIENCY We approach inference-time efficiency by reducing the effective input length processed by the LLM, building on the observation that only a subset of a long input is typically useful for any given request. We instantiate this idea at two levels of the inference stack.

- In [Chapter 3](#), we propose RECOMP, which augments the LLM with an external compressor that rewrites retrieved documents before they are passed to the model. We instantiate two variants: an extractive compressor that selects sentences from the input, and an abstractive compressor that rewrites the input into a concise summary. By training compressors with signals derived from the LLM’s downstream performance, we preserve task accuracy while reducing the number of tokens the LLM processes at inference time.
- In [Chapter 4](#), we revisit the assumption that only a subset of tokens is useful for a given query. We instead consider the weaker assumption that only a subset is useful at any given *decoding step*, while the full input may be useful overall. RefreshKV retains the entire input

in the Key-Value cache and alternates between attending to the full context and to a small dynamically selected subset. Rather than training an external selector, we show that the LLM’s own attention patterns provide an effective signal for choosing which tokens to attend to.

EVALUATION We then turn to evaluation methods for settings where standard reference-based metrics and single-turn benchmarks fall short. We focus on two properties of real-world use that academic evaluations rarely capture: open-endedness and interactivity.

- In [Chapter 5](#), we examine evaluation practices for long-form question answering. We find that prior automatic metrics do not capture overall human preferences, though some metrics do capture specific aspects of answer quality such as coherence and faithfulness. Revisiting human evaluation with expert annotators, we find that divergence among annotator preferences does not necessarily indicate poor annotation quality. It can instead reflect genuine differences in what annotators value. These findings motivate a shift toward targeted, multi-faceted evaluation for open-ended tasks.
- In [Chapter 6](#), we study an axis that standard evaluations often overlook: interaction beyond a single turn. Focusing on writing assistance as a representative use case, we design an interactive interface and collect KIWI, a dataset of interaction traces between expert annotators and LLMs. The dataset reveals that language models achieve low success rates on this task and surfaces error patterns specific to multi-turn settings, including inconsistency across turns and failures to follow precise instructions.

7.2 FUTURE DIRECTIONS

7.2.1 FROM INPUT COMPRESSION TO SYSTEM-LEVEL EFFICIENCY

This thesis addresses inference-time efficiency by reducing the effective input length processed by the language model, targeting the dominant computational bottleneck for long-context inputs. Increasingly, however, the cost of inference is shifting from input to output: language models are deployed in settings that generate long reasoning chains [Guo et al. 2025; Muennighoff et al. 2025] and execute extended agentic loops [Liu et al. 2025; Research et al. 2026; Su et al. 2025], in which a substantial portion of inference compute is spent on what the model produces rather than what it reads. While input-side methods remain relevant in these settings [Lu et al. 2025; Wu et al. 2026], the shift also calls for efficiency methods that target the output side and, more broadly, the systems in which models now operate.

EFFICIENT THINKING AND TOOL USE. A first direction is teaching models to reason and leverage tools more efficiently. Recent work incorporates inference cost as part of the training reward, encouraging models to use fewer reasoning tokens or tool calls without sacrificing task performance [Arora and Zanette 2025; Wang et al. 2025a]. A complementary line dynamically allocates inference budget across requests, spending more compute on harder problems and less on easier ones [Damani et al. 2025; Liu et al. 2026a]. Extending these ideas to richer settings, such as long-horizon agentic tasks, remains an open direction.

EFFICIENT SYSTEM DESIGN. A second direction concerns the system surrounding the model. As tool use becomes a primary mode of LLM deployment, inference latency depends not only on the model itself but on the *scaffold* that orchestrates its calls, i.e. how tools are invoked, whether calls execute sequentially or in parallel, and how intermediate results are propagated. Recent work on *adaptive parallel reasoning* [Lian et al. 2025; Pan et al. 2025] illustrates the gains available from

rethinking the scaffold itself, allowing flexible interleaving of parallel and sequential computation. Future work should extend this perspective to tool use more broadly: designing scaffolds that exploit independence between tool calls, recover from slow or failing tools without blocking the larger trajectory, and jointly optimize the model and the surrounding system rather than treating either as fixed. Efficiency in this regime is a property of the compound system, beyond the language model alone.

7.2.2 BEYOND POINT EVALUATION

This thesis takes a step towards more realistic evaluation for LLMs, focusing on open-ended tasks and multi-turn interactions. It expands the types of tasks considered for evaluation, but still operates within the frame of point evaluation: performance measured for a single user, on a single output, at a single moment. As LLMs are deployed across all walks of life [Appel et al. 2025], evaluation should move beyond this frame along three dimensions.

First, evaluation should consider **different users**. Different user groups interact with LLMs in different ways and encounter different failure modes; evaluations that average over a generic user miss these failures entirely. In recent work [Huh et al. 2024], we study how Blind and Low Vision (BLV) users interact with vision-language models, finding that VLMs frequently hallucinate when presented with the blurry images typical of photos taken by BLV users, a failure mode specific to the group of BLV users. Future work should systematically characterize failure patterns across user groups whose interaction patterns and needs differ from the implicit default user that benchmarks assume.

Second, evaluation should consider **diversity across users**. As demonstrated by our analysis on long-form answers in Chapter 5, different users expect different responses to the same query, reflecting differences in preferences, context, and goals [Balepur et al. 2025; Jiang et al. 2025; Zhang et al. 2025]. A model that produces a single high-quality response is not the same as a model that produces a useful range of responses across a user population. Recent work we

contributed to [Liu et al. 2026b] finds that current language models struggle to generate diverse responses. Future work should both characterize this limitation more deeply and develop methods for eliciting diversity where it is warranted.

Third, evaluation should consider **longitudinal impact**. Current evaluations capture a single snapshot: how well does the model perform on this query, in this session? But LLMs are increasingly woven into how people work, learn, and make decisions over months and years [Shao et al. 2026; Shen and Tamkin 2026]. The downstream effects on user expertise, productivity, and decision quality, including effects that may not appear within a single interaction, are central to whether these systems are beneficial in the long run. Developing evaluation methods that capture these longer arcs is a substantially harder problem than the snapshot evaluation studied in this thesis, but a necessary one to measure LLMs impact to humanity.

7.2.3 FROM DIAGNOSING FAILURE TO ADAPTING MODELS

Many of the failure modes identified in this thesis can be attributed to domain shift. The observation that language models fail to follow precise instructions in expert writing tasks [Xu et al. 2024a], and that vision-language models hallucinate when given the blurry images typical of BLV users [Huh et al. 2024], likely stem from underrepresentation of these domains in training data. Indeed, follow-up work [Yuan et al. 2025] demonstrates that targeted training with carefully crafted data can mitigate some of these failures. This points toward a methodological program complementary to the work in this thesis: rather than only diagnosing where general-purpose models fail, we can adapt them to the domains where they do.

The central bottleneck for this program is data. For many domains where adaptation matters most, real training data is either expensive to collect, which requires domain experts whose time is scarce, or unavailable for use, due to privacy constraints on the underlying records. We identify synthetic data generation as a promising direction for addressing both forms of scarcity, and have taken initial steps along two complementary lines.

First, synthetic data offers a scalable alternative to expert human annotation. We present an agentic framework [Xu et al. 2026b] for automatically generating challenging deep-search questions used to train search agents. Collecting such data from humans is prohibitively expensive due to long and complex exploration trajectories. We propose a dual-agent framework in which a generator agent produces candidate data and a search agent executes against it, with the search agent’s execution feedback guiding the generator toward data that is both correct and appropriately difficult. The framework illustrates how synthetic data generation can be structured around the downstream agent’s behavior — generating data that is useful for training, rather than data that merely matches a reference distribution.

Second, when adaptation requires data that exists but cannot be used directly due to privacy, synthetic generation offers a path forward. We introduce **DP-RFT** Xu et al. 2026a, a privacy-preserving framework that uses private documents as a reward signal. **DP-RFT** uses a privacy-preserving nearest-neighbor vote as reward to train the data generator to produce synthetic data that resembles the private distribution without exposing it. This makes it possible to adapt models to domains whose data is sensitive (medical, legal, enterprise) while preserving formal privacy guarantees.

8 | APPENDICES

APPENDIX A

APPENDIX C: SUPPLEMENTARY MATERIAL FOR CHAPTER 3

8.0.1 COMPRESSOR TRAINING DATA GENERATION

We report the statistics of the data used to train compressors in Table C1. We use SpaCy [Honnibal et al. 2020] to extract named entities.

EXTRACTIVE DATA GENERATION We generate data using the training data for the four datasets we tested (Wikitext, NQ, TQA and HotpotQA). We use the NLTK package to perform sentence splitting. We remove examples without any negatives.

ABSTRACTIVE DATA GENERATION We report prompt used to generate summaries in Table C10. We queried the Open AI API with temperature of 0.7 and top p = 1. For the language modeling task, we use an ensemble of four prompts and choose the one which leads to the lowest perplexity as the target. If none of the summaries lead to perplexity decrease, we treat an empty summary as target. We queried the OpenAI API with temperature of 0.7 and top p = 1. We generate four summaries per example for randomly sampled 2% of the training data (48,013 examples).

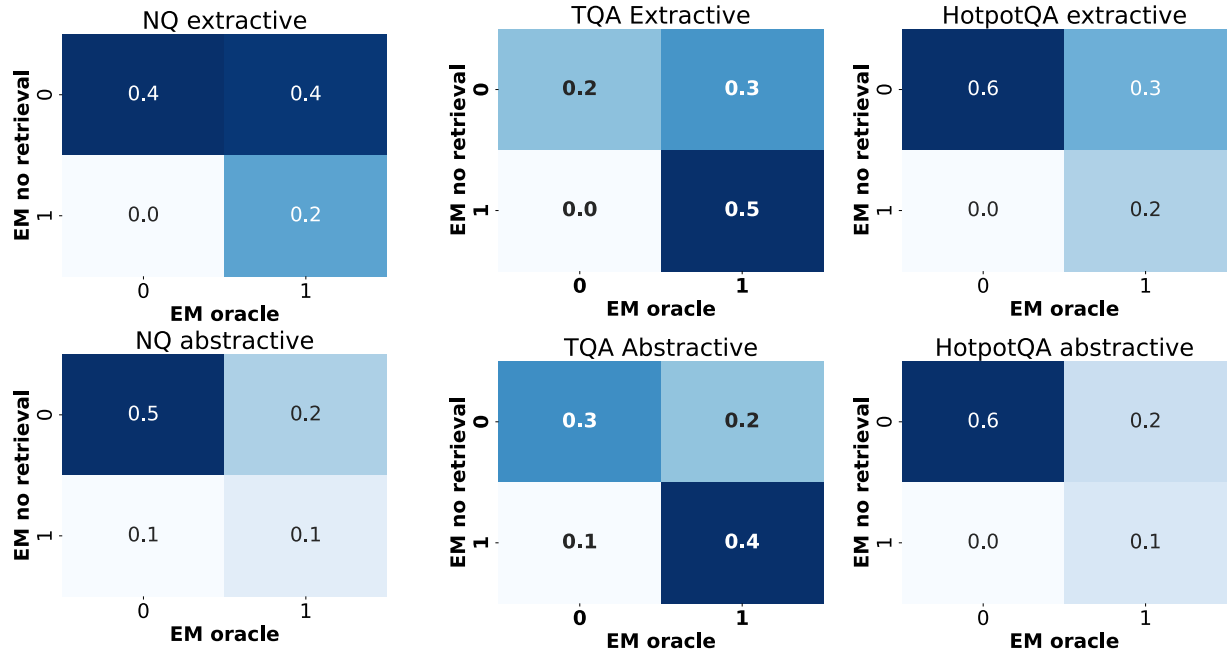


Figure C1: We report the data distribution on NQ dev set, TriviaQA dev set and HotpotQA dev set comparing the end task performance when prepending the oracle compression method (oracle sentence or GPT-3 summaries) and when not prepending anything for the base model (Flan-UL2).

Table C1: Training data statistics for abstractive and extractive compressors. “% filtered” refers to the percentage of examples not included. For extractive compressor, we filter examples for which prepending sentences with top contriever score leads to the correct answer. For abstractive compressor, we filter examples for which prepending GPT-3.5-turbo generated summary and no retrieval both produce incorrect answers. $|\mathcal{N}|$ refers to the average number of negative examples for constrastive training.

Dataset	Extractive				Abstractive			
	Train	Validation	% filtered	$ \mathcal{N} $	Train	Validation	% filtered	% empty
NQ	42,149	9,769	46	4.44	39,466	4,931	50	25
TQA	70,032	8,753	56	4.37	48,322	5,887	32	16
HotpotQA	24,526	3,068	69	4.33	26,556	2,937	42	4
Wikitext	1,398,318	1,5483	41	4.04	38,410	9,603	0	24

8.0.2 COMPRESSOR TRAINING DETAILS

EXTRACTIVE COMPRESSOR For language modeling, we use the contriever checkpoint¹ trained with unsupervised data. For the QA tasks, we use the contriever checkpoint fine-tuned on the MSMARCO task [Campos et al. 2016]², following prior work [Shi et al. 2023b; Si et al. 2022]. We implement the model using the Transformers [Wolf et al. 2019] and the sentence-transformer library [Reimers and Gurevych 2019]. We train with Adam optimizer [Kingma and Ba 2014], using a batch size of 64, learning rate of 2e-5 and 1000 warmup steps for 3 epochs. We report results on the model with the best reranked perplexity on our validation set for the language modeling task and the best reranked accuracy for the QA tasks.

ABSTRACTIVE COMPRESSOR We implement the model using the Transformers [Wolf et al. 2019]. We train abstractive summarizer with Adam optimizer [Kingma and Ba 2014], using a batch size of 16, learning rate of 1e-5 and 1000 warmup steps for 3 epochs.

¹<https://huggingface.co/facebook/contriever>

²<https://huggingface.co/facebook/contriever-msmarco>

Table C2: Inference speed of FLAN-UL2 on NQ dev set.

Setting	# toks	Inference time	Compression time	Total time	# example / s
No docs	0	4,104s	0s	4,104s	0.88
Top 5 docs	660	10,584s	0s	1,0584s	0.34
Contriever (ours)	37	4,880s	113s	4,993s	0.72
T5 (ours)	36	4,936s	871s	5,807s	0.62

8.0.3 INFERENCE SPEED

We report the number of tokens as a measure for efficiency in the main paper. Here, we report inference speed measured by GPU time as an additional metric for efficiency. Specifically, we report the GPU time for NQ dev set with FLAN-UL2 as the base model. We run FLAN-UL2 on 4 A40 GPUs. For compression, we run contriver and T5 on a single A40 GPU (Table C2). Our methods, even when including the time taken for compressor inference, improve throughput compared to prepending the full sets of documents, with contriever being more efficient, enabling 2x throughput.

We note that unlike the number of tokens, inference speed is contingent on implementation and size of the base model. For instance, bigger models will suffer from more latency with more input tokens and thus RECOMP can bring more speed-up.

8.0.4 COMPRESSING DOCUMENTS RETRIEVED BY DIFFERENT RETRIEVAL MODELS

We experiment with using our compressor trained with one retrieval model to compress texts retrieved by other retrievers. We report results on NQ (Table C3, compressing documents retrieved by contriever and DPR with compressors trained with contriever-ms-marco. We see that both our extractive and abstractive compressor generalize robustly to documents retrieved by another retrieval system, with the best compressor outperforming prepending top 1 documents with 4x compression.

Table C3: Compressing retrieved documents for contriever and DPR on NQ.

Doc	Compression	DPR		Contriever	
		# tokens	EM	# tokens	EM
No	-	0	21.99	0	21.99
Top 1	-	133	36.59	130	27.53
Top 5	-	667	42.44	652	33.19
Top 5	Contriever	26	25.60	31	23.55
Top 5	Extractive (ours)	36	35.76	35	30.66
Top 5	T5	10	26.84	10	24.32
Top 5	Abstractive (ours)	35	37.40	28	30.55

Table C4: Comparing prepending top 5 documents and prepending GPT-3.5-turbo compressed documents.

GPT 3.5 EM	Top 5 EM (no compression)	% Data		
		NQ	TQA	HotpotQA
0	0	53	31	61
0	1	10	7	8
1	0	7	7	6
1	1	30	55	25

8.0.5 MORE ANALYSIS ON ABSTRACTIVE COMPRESSOR FOR THE QA TASKS

8.0.5.1 IS THE TEACHER MODEL A GOOD SUMMARIZER?

As shown in Table 3.2, compressing with the teacher model (GPT-3.5-turbo) results in slight performance drop compared to prepending all top 5 documents for all three datasets (NQ, HotpotQA and TQA). We look closer into where the performance gap comes from by comparing the end task performance (Exact Match) between prepending GPT-3.5-turbo generated summaries and prepending the top 5 documents (Table C4). Compressing with GPT-3.5-turbo leads to performance decreases for 10%, 7% and 8% of data on NQ, TQA and HotpotQA. Notably, it also leads to an increase for a small portion of data (7%, 7% and 6%) on the three datasets.

Table C5: Manual analysis of when compressing with GPT-3.5-turbo leads to different performance compared to prepending top 5 documents. We report % examples belonging to each reason.

Reason	NQ	TQA	HotpotQA
Ambiguous question	17	11	5
Semantic equivalence	44	32	15
Insufficient evidence	11	42	50
Lost in the middle	11	10	15
Incorrect summary	17	5	15

MANUAL ANALYSIS We randomly sample 20 summaries for which compressing with GPT-3.5-turbo leads to different performance than prepending top 5 documents per datasets to conduct manual analysis. We report manual analysis results in Table C5. **Ambiguous questions** refer to the cases where the question itself is ambiguous [Min et al. 2020] and answers from both systems are valid. **Semantic equivalence** are cases where both answers are semantically similar. As we can see, these two categories constitute a non-trivial portion of the data analyzed. **Insufficient evidence** refers to the cases where the retrieved documents cannot sufficiently support the gold answer. This can include cases where (1) GPT-3.5-turbo hallucinates the gold answer or (2) when prepending top 5 documents, the model generates the correct answer, yet it cannot be fully supported by the retrieved documents. This also demonstrates headroom in improving the retrieval system, especially for more challenging dataset such as HotpotQA. **Lost in the middle** refers to cases where the model is unable to extract the correct answer from the top 5 documents while compression helps. Finally, **Incorrect summary** refers to genuine errors caused by compressing the retrieved documents.

8.0.5.2 DISTILLATION PERFORMANCE ANALYSIS

We examine whether the abstractive compressors successfully distill the teacher model on the three QA datasets. We compare performances of FLAN-UL2 when compressing with our T5 model and when compressing with the oracle GPT-3.5-turbo model (Table C6. We see that models face

Table C6: Analysis on distillation performance, grouped by whether the generated summary is an empty summary. The left two column denotes whether the summaries generated by T5 or oracle GPT-3.5-turbo is empty (**E**) or not (**N**). We report the **EM** for the T5 system and the oracle system (**EM(O)**) and the percentage of data belong to this slice.

T5	O	NQ			TQA			HotpotQA			Comment
		EM	EM(O)	%	EM	EM(O)	%	EM	EM(O)	%	
N	N	0.47	0.49	60	0.68	0.76	70	0.30	0.34	85	Both generate a summary.
N	E	0.31	1.0	4	0.26	1.0	6	0.12	1.0	3	Incorrect selective augmentation.
E	N	0.11	0.25	32	0.32	0.41	21	0.17	0.24	11	
E	E	1.0	1.0	4	1.0	1.0	3	1.0	1.0	1.0	T5 correctly generates an empty summary.

challenges in performing selective distillation for all three datasets. Our systems fail at performing selective augmentation for 36%, 27% and 14% of the data on NQ, TQA and HotpotQA (it either attempts to generate a summary when the teacher model fails to do so, or outputs an empty summary when generating a summary will lead to better performance). When both systems generate a summary, our distillation is relatively successful on NQ (4% performance drop) but less effective on TQA (11%) and HotpotQA (12%) performance drop.

SELECTIVE AUGMENTATION We further calculate performance of selective augmentation against the oracle method, i.e. we treat selective augmentation as a binary classification task and evaluate whether our compressor correctly chooses to output an empty summary (or not). The F1 performance is 0.19, 0.20 and 0.10 for NQ, TQA and HotpotQA respectively. The performance of a random baseline, which randomly outputs an empty summary at the same frequency of the oracle method, is 0.10, 0.08, and 0.00 for NQ, TQA and HotpotQA. This shows that although our compressors learn to perform selective augmentation to some extent, there is significant headroom to improve selective augmentation performance.

8.0.5.3 TRANSFERRING LEARNED COMPRESSORS TO ANOTHER MODEL FOR QA TASKS

We report results of transferring our compressors to LLaMA-13B [Touvron et al. 2023] model in Table C7. We use exactly the same input (e.g. same in-context example and prepended documents) as the FLAN-UL2 model. We report oracle for the FLAN-UL2 model (*Oracle (w/ FLAN)*) as well as oracle for LLaMA-2. While we see successful transfer for the language modelling task from GPT2 to GPT2-XL and GPT-J, transferring compressors trained for QA tasks to another model yields rather negative results. First, we see that oracle compression for FLAN-UL2 model significantly lags behind the oracle method for LLaMA-2 for both compression methods and all three datasets. As oracle method depends on parametric knowledge of the model (i.e. the model relies on parametric when an empty summary is prepended), one potential reason for transfer failure is that compression model trained with one model might not transfer to another model with parametric knowledge differences. We also note that for summaries generated by GPT-3.5 on HotpotQA, which work almost as well as prepending top 5 documents for FLAN-UL2, lags behind prepending top 5 documents when prepended to LLaMA. This shows that summaries which can be effectively consumed by one model might not be able to transfer to another model. Future work should look into how to build compression model that work across different models, especially for downstream tasks such as question answering.

Table C7: Open-domain QA results on LLaMA-13B. We report the results of oracle compressions with Flan-UL2, the base model for the compressors, (*Oracle w/ FLAN*) and the oracle compression results for LLaMA-13B.

In-context evidence	# tok	NQ		# tok	TQA		HotpotQA		
		EM	F1		EM	F1	# tok	EM	F1
-	0	30.89	40.73	0	65.00	71.18	0	24.20	34.50
<i>RALM without compression</i>									
Top 1 document	132	33.35	43.13	136	66.62	73.10	138	34.40	44.17
Top 5 documents	660	37.04	47.60	667	70.61	77.51	684	37.00	47.11
<i>Phrase / token level compression</i>									
Top 5 documents (BoW)	450	33.05	43.36	259	66.59	73.40	255	30.00	39.13
Top 5 documents (NE)	338	34.60	44.91	128	65.88	72.59	157	29.20	37.93
<i>Extractive compression of top 5 documents</i>									
Oracle	31	56.62	68.89	31	84.61	80.46	69	42.20	51.34
Oracle (w/ FLAN)	34	40.89	50.06	32	68.52	74.96	70	35.20	45.13
Random	32	30.33	39.85	31	62.80	69.25	61	27.40	36.27
Contriever	36	32.52	42.01	40	65.88	72.44	78	34.60	43.99
Ours (init. w/ Contriever)	37	34.38	44.15	38	65.28	71.85	75	33.20	42.88
<i>Abstractive compression of top 5 documents</i>									
Oracle	50	45.60	84.87	38	74.37	79.83	98	41.40	51.54
Oracle (w/ FLAN)	51	38.98	49.40	37	69.86	76.46	102	35.40	46.17
GPT-3.5	56	35.71	46.05	41	67.90	74.88	107	34.60	45.40
T5	10	33.38	43.54	7	63.18	70.92	7	30.40	40.60
Ours (init. w/ T5)	36	36.32	46.10	32	66.27	73.12	81	30.80	40.61

Input: Base LM M , Compressor encoder enc_θ , Training data $\{\mathbf{x}_i, \mathbf{S}_i, \mathbf{y}_i\}_1^T$ where $\mathbf{x}_i$ is input, $\mathbf{S}_i = \{\mathbf{s}_j\}_1^n$ is a set of candidate sentences from the retrieved document for $\mathbf{x}_i$, $\mathbf{y}_i$ is the target answer.

Output: An updated extractive compressor encoder enc_θ

```

1:  $\mathcal{T} \leftarrow \emptyset$ 
2: for  $i \in \{1, \dots, T\}$  do
3:    $\mathbf{p}_i \leftarrow \text{argMax}_{\mathbf{s}_j \in \{\mathbf{S}_i\}} \text{Score}(M, \mathbf{y}_i, [\mathbf{s}_j; \mathbf{x}_i])$ 
4:   for  $j \in \{1, \dots, n\}$  do
5:      $\mathcal{L} \leftarrow \emptyset$ 
6:     if  $\text{Score}(M, \mathbf{y}_i, [\mathbf{s}_j; \mathbf{x}_i]) < \text{Score}(M, \mathbf{y}_i, [\mathbf{p}_i; \mathbf{x}_i])$  then
7:        $\mathcal{L} \leftarrow \mathcal{L} \cup \mathbf{s}_j$ 
8:     end if
9:   end for
10:  if  $|\mathcal{L}| > 0$  then
11:     $\mathcal{N}_i \leftarrow \text{argTop5}_{\mathbf{s}_j \in \mathcal{L}}(\langle \text{enc}_\theta(\mathbf{s}_j), \text{enc}_\theta(\mathbf{x}_i) \rangle)$ 
12:     $\mathcal{T} \leftarrow \mathcal{T} \cup \{(\mathbf{x}_i, \mathbf{p}_i, \mathcal{N}_i)\}$ 
13:  end if
14: end for
15:  $\text{enc}_\theta = \text{Finetune}(\text{enc}_\theta, \mathcal{T})$ 

```

Figure C2: Learning an extractive compressor for QA task. The **Score** here is the exact match between the decoded answer and the gold answers.

Input: Teacher LM M_t , Base LM M , Summarization prompt p , Compressor encdec_θ , Training data $\{\mathbf{x}_i, \mathbf{D}_i, \mathbf{y}_i\}_1^T$ where $\mathbf{x}_i$ is input, $\mathbf{D}_i$ is the set of retrieved document for $\mathbf{x}_i$, $\mathbf{y}_i$ is the target answer.

Output: An updated encdec_θ

```

1:  $\mathcal{T} \leftarrow \emptyset$ 
2: for  $i \in \{1, \dots, T\}$  do
3:    $\mathbf{s}_i = \text{Decode}(M_t, [p; \mathbf{x}_i; \mathbf{D}_i])$ 
4:    $v_s = \text{Score}(M, \mathbf{y}_i, [\mathbf{s}_i; \mathbf{x}_i])$ 
5:    $v_d = \text{Score}(M, \mathbf{y}_i, [\mathbf{x}_i])$ 
6:   if  $v_s < v_d$  then
7:      $T \leftarrow T \cup \{(\mathbf{x}_i, \mathbf{D}_i, \emptyset)\}$ 
8:     continue
9:   end if
10:   $T \leftarrow T \cup \{(\mathbf{x}_i, \mathbf{D}_i, \mathbf{s}_i)\}$ 
11: end for
12:  $\text{encdec}_\theta = \text{Finetune}(\text{encdec}_\theta, T)$ 

```

Figure C3: Learning an abstractive compressor for QA task. The **Score** here is the exact match between the decoded answer and the gold answers.

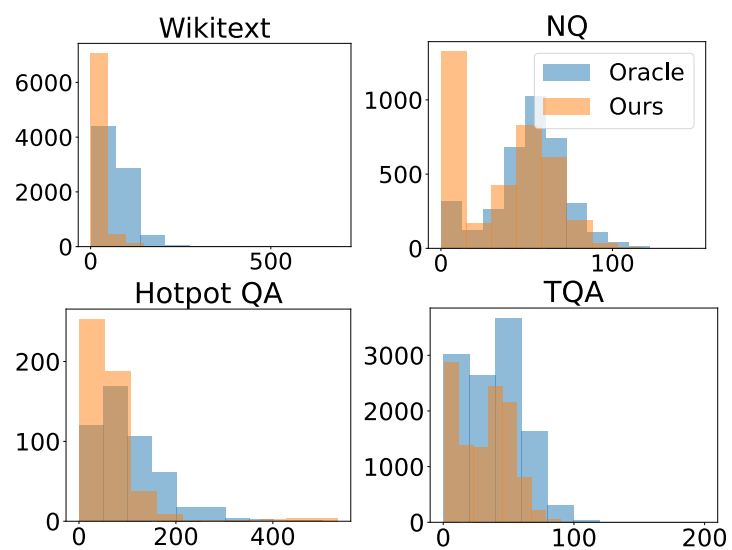


Figure C4: Histogram of abstractive summary length (# tokens) distribution for testing data of NQ,TQA, HotpotQA and Wikitext.

Table C8: Example abstractive and extractive compression on wikitext-103 dev set and NQ.

Wikitext-103	Original Top 1 document
Input present in most of the Mediterranean Sea, only missing from the section east of Crete, and along only the north @-@ west coast of the Black Sea	Sea of Crete" Sea of Crete The Sea of Crete (, ""Kritiko Pelagos"") or Cretan Sea, is a sea, part of the Aegean Sea, located in its Southern extremity. The sea stretches to the North of the island of Crete, East of the islands of Kythera and Antikythera, South of the Cyclades, and West of the Dodecanese islands of Rhodes, Karpathos and Kassos. The bounding sea to the West is the Ionian Sea. To the Northwest is the Myrtoan Sea, a subdivision of the Mediterranean Sea that lies between the Cyclades and Peloponnese. To the East-SE is the rest of the Mediterranean Sea,
Method	Compressed document
BoW	present Mediterranean Sea missing section east Crete along north west coast Black The Kritiko Pelagos Cretan sea part Aegean located Southern extremity stretches North island East islands Kythera Antikythera South Cyclades West Dodecanese Rhodes Karpathos Kassos bounding Ionian To Northwest Myrtoan subdivision lies Peloponnese SE rest
NE	Kythera the Aegean Sea the Ionian Sea Crete Southern South Rhodes the Myrtoan Sea Cretan Sea Antikythera Dodecanese Kassos Karpathos West the Black Sea & Sea of Crete Peloponnese the Mediterranean Sea Cyclades
Extractive compression	To the Northwest is the Myrtoan Sea, a subdivision of the Mediterranean Sea that lies between the Cyclades and Peloponnese.
NQ Input	Original Top 5 document
who got the first nobel prize in physics	receive a diploma, a medal and a document confirming the prize amount. Nobel Prize in Physics The Nobel Prize in Physics () is a yearly award given by the Royal Swedish Academy of Sciences for those who have made the most outstanding contributions for mankind in the field of physics. It is one of the five Nobel Prizes established by the will of Alfred Nobel in 1895 and awarded since 1901; the others being the Nobel Prize in Chemistry, Nobel Prize in Literature, Nobel Peace Prize, and Nobel Prize in Physiology or Medicine. The first Nobel Prize in Physics was science, Ernest Lawrence won the Nobel Prize in Physics in 1939. Lars Onsager won the 1968 Nobel Prize in Chemistry. Norman Borlaug, father of the Green Revolution, won the Nobel Peace Prize in 1970. Christian B. Anfinsen won the Nobel Prize for chemistry in 1972. Ivar Giaever won the Nobel Prize in Physics 1973. Carl Richard Hagen is noted for his work in physics. In engineering, Clayton Jacobson II is credited with the invention of the modern personal watercraft. Ole Singstad was a pioneer of underwater tunnels. Ole Evinrude invented the first outboard motor with practical commercial application, recognizable today Nobel Prize in Physics The Nobel Prize in Physics () is a yearly award given by the Royal Swedish Academy of Sciences for those who have made the most outstanding contributions for mankind in the field of physics. It is one of the five Nobel Prizes established by the will of Alfred Nobel in 1895 and awarded since 1901; the others being the Nobel Prize in Chemistry, Nobel Prize in Literature, Nobel Peace Prize, and Nobel Prize in Physiology or Medicine. The first Nobel Prize in Physics was awarded to physicist Wilhelm Röntgen in recognition of the extraordinary services he was also awarded the Abel prize. In addition, eight "normaliens" have gone on to receive the Nobel Prize in Physics: Claude Cohen-Tannoudji, Pierre-Gilles de Gennes, Albert Fert, Alfred Kastler, Gabriel Lippmann, Louis N 00e9el, Jean Baptiste Perrin and Serge Haroche, while other ENS physicists include such major figures as Paul Langevin, famous for developing Langevin dynamics and the Langevin equation. Alumnus Paul Sabatier won the Nobel Prize in Chemistry. A ranking of universities worldwide based on ratios of alumni to Nobel prize-winners published in 2016 by American scholars Stephen Hsu and Jonathan Wai placed ENS as the first university worldwide, far rendered by the discovery of the remarkable rays (or x-rays). This award is administered by the Nobel Foundation and widely regarded as the most prestigious award that a scientist can receive in physics. It is presented in Stockholm at an annual ceremony on 10 December, the anniversary of Nobel's death. Through 2018, a total of 209 individuals have been awarded the prize. Only three women (1.4% of laureates) have won the Nobel Prize in Physics: Marie Curie in 1903, Maria Goeppert Mayer in 1963, and Donna Strickland in 2018. Alfred Nobel, in his last will and testament, stated that his
Method	Compressed document
T5	Wilhelm Röntgen received the first Nobel Prize in Physics in recognition of his extraordinary services. It is one of the five Nobel Prizes established by Alfred Nobel in 1895 and awarded since 1901.
GPT-3.5-turbo	The first Nobel Prize in Physics was awarded to physicist Wilhelm Röntgen in 1901 for his discovery of the remarkable rays (or x-rays). Since then, 209 individuals have been awarded the prize, with only three women (1.4% of laureates) having won it.

Table C9: Example input to the Flan-UL2 for NQ with in-context examples and retrieved documents.

Dataset	Prompts
NQ	who won a million on deal or no deal Answer: Tomorrow Rodriguez who is the woman washing the car in cool hand luke Answer: Joy Harmon who is the actor that plays ragnar on vikings Answer: Travis Fimmel who said it's better to have loved and lost Answer: Alfred , Lord Tennyson name the first indian woman to be crowned as miss world Answer: Reita Faria Retrieved Docs Question Answer:

Table C10: Prompts used to generated summaries from GPT-3.5-turbo. query and docs represent the actual input query and retrieved documents.

Dataset	Prompts
NQ	Compress the information in the retrieved documents into a 2-sentence summary that could be used to answer the question: Question: query Retrieved documents: docs Compressed documents:
TQA	Compress the information in the retrieved documents into a 2-sentence summary that could be used to answer the question: Question: query Retrieved documents: docs Compressed documents:
HotpotQA	Source documents: docs Question: query Generate a reasoning chain to answer the question:
Wikitext	Generate the next two sentences of the given query using the information from the provided documents. \nSource Documents: docs \nQuery: query \n
Wikitext	Select sentences from the retrieved docs that are most likely be in the next sentence.\nSource Documents: docs \nQuery: query\n
Wikitext	Generate the next one sentence of the given query using the information from the provided documents\nSource Documents: docs \nQuery: query \n
Wikitext	Summarize the information from the provided documents\nSource Documents: docs \nQuery: query\n

Table C11: Case study of how compressing the retrieved documents helps the model to identify the right answer from NQ dev set.

<i>Question: host of the late show who was once a correspondent for the daily show. Gold answer: Stephen Colbert</i>		
Type	In-context documents	Predicted Answers
None		Chelsea Handler
Top 5	by Conan O'Brien, in 2009. Leno explained that he did not want to see a repeat of the hard feelings and controversy that occurred when he was given the show over David Letterman following Carson's retirement in 1992. O'Brien's last "Late Night" episode was taped on February 20, 2009. Former Saturday Night Live alum Jimmy Fallon took over as host of "Late Night with Jimmy Fallon" on March 2, 2009. The Colbert Report that aired four days a week on Comedy Central from October 17, 2005, was hosted by Stephen Colbert, one of the regulars on Comedy Central's The Daily season as host began with a notable interview with former British prime minister Tony Blair. The live interview occurred the night before a book signing at Eason's which attracted international attention when Blair was pelted with shoes and eggs and successfully evaded an attempted citizen's arrest on charges of war crimes. On 1 February 2013, Pat Kenny returned to host that night's edition when Tubridy's father died. In 2015, Tubridy's tone and choice of questions when interviewing Anti-Austerity Alliance TD Paul Murphy in relation to the campaign against the implementation of a water tax was much criticised. Opponents of the 'Michigan, interviewing Eminem. Colbert has been given near-full control of the show, with little interference from CBS management in regard to format. Colbert brought most of his staff from "The Colbert Report" with him to "The Late Show", as well as outsiders such as Brian Stack, who is best known for his work on Conan O'Brien's programs, and Jon Stewart, former host of Colbert's previous sister program "The Daily Show", who is credited as executive producer. Colbert no longer uses the character he had portrayed on "The Colbert Report", jokingly remarking to Jeb Bush that "I used to play a Show" has had three regular hosts: Gay Byrne, Pat Kenny and Ryan Tubridy. Frank Hall deputised for Byrne for one season in the 1960s. There have been at least four occasions on which another presenter has hosted the show. The first was when Byrne became unexpectedly and seriously ill. Frequent panelist Ted Bonner presented instead. The second time was towards the end of a show about feminism, when Byrne ushered a young Marian Finucane into his seat to present the remainder of the show. On another occasion, radio broadcaster and former news reader Andy O'Mahony replaced Byrne for an interview popular acclaim. Colbert would host the program until he was chosen to replace David Letterman as host of CBS's "Late Show" in 2015. Ed Helms, a former correspondent from 2002 to 2006, also starred on NBC's "The Office" and was a main character in the 2009 hit "The Hangover". After filling in as host during Stewart's two-month absence in the summer of 2013, John Oliver went on to host his own show on HBO, "Last Week Tonight with John Oliver". In 2016, former correspondent Samantha Bee launched her own late-night talk show "Full Frontal with Samantha Bee". Bee's husband Jason	Samantha Bee
GPT-3.5-turbo	Former Daily Show correspondent Stephen Colbert was chosen to replace David Letterman as host of CBS's "Late Show" in 2015, while Ed Helms, a former correspondent from 2002 to 2006, also starred on NBC's "The Office" and John Oliver, who filled in as host during Jon Stewart's absence in 2013, went on to host his own show on HBO.	
T5 (ours)	Stephen Colbert was a former correspondent for The Daily Show and later became host of CBS's "Late Show" in 2015. He has since brought most of his staff from "The Colbert Report" with him to "The Late Show", with little interference from CBS management in regard to format.	Stephen Colbert

Table C12: Example summaries and their manual analysis labels. See Table C13 for more example.

Dataset	Model	Query, Passages and Summary	Evaluation
NQ	Ours	Question: when will miraculous ladybug season 2 episode 12 come out Passages: 2016 on TVNZ's TV2. In Japan, Disney Channel streamed the episode "Stormy Weather" through its mobile application on 1 July 2018, before the official premiere on 23 July in the same year. The second season premiere is scheduled for a global launch around September–November 2017 in Europe, At a panel at San Diego Comic-Con 2017, it was announced that the second season would have its North American release on Netflix in December 2017, with 13 episodes to be released. KidsClick will start airing season 2 of this show in the US starting 30 August 2018, marking the first time that Korea on 1 September 2015 on EBS1. In the United States, the series debuted on Nickelodeon on 6 December. In the United Kingdom and Ireland, the show premiered on 30 January 2016 on Disney Channel. A Christmas special was released in 2016 and the second season premiered in French on TF1 and in English on Disney Channel UK in 2017. Netflix acquired the U.S. video-on-demand streaming rights and further seasons are in production. Set in modern-day Paris, the series focuses on teenagers Marinette Dupain-Cheng and Adrien Agreste. When evil arises, Marinette transforms into her superhero persona Ladybug, while Adrien transforms rights in Europe, Eastern Europe and Scandinavia, and free-to-air rights in Spain, Germany, Russia and Turkey. South Korea was the first country to premiere "Ladybug", with girl group Fiestar to sing its translated theme song. It aired on 1 September 2015 on EBS1, and ran for 13 episodes until November 2015, with repeats through February 2016, and its second half of the season airing from 1 March 2016. SK Broadband, having participated in the production, provided the episodes on video on demand exclusively to subscribers of their IPTV platform B TV, about a half-hour following the South Korean broadcast of Season 2 of this show airing on American over-the-air television. A third season is also in development. On 22 January 2018, Zag posted on Instagram that the crew was working on a fourth and fifth season. On 31 August premiered a new series with Miraculous: Tales of Ladybug & Cat Noir which this time is in chibi style. On 5 September 2018, Skydance Media has acquired the live-action film and live-action television rights. Kimberly Cooper, a blog writer who has contributed to news media such as "The Huffington Post", wrote that the show has inspired teens and adults to create Channel on 1 November 2016. In the United Kingdom and Ireland, the show premiered on 30 January 2016 on Disney Channel The first series was also made available to Netflix subscribers in the UK. Among the free-to-air terrestrial television broadcasters in the Republic of Ireland, Raidió Teilifís Éireann premiered the show in 2018 on RTÉ2's teenage programming block, TRTÉ. In Australia, the Australian Broadcasting Corporation released the show on its ABC iview streaming service on 1 January 2016, and premiered the show on its linear channel ABC3 on 22 March 2016. In New Zealand, the show premiered on 27 April Summary: Miraculous Ladybug season 2 episode 12 is scheduled for release in December 2017 on Netflix in North America. The show premiered in South Korea on 1 September 2015 and has since been aired in various countries, including in the United States, the UK, and New Zealand.	Not faithful, Comprehensive

Table C13: Example summaries and their manual analysis labels (continued).

Dataset	Model	Query, Passages and Summary	Evaluation
HotpotQA	GPT-3.5	Question: The composer of the music for the ballet "The Seasons" was the director of what organization from 1905 to 1928? Passages: The Seasons (ballet) The Seasons (, ""Vremena goda""; also) is an allegorical ballet in one act, four scenes, by the choreographer Marius Petipa, with music by Alexander Glazunov, his Op. 67. The work was composed in 1899 and first performed by the Imperial Ballet in 1900 in St. Petersburg, Russia. The score for Marius Petipa's ""Les Saisons"" (""The Seasons"") was originally intended to have been composed by the Italian composer and conductor Riccardo Drigo, who was Glazunov's colleague and close friend. Since 1886, Drigo held the posts of director of music and ""chef d'orchestre"" to the Ballet of the harmonium, guitar and even mandolin). ""The Seasons"" was commenced shortly after the premiere of Tchaikovsky's First Piano Concerto, and continued while he was completing his first ballet, ""Swan Lake"". In 1875, Nikolay Matveyevich Bernard, the editor of the St. Petersburg music magazine ""Nouvellist"", commissioned Tchaikovsky to write 12 short piano pieces, one for each month of the year. Bernard suggested a subtitle for each month's piece. Tchaikovsky accepted the commission and all of Bernard's subtitles, and in the December 1875 edition of the magazine, readers were promised a new Tchaikovsky piece each month throughout 1876. The January and February pieces The Seasons (Cage) The Seasons is a ballet with music by John Cage and choreography by Merce Cunningham, first performed in 1947. It was Cage's first piece for orchestra and also the first to use what Cage later called the ""gamut"" technique, albeit in an early form. Cage composed the music in early 1947, in the midst of working on ""Sonatas and Interludes"". A piano version was first completed, and an orchestral arrangement followed. Cage dedicated ""The Seasons"" to Lincoln Kirstein. The ballet was premiered on May 17, 1947 by the Ballet Society (by which the work was commissioned) at a new orchestra version. Morton Gould retained the piano part for many of the pieces and orchestrated the work throughout, recording it with himself at the piano in 1951 for American Columbia. In 1965, Kurt-Heinz Stolze orchestrated a number of the pieces as part of the music for John Cranko's ballet ""Onegin"". More recent orchestral versions have been produced by David Matthews (for symphony orchestra), Peter Breiner (for solo violin and symphony orchestra), and Georgii Cherkov (for solo piano and symphony orchestra). French composer Philippe Sarde arranged the Barcarolle as a main theme for the 1988 movie ""The Bear"". Slava Cranko was not invited to direct, and Sir John Gielgud was brought in. Prosecuted for homosexual activity, Cranko left the UK for Stuttgart, and in 1961 was appointed director of the Stuttgart Ballet, where he assembled a group of talented performers such as Marcia Haydée, Egon Madsen, Richard Cragun, Birgit Keil and Suzanne Hanke. Among his following choreographies were ""Romeo and Juliet"" by William Shakespeare in 1962, set to music by Prokofiev, ""Onegin"" in 1965, an adaptation of the verse novel ""Eugene Onegin"" by Alexander Pushkin, set to music by Tchaikovsky (mainly ""The Seasons""), orchestrated by Kurt-Heinz Stolze, ""The Taming Summary: 1. The composer of the music for the ballet ""The Seasons"" is Alexander Glazunov. 2. Alexander Glazunov composed the music for the ballet ""The Seasons"" in 1899. 3. The ballet ""The Seasons"" was first performed by the Imperial Ballet in 1900. 4. From 1905 to 1928, the director of an organization was Marius Petipa. 5. Marius Petipa was the choreographer of the ballet ""The Seasons"". 6. Therefore, the composer of the music for the ballet ""The Seasons"" was the director of the Imperial Ballet from 1905 to 1928.	Not Faithful, Comprehensive

APPENDIX B

APPENDIX D: SUPPLEMENTARY MATERIAL FOR CHAPTER 4

8.0.6 IMPLEMENTATION DETAILS

COMPATIBILITY WITH FLASH ATTENTION FlashAttention [Dao 2024] substantially improves the efficiency of standard attention computation. It reduces data movements on GPU by directly producing the output for the attention blocks without storing the $O(L^2)$ attention matrix. However, we rely on these attention scores to select the top K tokens during the full attention steps and construct our partial KV cache C_p (lines 9-10 of Algorithm 4.2). To make our method compatible with Flash Attention, we implement an extra step to re-compute the attention score at the full attention step. As we do not perform full attention at every generation step, this does not introduce significant overhead. For methods that require accessing attention score (e.g. H₂O), we apply the same procedure to make them compatible with Flash Attention.

BASELINE SETTINGS For StreamingLLM, we follow the original paper and maintain a cache with 4 sink tokens and $K - 4$ recent tokens. For H₂O, we set the heavy hitter size and recent cache size to be $K/2$ each following [Zhang et al. 2024b]. For SnapKV, we set the observation window size to 1 and the kernel size to 7 for both RefreshKV and SnapKV following Li et al. [2024]. We apply the same aggregation method (max over all query heads) for SnapKV and H₂O for the GQA models.

CONTINUED PRETRAINING We randomly sample a subset of 200k sequences from the Arxiv split of RedPajama dataset³ and filter out sequences with less than 8192 tokens. We train Llama-3.1-8B for one epoch with a global batch size of 64 and a learning rate of 5e-6. We use 20 warm-up steps and a linear schedule with 0 weight decay. We use the AdamW Optimizer. We use Fully Sharded

³<https://huggingface.co/datasets/togethercomputer/RedPajama-Data-1T>

Dataset	# Example	# In	# Out
RULER	1.3K	32K	<30
QMSum	100	10K	0.1K
GovReport	100	10K	0.7K
NovelSumm	103	100K	1.0K
HTML To TSV (0.5K)	50	18K	0.5K
HTML To TSV (1K)	50	35K	1.6K
HTML To TSV (2K)	50	38K	4.6K
Chain of Keys	100	22K	50

Table D1: Dataset statistics. We report the number of tokens for both the input context and output generation for each dataset, as well as total number of examples.

	Vanilla	H ₂ O	StreamingLLM	SnapKV	RefreshKV (Ours)
Memory	L	K	K	K	$L + K$
Time	$T \times L$	$T \times K$	$T \times K$	$T \times K$	$T \times \frac{L}{S} + T \times K$

Table D2: Comparing memory (KV cache size for L input tokens) and time (attention computation for generating the next T tokens) of RefreshKV and baselines. We denote S as stride and use the same KV cache size (K) for the partial cache for our method and complete cache for eviction-based baselines.

Data Parallel [Zhao et al. 2023b] and 8-bit optimizer [Dettmers et al. 2021] to improve training efficiency. Training is done on 4 H100 80 GB GPUs.

8.0.7 MEMORY AND TIME REQUIREMENT COMPARISON

Table D2 compares the memory and attention compute requirements of **RefreshKV** with baselines. We report the memory required to store the KV cache for the L input tokens, and attention compute required to generate the next T tokens.⁴ We set our partial cache to be the same size as the complete cache of the eviction-based methods. Under this setting, RefreshKV requires larger KV cache memory compared to eviction-based baselines, but similar to vanilla attention ($L + K$ vs L , where $K \ll L$). However, our decoding latency is on par with the baselines. Our efficiency depends on two sets of hyperparameters – the partial cache size K , and QC stride and s ,

⁴The KV memory requirements also increases with T . We do not account for this in the table.

Method	Agg	Llama-3.1-8B	Qwen-2-7B
Vanilla	-	2.22/7.07	2.33/8.26
RefreshKV	First	2.34/7.43	2.49/8.78
RefreshKV	Mean	2.32/7.40	2.47/8.73
RefreshKV	Max	2.32/7.40	2.47/8.72

Table D3: Results comparing different methods to aggregate attention scores for GQA models. We experiment with taking the attention score of the first query head, the average and max attention scores of the query heads in the same group to select topK KV cache. For StreamingLLM and RefreshKV, we set $K = 1/8L$ and stride as 10.

which determines how often full attention is performed. By setting $K \ll L$ and a large S , we can achieve wall clock times similar to KV eviction-based baselines.

8.0.8 ATTENTION SCORE AGGREGATION FOR MODELS WITH GQA

We report language modeling results with different aggregation methods across attention scores of query heads in the same group for models with Grouped Query Attention in Table D3. We see that aggregating over the attention score of the entire group works better than using attention score of one of the head, with taking the max slightly outperforming mean.

8.0.9 TUNING s FOR QUERY SIMILARITY SCHEDULE

To choose a similarity threshold s for the dynamic schedule, we run RefreshKV on a held-out set of 50 examples from the Book split of the RedPajama dataset. We evaluate on QC stride of {5, 10} with threshold s of {0.80, 0.85, 0.90, 0.95} for Llama-3.1-8B and Qwen2-7B.

Table D4 reports the results of different settings for perplexity and decoding time measured on one A100 machine with batch size of 1. We can see that for Llama-3.1-8B, setting a threshold of 0.85 achieves similar performance for both stride compared to 0.90 and 0.95. In contrast the performance of Qwen2-7B continues to increase going from threshold of 0.80 to 0.95. Therefore, we set the threshold to 0.85 for Llama-3.1-8B and 0.95 for Qwen2-7B.

Method	QC stride	s	Book PPL	Time
<i>Llama-3.1-8B</i>				
Vanilla	-	-	6.70	7.54
RefreshKV	5	0.80	6.92	6.42
RefreshKV	5	0.85	6.86	6.64
RefreshKV	5	0.90	6.88	7.01
RefreshKV	5	0.95	6.88	7.53
RefreshKV	10	0.80	6.95	6.37
RefreshKV	10	0.85	6.96	6.52
RefreshKV	10	0.90	6.96	6.54
RefreshKV	10	0.95	6.95	7.07
<i>Qwen-2-7B</i>				
Vanilla	-	-	7.44	9.11
RefreshKV	5	0.80	7.86	6.50
RefreshKV	5	0.85	7.80	6.64
RefreshKV	5	0.90	7.73	6.91
RefreshKV	5	0.95	7.66	7.14
RefreshKV	10	0.80	7.95	6.37
RefreshKV	10	0.85	7.87	6.41
RefreshKV	10	0.90	7.84	6.62
RefreshKV	10	0.95	7.82	6.67

Table D4: Results of different similarity threshold s on the held-out set of the Book dataset across two QC stride.

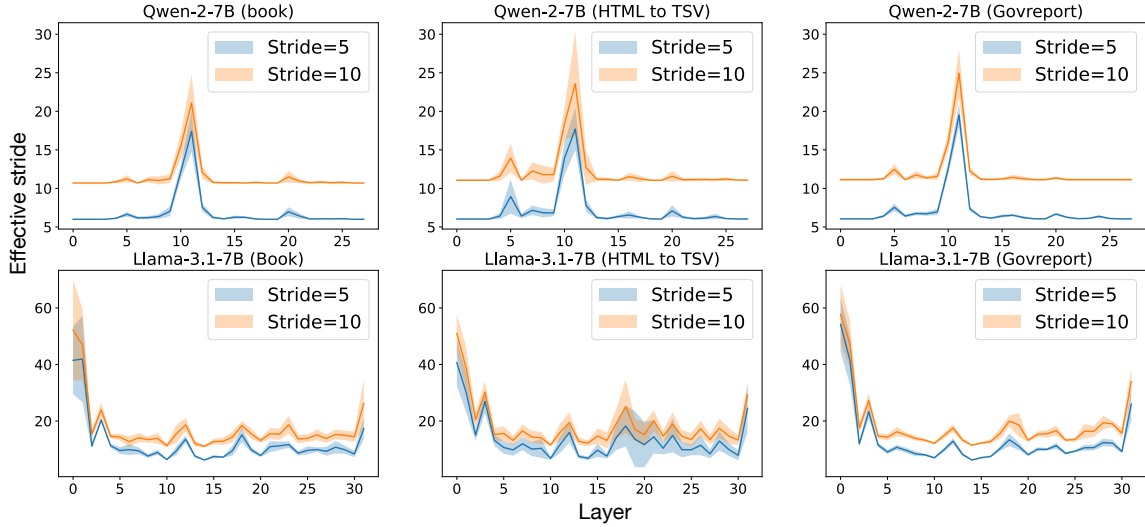


Figure D1: Effective stride across layer for Llama-3.1-8B (similarity threshold=0.85) and Qwen2-7B (similarity threshold=0.95) in three datasets. We sample 10 examples from each dataset to estimate the effective stride.

8.0.10 EFFECTIVE STRIDE

We plot the effective stride across layers for Llama-3.1-8B and Qwen2-7B in Figure D1 for the three tasks reported in Table 4.3. Leveraging query similarity enables dynamic strides across layers for both models. We observe distinct pattern for the two models, with Llama-3.1-8B having a larger stride in the first few layer and Qwen2-7B in the middle layer. We also observe slightly different patterns for different tasks, showing that our method enables flexible scheduling based on the context.

8.0.11 CHAIN-OF-KEY TASK SET-UP

TASK SET-UP The model is provided with a long lists of keys, each of which contains W number of words, for instance: apricot-waggish where $W = 2$. The model is tasked to generate a sequence which consists of a list of T keys from the context, such that the first word of the next key is the last word of the current key. For example: waggish-fishery, fishery-mosquito, mosquito-perfume, perfume-panda, panda-juice for $T = 5$. We provide an example input in Table D6.

Method	Stride	0.5K	2K	8K	Aggregated
<i>Llama-3.1-8B</i>					
Vanilla		43	31	23	33
Streaming		4	1	0	2
SnapKV		0	0	0	0
H2O		0	0	0	0
Refresh	QC=5	31	15	4	17
Refresh	QC=10	16	7	1	8
<i>Qwen-2-7B</i>					
Vanilla		36	22	15	24
Streaming		10	3	0	5
SnapKV		0	0	0	0
H2O		0	0	0	0
Refresh	QC=5	20	6	3	10
Refresh	QC=10	14	2	1	6

Table D5: Breakdown of HTML tasks based on output length.

DATA GENERATION We first generate a list of English words. We then pair each word with another word to form a list of keys. We ensure that for each key k_1 in the context, there exists exactly one other key k_2 that satisfies the constraint (i.e. the first word of k_2 is the last word of k_1). The keys are randomly shuffled in the context.

EVALUATION We evaluate correctness of the generated output by the length of a valid chain, divided by T . A valid chain needs to satisfy two criteria: (a) all the key must be in the context and (b) the first word of the current key must be the last word of the previous key. We provide example outputs and their correctness score in Table D7.

8.0.12 RESULTS ON LONGPROC TASKS WITH SHORT INPUTS

TASK SET-UP We report results on 4 more tasks from **LongProc** [Ye et al. 2025]: **Path Traversal**, **Travel Planning**, **Countdown** and **Theory-of-mind tracking**. These tasks consist of input with less than 10K tokens. While **Path Traversal** consists of a version with 12K input tokens, we exclude it from our main results as none of the open sourced models are able to perform the task in

Input

"You are given many keys composed of a few words. Your task is to generate a chain of 10 keys such that the first word of the current key is the last word of the previous key. Separate the keys with comma. Example: waggish-fishery, fishery-mosquito, mosquito-perfume, perfume-panda, panda-juice, juice-willow, willow-bronco, bronco-creditor, creditor-bathhouse, bathhouse-woman. You must generate keys that are in the context. DO NOT REPEAT THE EXAMPLE.

Context:Name of key: toga-roommate

Name of key: appetiser-cenario

Name of key: normalization-tacit

Name of key: intensity-ping

Name of key: innate-cummerbund

Name of key: tentacle-lining [...omitted...]

Name of key: breath-yielding

Name of key: schema-festive

You are given many keys composed of a few words. Your task is to generate a chain of 10 keys such that the first word of the current key is the last word of the previous key. Separate the keys with comma.You must generate keys that are in the context. Chain of ten keys:"

Table D6: Example input for the chain-of-key task where $W = 2$ and $T = 10$.

Output	Score
impossible-crawdad, crawdad-vehicle, vehicle-uncertainty, uncertainty-welfare, welfare-outrigger, outrigger-historical, historical-gator, gator-hugger, hugger-debris, debris-precious	1 (fully correct)
annoying-pentagon, pentagon-fit, fit-waggish , waggish-fishery, fishery-mosquito , mosquito-perfume, perfume-panda, panda-juice , juice-willow, willow-bronco	0.2 (correct up to the second key)
impossible-crawdad, crawdad-vehicle, vehicle-uncertainty, welfare-outrigger, outrigger-historical, historical-gator, gator-hugger, hugger-debris, debris-precious, uncertainty-welfare	0.3 (correct up to the third key)

Table D7: Example output for the chain-of-key task where $W = 2$ and $T = 10$ and their score. Keys that are not in the context are highlighted in **red**.

vanilla setting. We report results on 50 samples for each task. We set $K = 1/8L$ for RefreshKV and baselines.

Method	stride	Path Traversal	ToM Tracking	Countdown	Travel Planning
<i>Llama-3.1-8B</i>					
Vanilla	-	17	40	67	62
StreamingLLM	-	0	0	0	0
H ₂ O	-	0	0	0	2
SnapKV	-	1	0	12	0
RefreshKV	QC=5	5	14	44	38
RefreshKV	QC=10	1	5	42	18
<i>Qwen-2-7B</i>					
Vanilla	-	7	12	11	48
StreamingLLM	-	2	0	6	0
H ₂ O	-	2	0	6	0
SnapKV	-	0	0	14	2
RefreshKV	QC=5	3	6	14	26
RefreshKV	QC=10	2	2	10	4

Table D8: Performance on long-context tasks with short outputs from LongProc benchmark for LLaMA-3.1-8B-Instruct and Qwen-2-7B-Instruct.

EVALUATION We follow evaluation practice of the original paper []. For Countdown and Travel Planning, we report correctness of the final solution using rule-based validators. For Path Traversal and ToM Tracking, we report accuracy.

RESULTS Results of RefreshKV and baseline methods are in Table D8. We observe similar trend as the **HTML to TSV** task – Most of the baselines fail completely on the task. RefreshKV with $QC = 5$ recovers approximately 50% and 60% performance of full attention for Llama-3.1-8B and Qwen2-7B respectively.

8.0.13 DETAILED RULER RESULTS

We follow the suite of evaluation tasks introduced in [Hsieh et al. 2024], which consists of the 13 tasks.⁵ We refer the readers to Hsieh et al. [] for detailed description and examples of each task and Appendix B for the exact tasks configurations. We group them based on the types:

- **Single NIAH** An NIAH-styled task with one key and one value to retrieve. We include three

⁵<https://github.com/hsiehjackson/RULER>

variations of the task with different types of key, value and haystack.

- **Multi-key NIAH** An NIAH-styled task with distracting keys. We include three variations of the task with different types of key, value and haystack.
- **Multi-value NIAH** An NIAH-styled task with multiple values corresponding to the key.
- **Multi-query NIAH** An NIAH-styled task with multiple queries, each corresponding to a distinct key.
- **Variable Tracking** A NIAH-styled task that requires tracing through multiple hops.
- **Common word extraction** and **Frequent word extraction** require extracting the words based on the pattern in a list of words. Common word extraction expects a list of 10 most common words while frequent word extractions expect a list of 3 frequent words.
- **Question Answering** A task that requires answering a question given a set of documents. We include two variations of the tasks, corresponding to two question answering datasets.

PER-TASK RESULTS We report detailed performance of RULER subtasks in Table D9, grouped by task type. For both models, the best baselines (SnapKV) achieves comparable results as RefreshKV for tasks with short-form outputs, such as **Single NIAH**. However, for tasks that require longer outputs, such as **Multi-key** and **Multi-value NIAH**, RefreshKV outperform all the baselines.

Method	niah_single	multi_key	multi_query	multi_value	fwe	vt	cwe	qa
<i>Llama-3.1-8B</i>								
Vanilla	100	98	99	99	93	99	65	61
H ₂ O	7	7	6	6	78	38	39	34
Streaming	8	13	13	13	93	12	4	42
SnapKV	99	60	98	99	83	99	44	63
RefreshKV(QC=5)	100	91	98	99	81	99	44	60
RefreshKV(QC=10)	100	67	97	99	81	99	44	59
<i>Qwen-2-7B</i>								
Vanilla	100	90	75	87	84	86	27	50
H ₂ O	5	8	5	3	84	2	17	30
Streaming	8	11	13	12	80	15	14	39
SnapKV	69	51	54	43	81	87	27	50
RefreshKV(QC=5)	99	79	70	85	70	87	27	50
RefreshKV(QC=10)	97	54	63	67	80	87	27	49

Table D9: Detailed performance of RULER subtasks with $L = 32K$. For non-vanilla methods, we set the $K = 1/8L$.

Question: Why is it so important for humans to have a balanced nutrition but not for animals?

Most animals have a fairly simple diet, carnivores eat only meat their whole life, cows eat exclusively grass etc. So why are human bodies so picky and need a balance of protein, fat, carbs etc from different sources to perform well?

Answer A

Your question includes a few common misconceptions, one that's been addressed already. Humans need a balanced diet, but that doesn't necessarily imply a vast array of food types. That simply means the nutritional value of our food has to cover our needs. It doesn't have to be varied. You can very well thrive by eating the same handful of things every day.

Answer B

The human body is much more complex than the bodies of most animals. Humans need a balance of nutrients in order to function properly. Protein, fat, and carbohydrates are all essential nutrients that the body needs in order to function.

Animals generally have a simpler diet than humans. For example, carnivores only eat meat, while cows only eat grass. However, the human body is much more complex and needs a variety of different nutrients to function properly. A balanced diet is essential for humans in order to maintain good health.

Which one of Answer A and B is better to the question?

☒ Answer A^[1] ☐ Answer B^[2]

Is it difficult to decide between the two answers because of their quality (e.g., they are similarly good or bad)?

☐ Yes.^[3] ☒ No.^[4]

Your explanation:

Please explain your choice by providing explicit comparisons of Answer A and B on the following aspects (if relevant). You can add comparisons of other aspects if they are not listed below. When providing quotations from the answers, **please also note where the quotation is from** (i.e., Answer A or B).

- If it is hard to choose between Answer A and B, please note the reasons here.
- Does the question contain **false assumptions**? If so, do Answer A and B point these out and correct them?
- Do Answer A and B **consider all aspects** asked in the question? If not, please note which aspect in the question is not addressed in Answer A or B.
- Does Answer A and B contain **fatal grammar mistakes** that make the answer hard to understand? If so, please provide quotations from the answer(s) that are ungrammatical.
- Are Answer A and B **well formulated** and **easy to follow**? If not, please provide quotations from the answer(s) and explain why.
- Do Answer A and B **contain irrelevant information**? If so, please provide quotations from the answer(s) and explain why they are irrelevant.
- Do Answer A and B **contain factually incorrect information or misconceptions**? If so, please (1) provide quotations from the answer(s); (2) explain why it's incorrect; and (3) provide a reference (e.g., link to a web page) if available.

Figure A1: Screenshot of annotation interface for collecting expert evaluation.

APPENDIX C

APPENDIX A: SUPPLEMENTARY MATERIAL FOR CHAPTER 5

8.0.14 EXPERT ANNOTATION

QUESTION CLUSTERING Four domains (biology, physics, chemistry, and economics) are marked in the ELI5 posts (i.e., flairs), and two (tech/cs and law) are identified by using a dense passage retrieval [Karpukhin et al. 2020a] and KMeans from scikit-learn [Pedregosa et al. 2011]. Specifically, we use DPR to encode question of all posts whose flair is marked as *others*. Then, we run KMeans to find two big groups of questions whose domains can be reliably marked as tech/cs and law.

ANNOTATORS Experts are hired based on their academic background and English proficiency. No other demographic and geographic restrictions were applied. For each question domain, we aimed to hire three domain experts who have at least a bachelor’s degree in the domain through a paid pilot study. Thirty-five potential experts participated in a paid pilot study with 5 question-answer pairs. We paid \$3 per question-answer set. At the end, only 13 experts met the qualification requirements and were willing to continue because the task required substantive expertise as well as time and attention commitment.

Category	Question		Model		Human	
	Median	Mean (std)	Median	Mean (std)	Median	Mean (std)
Biology	20.50	49.40 (60.54)	74.00	75.70 (21.08)	56.00	79.20 (57.20)
Physics	25.00	31.85 (18.70)	70.50	75.10 (27.06)	55.50	88.77 (82.91)
Chemistry	38.50	44.90 (29.13)	60.50	90.10 (92.79)	101.00	124.43 (77.59)
Economics	36.50	39.70 (30.93)	104.50	109.50 (50.75)	66.00	88.80 (93.21)
Law	21.50	27.30 (19.38)	111.50	126.90 (75.31)	72.50	115.83 (146.48)
TechCS	21.50	35.10 (35.12)	91.00	94.90 (40.67)	105.00	112.43 (58.99)
History	48.50	65.70 (57.87)	72.00	84.53 (58.24)	68.00	158.08 (168.97)
All	27.50	41.99 (41.01)	75.00	93.20 (59.93)	75.00	108.47 (106.56)

Table A1: Statistics of the text length in our expert-annotated dataset. For each category, there are 20 questions. Each question has either a pair of human-written answers (H/H) or a pair of human-written and model-generated answers (H/M). The domain of history has 15 questions in the H/M setting and 5 in H/H. The other six domains have 10 questions in each setting. There are 140 questions, 205 human-written answers, and 75 model-generated answers.

8.0.14.1 JUSTIFICATION ANALYSIS

Data statistics of explanations collected are in Table A2. Examples of explanation and extracted aspects in our manual analysis can be found in Table A3.

PREPROCESSING To construct the masked comments, we first preprocess the justifications such that all mentions of the answer entity is prepended with the word “Answer” (i.e. replacing “Option A”, “A” with “Answer A”). We then mask out any mentions of “A” and “B” in the comment. We

Split	# data	Avg. # word	Avg. # span
Expert	259	174	5
WEBGPT	292	46	3

Table A2: Data statistics for computational analysis of free-form justifications. The span refers to the masked reference of candidate answer in the justifications.

remove comments that do not contain answer entities after preprocessing, resulting in 259 (out of 260) expert comments and 292 (out of 305) WEBGPT comments.

8.0.14.2 EXAMPLE OF COMMENTS MENTIONING DIFFERENT ASPECTS FOR SECTION 5.3.3

See Table A5.

8.0.15 AUTOMATIC METRIC IMPLEMENTATION DETAILS

Length statistics of the answers evaluated in 5.4.1 are reported in Table A8. We truncate the input if it exceeds the context window for the model. Less than 5% of the comparison data are truncated.

ROUGE-L For each answer, we calculate ROUGE-L against the set of reference answers from ELI5 and use the maximal ROUGE-L.

BERTSCORE We use the default roberta-large model for English⁶ and report the maximal F1 BERT score against the set of reference answers.

BLEURT We use the BLEURT-20 checkpoint as recommended and report the maximal BLEURT score against the set of reference answers.

⁶https://github.com/Tiiiger/bert_score

Aspect	Source	Comments
Factuality	Expert	[...] Answer B contains some incorrect information regarding the humans being more complex than animals and repeating same points twice. [...]
Factuality	WEBGPT	A claims pi bonds are the weakest, which its sources don't state , only calling them weaker than sigma bonds. A is also a little repetitive. B is much easier to follow and much simpler to understand.
Easy to understand	Expert	[...] Of course, there is more to inflation than is provided by answer B, but it is concise, factual, and easy to understand for someone that does not have a background in economics . [...]
Relevance	Expert	For this question, Answer A is far better choice as it has accurate and scientific information relevant to the question . While answer B has irrelevant information by mentioning his personal experience of controlling the darkness which is totally over simplified statement. [...]
Well-structured	Expert	[...] However, I decided that Answer B has provided more details and is more well-structured compared to Answer A . [...]
Completeness	Expert	For this question, answer B is better choice as it covers all aspects of the questions and explains the whole process with scientific facts. While answer A contains incomplete information which cannot clear the doubts of reader. [...]
Grammar	Expert	I believe option "A" is the better choice as it explains the meaning of a filibuster. Option B lacks formal writing and even states the words, "to shut him up". [...]
Example	Expert	Both answers state the same information almost word for word. However, answer A provides a clearer example for people who may not have experience in biology . [...]
Specificity	Expert	For this question, it is difficult to decide which is better option because both the answers are not up to the mark to clear the concept. Still, answer A seems better option as it describes the process in detail and mentioning some hormones that involves in the process. [...]
Conciseness	WEBGPT	A is easier to follow, much more concise , and answers two possible interpretations of the question - the word's definition and the economic idea. B is overly detailed and needlessly argues with the use of austerity. A is much better.

Table A3: Free-form justifications written by experts and their corresponding aspects.

SELF-BLEU We calculate Self-BLEU by regarding one sentence as hypothesis and all others in the same answer paragraph as reference. We report self-BLEU-5 as a measure of coherence.

LENGTH We use the Stanza toolkit [Qi et al. 2020] for word tokenization.

Domain	Question	Answer A	Answer B	Expert 1	Expert 2
Physics	How does a speaker vibrate at multiple frequencies simultaneously to deliver sounds to our ears?	This has been asked many times and the answer is they don't. If you listen to the song being played live on purely acoustic instruments even though they are being played separately and emitting their own frequencies, what you hear (and by extension, what a microphone captures) at any given time is just ONE frequency that's the "sum" of all the others combined. A speaker is just a reverse microphone.	Imagine an ocean with a consistent wave. It flows up and down, with equal distance between the two waves at any time. Now imagine I push a larger, shorter wave into this ocean. The two waves will collide, resulting in some new wave pattern. This new wave pattern is a combination of those two waves. Speakers work similarly. If I combine two sound waves, I get a new combination wave that sounds different.	Justifications: In technical terms, ocean waves stated in answer B are transverse waves and sound waves are longitudinal waves. [...] So in terms of technical approach and considering overall accuracy answer A is selected. Preference: A Aspects: Factuality	Justifications: I go with answer B because I like the analogy with the ocean waves , and due to how visual the explanation is it is easier to understand in my opinion. [...] Preference: B Aspects: Example, Easy-to-follow
Economics	What are toll roads or "turn-pikes" used for?	There is a toll road near my house. It was built by a private company on private land. It represents a significant short-cut between two state-built roads. The company built the road, and collects tolls for its use, in an effort to make money. So, in this case, the toll goes to repay the initial investment to acquire the land and build the road. It also goes toward providing snow removal, maintenance, and repair.	The money from the tolls usually goes towards maintaining the road, and/or is like a travel tax that the state can use for other projects. Sometimes tolls are only there to cover the cost of constructing the road and then once it's paid off they take the toll off (that's less common though).	Justifications: Answer A better answers the original question, although both could have gone into more detail. [...] Answer A does address the latter point . Preference: A Aspects: Completeness	Justifications: [...] Answer A explains only a single use case of the use of tolls while the question demands a broader answer which is mentioned in Answer B . [...] A contains irrelevant information . [...] Preference: B Aspects: Completeness, Relevance
Biology	How are sperms made in the body? More specifically, how does a Sperm become X or Y sperm.	Not going too much into the scientific details, mainly because I don't remember them from high school 2 decades ago, but basically the cell division that occurs to form the reproductive cells (sperm/eggs) in humans is not the same as the cell division for non-reproductive cells. When the "normal" cells split, they create complete copies of each chromosome pair (your DNA is made of pairs of each chromosome. One comes from the father, one from the mother), so the child cells end up with a complete set of DNA. Reproductive cells split the chromosome pairs. The child cells only receive one chromosome from each pair. In the case of the sex chromosome pair, a male has an XY pair and a female has an XX pair. So when a male creates reproductive cells (sperm), one sperm will receive the X chromosome and the other will have the Y chromosome.	When most cells divide, they copy themselves once then split once, so both copied cells have X "and" Y. Sperm basically come from 1 copy but 2 splits, so you get one X sperm and one Y sperm from the second split.	Comments: Both answers state the same information almost word for word. However, answer A provides a clearer example for people who may not have experience in biology . [...] Preference: A Aspects: Example	Comments: Answer B doesn't distinguish between men and women which is pertinent in this question. Answer B lacks detail to make the answer clear . [...] Answer A has a better flow, is more comprehensive and better answers the question." Preference: A Aspects: Detailed, Easy to follow

Table A4: Example annotations by domain experts comparing long-form answers, either generated from GPT3 or human written, showing their preferences, free-form justifications and aspects. The first two examples illustrate that experts disagree with each other because they value different aspects.

Aspect	Question	Answer A	Answer B	Comment
Factuality	Does your body absorb all blood clots? How does it do it? Does location affect it?	(human) Kind of. Your blood will dissolve clots over time. I've got a clotting disorder and need to take thinners to make that happen faster, but for most they break down clots naturally settlement on how much blood flow there is to an area and if there's any muscle action to help break them up. [...]	(machine) No, your body does not absorb all blood clots. Clots are usually broken down by the body's natural clotting mechanism, but some clots may persist and cause problems. Location can affect how quickly a clot is broken down. For example, clots in the veins are usually broken down more slowly than clots in the arteries.	(expert) [...] Answer A mentioning some inaccurate and irrelevant information, like role of blood thinner which is not correct as they cannot break down the clots. [...]
Completeness	Why do people go bald on the top of their head and still have some on the sides and not vice-versa?	(machine) There are a few reasons why people go bald on the top of their head and still have some hair on the sides. One reason is that the hair follicles on the top of the head are more sensitive to a hormone called dihydrotestosterone (DHT). DHT is [...] found in both men and women. [...] Another reason is that the hair on the sides and back of the head is not as exposed to the sun and other environmental factors, [...]	(human) Seems unsettled but here's a theory: "The most recent hypothesis suggests that the hair loss process begins during puberty, when growth of the skull and the muscles in the forehead and neck increases the tension in a tight band of tissue stretching over the top of the head. The more DHT (a type of testosterone) there is, the more these muscles contract, which causes inflammation and brings in more DHT. [...]"	(expert) It is difficult to decide but I believe answer A is better choice as it describes both the hormonal and environmental causes behind the baldness of top. And Answer B only focuses on one theory which might not be 100 percent accurate. [...] According to research, baldness is due to genes. In 95 percent cases, balding is due to androgenetic alopecia, more commonly known as male pattern baldness, which is hereditary condition. [...]
Specificity	"Why do solar systems look mostly 'flat'? I mean like, it seems as though all the planets are rotating on roughly the same axis. Why is there little to no variation?"	(machine) The solar system is not actually a perfect plane, but it is well on its way to an ideal stage. Pluto actually lies at an awkward incline, due to its low gravity and isolated orbit. The asteroid belt, in its most oblique locations, also inclines at about 20 degrees from the main plane. Aside from these few exceptions though, the power of angular momentum has left the planets of our solar system in a tidy, predictable plane. The reason for this is that [...]	(human) The solar system is flat due to the law of conservation of total angular momentum. This law states that the total amount of spinning in any isolated system will always remain constant. This is what caused the solar system to flatten as time went on and becoming a flat disc.	(WebGPT) B is overly brief and simplistic, not explaining its concepts well enough for them to be understandable. A is longer, but it is very useful, interesting, and detailed. It answers the question about the solar system being flat thoroughly and coherently. It is a much better answer than B.
Use of examples	(human) If under anesthesia do you get your sleep needed for the day?	No. The unconsciousness induced by both IV anesthetics and inhaled anesthetics is very different from restorative sleep. [...] True ELI5: Sleep = charging your phone on airplane mode. Anesthesia = turning your phone off but not charging it. [...]	(machine) There is no clear consensus on whether or not people under anesthesia get the sleep they need for the day. Some studies suggest that people under anesthesia do get some sleep, but it is not clear how deep or restful this sleep is. Other studies suggest that people under anesthesia do not get any meaningful sleep at all. Therefore, it is difficult to say definitively whether or not people under anesthesia get the sleep they need for the day.	(expert) For this question, answer A is better option because it has proper explanation of relevant information which is being asked in the question. [...] In answer A, relevant detail of the answer is explained using the example of equating sleep to charging the phone and anesthesia to turning it off but not charging which clear the idea about this concept. [...]

Table A5: Examples of some of the aspects that occur in the manual analysis described in Section 5.3.3.

QG LIKELIHOOD Given a question q and an answer paragraph a , we estimate $p(q|a)$ by computing the average log-likelihood of the question tokens conditioned on the passage using T0. Following previous work [Sachan et al. 2022], we append a natural language instruction “Which question does this passage answer?” to the answer, denoted as a' .

$$\log p(q|a) = \frac{1}{|q|} \sum_t \log p(q_t | q_{<t}, a'; \Theta)$$

where Θ denotes the parameter of the language model and $|q|$ denotes the number of tokens in the question.

BARTSCORE We use the BART model finetuned on the CNN/DM dataset (facebook/bart-large-cnn).

RANKGEN Given a question q and an answer paragraph a , we first encode them through the RankGen encoder, which projects them to fixed-size vectors (q, a) . We then determine their relevance by calculating the dot product between the two vectors $q \cdot a$. We use the T5-XXL (11B) encoder trained on both in-book negative and generative negatives.

QAFACTEVAL QAFactEval [Fabbri et al. 2022] is a recently proposed QA-based metric that has shown superior performance on several summarization factuality benchmark [Laban et al. 2022; Maynez et al. 2020]. The pipeline is carefully chosen from extensive experiments on various combinations of components in the QA-based metrics. The final pipeline consists of (1) NP from S as $Ans(S)$ (2) BART-large [Lewis et al. 2020a] as Q_G (3) Electra-large [Clark et al. 2020] as Q_A and (4) learned metrics **LERC** [Chen et al. 2020] as $Sim(p_i, s_i)$. They further include an answerability classification module to determine if the question is answerable given the document D . We report the **LERC**, which uses the learned metrics to compare Ans_S and $Ans_D(a)$ and shows better performance compared to other metrics in our initial experiments.

Category	# QA pairs	Consistency	Accuracy
Biology	11	100%	82%
Physics	13	100%	62%
Economics	12	92%	83%
History	13	100%	100%

Table A6: Performance of 2 shot question answer evaluation using GPT3 text-davinci-003. Consistency reports the percentage of the model generate the same preferred answer across three API calls. Accuracy compares the majority votes among the three API calls against the human preference.

Split	# data	# non-tie data
train	12,318	9,153
dev	2,640	1,989
test	2,640	1,923
total	17,598	13,065

Table A7: Data statistics for human preference data used to train and evaluate the learned metric (Section 5.4.1.2). We collapse human rating such that the answer preferred is assigned score 1 and the other 0. Nakano et al. [] included tie data and assign them 50% soft labels, and excluded them from evaluation. However, we didn’t find them beneficial for model training and hence removed them from both training and valuation.

Answer Type	# answer	$ q $	$ a $	$ d $	$ j $
WEBGPT HUMAN	254	35	112	264	46
WEBGPT MODEL	6,095	35	137	328	
HURDLES HUMAN	442	17	300	-	-
HURDLES MODEL	1,135	17	182	-	
EXPERT HUMAN	205	42	108	-	176
EXPERT MODEL	75	42	93	-	

Table A8: Data statistics of answers compared in the human evaluation data. The number of comparison data can be found in Table 5.4. $|q|$, $|a|$, $|d|$ and $|j|$ represent the average number of words for question, answer paragraph, retrieved documents and justification. For WebGPT, justifications are only on a subset of comparison data. WebGPT and expert annotation data take both the title and the description of the reddit post as question following [], whereas Hurdles data only considers the title as question (hence shorter $|q|$).

Data	# data	# non-tie data	Aspect
HURDLES (human v.s. model)	486 / 214 / 194	419 / 164 / 151	Overall / Coherence / Factuality
HURDLES (model v.s. model)	521 / 262 / 260	370 / 195 / 169	Overall / Coherence / Factuality
WEBGPT (human v.s. model)	761 / 761 / 590	637 / 496 / 149	Overall / Coherence / Factuality
WEBGPT (model v.s. model)	17,598	13,065	Overall

Table A9: Data Statistics for human **comparison** evaluation data for each aspect. In all studies, overall score was mandatory but coherence / factuality scores were optional and hence the number of evaluation data available varies among different aspects. All human evaluation data is one-way annotated.

8.0.15.1 LEARNED METRICS

We use pytorch-transformers [Wolf et al. 2019] to implement our models. We use Quadro RTX 8000 GPUs to train our model.

LONGFORMER We use longformer-base, consisting of 149M parameters. The training batch size is set to 16, with the initial learning rate as $1e - 5$. We used AdamW optimizer and a linear learning rate schedule. We train the model for 5 epochs and report the result of the checkpoint with best validation accuracy. The training takes less than 5 hours with 4 GPUs.

GPT3 We use the API to fine-tune the model with a batch size of 64 and a learning rate multiplier 0.05 for six epochs. Fine-tuning text-curie001 model for each epoch on OpenAI cost \$11. We did not use the larger text-davinci-002 model, which would have cost \$110 per epoch.

8.0.16 PREVIOUSLY COLLECTED HUMAN EVALUATION DATA

Dataset statistics is shown in Table A9. We group the comparisons by whether they are (model-generated answers v.s. human-written answers) or (model-generated answers v.s. model-generated answers), and present overall statistics. The model-generated answers include four different set-ups from HURDLES (combination of nucleus sampling $p=\{0.6, 0.9\}$, and generation conditioning on {predicted, random} passages) and three different set-ups from WEBGPT. The

human-written answers are gold answers from the ELI5 subreddit for comparison with HURDLES answers, and human demonstrations for WEBGPT answers.

8.0.16.1 LFQA SYSTEMS

We describe the different LFQA systems developed by prior works, which are included in comparisons used for evaluating automatic metrics in Section 5.4.

HURDLES Krishna et al. [2021] presented a state-of-the-art LFQA system which includes a passage retriever [Guu et al. 2020] and an answer generation model [Roy et al. 2021].

WEBGPT Nakano et al. [2021] proposed to fine-tune GPT-3 [Brown et al. 2020a] to interact with a search engine and compose long-form answers based on the information found. The generated answers also contain a set of reference documents found online.

8.0.16.2 EVALUATION ASPECTS

We describe the different evaluation aspects conducted by prior human evaluation.

OVERALL Krishna et al. [2021] phrased the question as “Which generation answered the question better / was more relevant to the question?” while Nakano et al. [2021] developed detailed instructions with intermediate steps for comparing two answers, and dedicated an overall rating, phrased as “how useful the answer would be to the person asking the question, all things considered”.

COHERENCE Krishna et al. [2021] asked the human evaluators to choose the more coherent answer and listed repetition as a trait of incoherence.⁷ In Nakano et al. [2021], the instruction for coherence evaluation focuses on whether the answer makes sense, is easy to follow and is in a logical order.

⁷The wording was (which answer) “was more coherent / had less repetition”.

FACTUALITY Krishna et al. [2021] instructed human evaluators to judge factual correctness of answers, with no accompanying evidence documents but permission to use search engine over Wikipedia articles. In Nakano et al. [2021], the evaluation of factuality is focused on whether the generated answer could be entailed by the evidence documents and that it doesn’t hallucinate unsupported fact. Note that “faithfulness” to the evidence articles is a different notion from the “correctness” of the answer, as the evidence articles might not always be correct or up-to-date [Gao et al. 2022a].

8.0.16.3 GPT-3 Two-SHOT

We conduct a pilot study on prompting GPT3 `text-davinci-003` for the pair-wise answer evaluation task on a subset of our expert annotation data.

For each domain that has multiple experts (i.e., biology, physics, economics, and history), we evaluate on the questions for which all experts agreed on the label of the preferred answer. We randomly choose two question-answer sets as the in-context example and prompt the model on the rest of the question-answer sets. The prompt has the following format:

QUESTION: q

ANSWER1: a_1

ANSWER2: a_2

TASK: Choose the better answer.

BETTER ANSWER: ANSWER1 (or ANSWER2) is better.

For each question-answer set, we sample three times with top $p = 1$ and temperature = 0.7 to evaluate model’s consistency. The results are reported in Table A6.

Results are report in Table A6. The model is mostly self-consistent. Model also aligns with human on this small set of data where human have perfect agreement with each other, model aligns with human performance, despite variance across different domains. We leave further

investigation on utilizing large language model for automatic evaluation on long-form question answering to future work.

APPENDIX D

APPENDIX B: SUPPLEMENTARY MATERIAL FOR CHAPTER 6

8.0.17 IMPLEMENTATION DETAILS FOR THE INTERACTION SYSTEM

We implement the annotation interface with [gradio](https://www.gradio.app/)⁸. Figure B2 presents a screenshot of our annotation interface. We use the OpenAI API⁹ for gpt-family model inference and the Together AI API¹⁰ for llama2. Below are the prompt templates we use.

PROMPT FOR GENERATING INITIAL ANSWER We include the top 5 retrieved passages from the document set to generate the initial answer. `title_i` and `passage_i` refers to the title and text of the *i*-th retrieved passages using `question` as the query.

SYSTEM MESSAGE: You are a helpful assistant which answers a question based on a given set of documents. Please add reference (e.g. [0]) to the document in the answer.

USER MESSAGE: Documents:

[0] Paper title: `title_0`

Passage: `passage_0`

[1] Paper title: `title_1`

Passage: `passage_1`

[2] Paper title: `title_2`

Passage: `passage_2`

⁸<https://www.gradio.app/>

⁹<https://platform.openai.com/docs/api-reference>

¹⁰<https://docs.together.ai/reference/inference>

[3] Paper title: title_3

Passage: passage_3

[4] Paper title: title_4

Passage: passage_4

Question: question

Answer:

PROMPT FOR ANSWER REVISION OF INFORMATION SEEKING INSTRUCTIONS We first prompt the model to generate a summary s_t given the previous answer (a_{t-1}), the retrieved passages and the instruction i_t .

SYSTEM MESSAGE: You are a helpful assistant which generates an intermediate answer given a feedback to the previous answer. Please add reference (e.g. [0]) to the document in the answer.

USER MESSAGE: Documents:

[0] Paper title: title_0

Passage: passage_0

[1] Paper title: title_1

Passage: passage_1

[2] Paper title: title_2

Passage: passage_2

[3] Paper title: title_3

Passage: passage_3

[4] Paper title: title_4

Passage: passage_4

Previous answer: a_{t-1}

Feedback: i_t

Generate an answer to the feedback.

We then prompt the model to generate an updated answer, given the original question, the answer in the previous turn a_{t-1} , the instruction i_t and the summary s_t . For llama2 , we add an additional instruction at the end “Do not generate "Sure", directly generate the updated answer. Updated answer: ”.

SYSTEM MESSAGE: You are a helpful assistant which generates an intermediate answer given a feedback to the previous answer. Please add reference (e.g. [0]) to the document in the answer.

USER MESSAGE: Original question: question

Previous answer: a_{t-1}

Feedback: i_t

Extra information related to the feedback: s_t

Answer the original question and incorporate the feedback to the previous answer.

Preserve all relevant information from previous answer.

PROMPT FOR ANSWER REVISION OF STYLISTIC INSTRUCTIONS We include the original question, answer in the previous turn (a_{t-1}) and the instruction i_t in the prompt. For llama2 , we add an additional instruction at the end “Do not generate "Sure", directly generate the updated answer. Updated answer: ”.

SYSTEM MESSAGE: You are a helpful assistant which answers a question based on a given set of documents. Please add reference (e.g. [0]) to the document in the answer.

USER MESSAGE: Original question: question

Previous answer: a_{t-1}

Feedback: i_t

Answer the original question and incorporate the feedback to the previous answer.

Preserve all relevant information from previous answer.

PROMPT LENGTH On average, the prompt for generating an answer revision for information seeking instructions (with question, previous answer, instruction and retrieved passages) consists of 607 words. The retrieved passages consists of 560 words on average. The prompt for answer revision for stylistic instructions (with question, previous answer and instruction) consists of 458 words on average.

RETRIEVAL UNIT We use S2ORC to parse the paper into paragraphs and retrieve from the set of paragraphs in the relevant papers D .

8.0.18 EXAMPLES

Examples turns with good, neutral and bad rating can be found in Table B4. Example of questions annotated (§6.3.1) can be found in Table B3.

8.0.19 IMPLEMENTATION DETAILS FOR AUTOMATIC INSTRUCTION ANALYSIS WITH GPT-4

MANUAL ANALYSIS We randomly sample 50 interaction turns and manually group the instructions into fine-grained categories. We discover 8 fine-grained categories: requesting for more information (46%); expanding answer coverage (6%); asking for clarification (6%), categorization (6%), removal (10%), specific edits (2%), condense (12%) and reorganization (10%).

GPT-4 ANALYSIS We construct a prompt with definition and in-context examples for each of the category. The prompt we use to perform automatic instruction analysis is in Table B6. We prompt gpt-4 with temperature of 0 and top_p=1. The distribution classified by GPT-4 is presented in 6.3. We see a slight distribution differences between the large-scale GPT-4 analysis and small-scale manual analysis, with GPT-4 identifying more instructions as “specific edits” and less as “categorization”. We merge categories with fewer than 5% of examples into “Others”.

MORE STATISTICS We report additional statistics for each instruction type in Table B1. We filter out turns where the response is rated as neutral or bad with an edited answer, resulting in a total of 815 turns. Among the coarse-grained instruction types, we see that stylistic instructions are longer, yet with less edit ratio compared to information seeking ones. Specific edit instructions are the longest, while instructions asking for more information requires the most amount of edits to the previous answer (highest edit ratio).

8.0.20 IMPLEMENTATION DETAILS FOR ERROR ANALYSIS

MANUAL ANALYSIS We randomly sample 10 interaction turns per model which are rated as neutral or bad, resulting in 30 total turns. We then group them into 5 fine-grained categories based on the question and comment.

Type	# words			Len ratio	Edit distance	Edit ratio
	inst	src	target			
More information	17	227	340	1.60	162	0.85
Expand coverage	39	295	398	1.48	195	0.78
Clarification	27	287	324	1.23	137	0.58
Remove	24	396	339	0.87	91.31	0.24
Specific edit	75	278	325	1.22	93	0.38
Condense	15	388	234	0.64	213	0.55
Reorganize	26	391	339	0.87	91.31	0.24
Others	23	327	342	1.36	212	0.88
Stylistic	29	361	316	0.94	141	0.42
Info seeking	22	259	355	1.60	176	0.87
GPT-4	24	260	286	1.28	145	0.70
GPT-3.5	23	289	332	1.34	145	0.68
Llama	29	379	387	1.20	173	0.56
Total	26	310	336	1.27	158	0.64

Table B1: We report statistics for each individual turns in our dataset: number of words in the instruction, number of words in the previous answer (src); number of words in the answer (target); the average length fraction between the source and the target; the token-level edit distance; and the fraction between the edit distance and the source text. We filter out turns where the response is rated as neutral or bad and there isn't an edited answer.

GPT-4 ANALYSIS The prompt we use to perform automatic instruction analysis is in Table B7.

We prompt gpt-4 with temperature of 0 and top_p=1.

8.0.21 IMPLEMENTATION DETAILS FOR AUTOMATIC EVALUATION OF INSTRUCTION FOLLOWING

8.0.21.1 GPT-4

We prompt gpt-4-1106-preview¹¹ for this task, whose context window can fit the ten-shot examples. We decode with temperature of 0 and top_p=1. Below are the prompt templates for the different baselines.

ZERO-SHOT PROMPT TEMPLATE The model is given an instruction, the question, the answer in the previous turn a_{t-1} , the instruction i_t and the answer in the current turn a_t .

¹¹<https://platform.openai.com/docs/models>

SYSTEM MESSAGE: You are a helpful assistant in evaluating the quality of the outputs for a given instruction to update an answer for an question. Your goal is to score a given updated answer for the given instruction.

USER MESSAGE: Score the updated answer for the given instruction by comparing it with the original answer. You should give one of the two ratings: good, or bad.

Give a bad rating if the updated answer either (1) only partially followed the instruction (for example, the instruction specify an edit of a certain length or at a certain location but the updated answer didn't follow the constraint) or (2) introduced changes that are not mentioned in the instruction compared to the original answer or (3) provided a vague answer (e.g. without naming actual method) or (4) added the requested information but made the answer less coherent/correct.

Only give a good rating if the answer COMPLETELY followed the instruction and didn't have ANY issues mentioned above. Your response should be ONLY the ratings.

Question:

question

Original answer:

a_{t-1}

Instruction:

i_t

Updated answer:

a_t

Rating of the Updated answer:

#

FEW-SHOT PROMPT TEMPLATE Each few-shot example j contains the question, the answer in the previous turn, the instruction, the answer in the current turn and the rating of the answer.

SYSTEM MESSAGE: You are a helpful assistant in evaluating the quality of the outputs for a given instruction to update an answer for an question. Your goal is to score a given updated answer for the given instruction.

USER MESSAGE: Score the updated answer for the given instruction by comparing it with the original answer. You should give one of the two ratings: good, or bad.

Give a bad rating if the updated answer either (1) only partially followed the instruction (for example, the instruction specify an edit of a certain length or at a certain location but the updated answer didn't follow the constraint) or (2) introduced changes that are not mentioned in the instruction compared to the original answer or (3) provided a vague answer (e.g. without naming actual method) or (4) added the requested information but made the answer less coherent/correct.

Only give a good rating if the answer COMPLETELY followed the instruction and didn't have ANY issues mentioned above. Your response should be ONLY the ratings.

Question:

question_j

Original answer:

$a_{(t-1)j}$

Instruction:

i_{tj}

Updated answer:

a_{tj}

Rating of the Updated answer:

r_{tj}

Question:

question

Original answer:

a_{t-1}

Instruction:

i_t

Updated answer:

a_t

Rating of the Updated answer:

#

8.0.21.2 T5

We finetune the model to output target sequence “Rating : r_i ” with input sequence “Original question: q Previous answer: a_{t-1} Instruction: i_t Updated answer: a_t ”, where r_i is the target binary rating, q is the question, a_{t-1} is the answer in the previous turn, i_t is the instruction and a_t is the answer in the current turn. We use a batch size of 16 and an initial learning rate of $1e-4$ with Adam optimizer and a linear learning rate schedule. We train the model for 5 epochs and choose the checkpoint with the best validation accuracy. The hyperparameters are manually searched by the authors.

8.0.21.3 ADDITIONAL EXPERIMENTS ON THREE-WAY RATING PREDICTION

We report results from experiments on three-way rating prediction. The experimental setup (i.e. data splits, hyperparameters, etc.) is the same as §6.5, except that the task is to predict among {good, neutral, and bad}, instead of predicting between {good and bad}. We report macro precision, recall, F1 among the three classes.

RESULTS Results are reported in Table B2. We observe a similar trend as the results for binary rating prediction (Table 6.5). While  gpt-4 zero-shot is biased towards judging answers as good, adding ten-shot in-context examples boosts the F1 score from 40 to 48. Fine-tuned T5 achieves better F1 compared to zero-shot prompting but underperforms ten-shot prompting. Here again, we see that there is a substantial gap between human performance and model performance, suggesting that LLMs are not yet a reliable evaluator for this task.

Model	Acc	P	R	F1	% g/n/b
Majority	38	13	33	18	100/0/0
Random	35	34	34	34	40/36/24
 zero-shot	48	50	45	40	78/11/11
 one-shot	44	31	34	31	60/24/16
 ten-shot	48	48	48	48	41/38/21
T5(fine-tuned)	45	44	44	44	37/39/24
Human*	58 ₆	57 ₆	60 ₆	56 ₆	46 ₆ /25 ₅ /29 ₆

Table B2: Test set results on automatic three-way evaluation for instruction following edits.  stands for gpt-4 . We report the % of **g**ood, **n**eutral and **b**ad ratings in the model predictions in the last column.

8.0.22 ANALYSIS ON QUESTION GENERATION WITH GPT-4

During the initial phase of the project, we explored an automatic version of the pipeline described in §6.3.1 by using GPT-4 to generate questions. We describe the pipelines below:

STEP 1: EXTRACTING AND FILTERING RELATED WORKS We extract related work paragraphs from papers published in NLP venues (ACL, EMNLP, NAACL, etc.) using S2ORC [Lo et al. 2020]. We extract the papers cited and filter out paragraphs with more than 30% of the papers missing from S2ORC or with less than 4 cited papers available. This gives us an initial set of $(q, \mathbf{D})$.

STEP 2: PROMPTING LMS TO GENERATE QUESTIONS For each related work paragraph, we prompt GPT-4 to generate five questions using the below prompt. We set the temperature to be 1 and top_p=1. paper_title is the title of the paper from which the related work paragraph is extracted.

SYSTEM MESSAGE: You are a helpful assistant which generates five questions that the paragraph is addressing. The question should require multiple sentences to answer. Don’t ask multiple sub questions in a single question. Don’t refer to specific paper in the question.

USER MESSAGE: Paper title: paper_title

Passage: related work paragraph

STEP 3: QUESTION FILTERING We employ two filtering steps to filter out questions that do not fulfill the desiderata that we described in §6.3.1. First, we use a heuristic rule to filter out questions that contain keywords which make the question anchored to specific papers (e.g. “this paper”, “mentioned”, “author”). This process filters out 23% of the questions generated.

We then prompt GPT-4 on whether the generated question (question) can be answered by the related work paragraph using the following prompt. This process filters out 12% of the question which are rated as No or Partially by GPT-4.

SYSTEM MESSAGE: You are a helpful assistant.

USER MESSAGE: Check if the paragraph answers the question. Reply Yes, No or Partially.

Question: question

Paragraph: related work paragraph

MANUAL EXAMINATION We conduct a manual examination on the quality of the (q, D) pairs generated through this automatic pipeline. We look at both the question and an initial answer generated by GPT-4 given q, D , using the prompt described in §8.0.17. Two of the authors randomly sample 50 (question, initial answer) pairs to examine both the quality of the question and the initial answer.

We found that only 60% of the questions fulfill our desiderata described in §6.3.1. 28% of the question do not make sense to a researcher (e.g. “How do previous approaches to multi-task learning and domain adaptation try to create a universal representation space using encoders

How are features of text and images fused in multi-modal summarization models?
 What are the different methods proposed for improving factual consistency in summaries generated by summarization systems?
 Is it possible to extract the data used to train a language model and if so, how to do it?
 How are pre-training corpora constructed for language models?
 How are cross-lingual language models pre-trained?

Table B3: Example questions annotated (§6.3.1).

and architectures?”), 6% of the questions are too specific or niche (e.g. “What are the primary components used by the VGVAE model to represent the semantics and syntax of a sentence in a monolingual setting?”) and 6% of the questions do not stand alone (e.g. “How do labels like "other" or "information providing" act indicate issues with these models?”). For the questions that fulfill our desiderata, GPT-4 is unable to generate a good initial answer for 26% of them, often due to retrieval failure (e.g. “The documents provided do not discuss the potential issues with using word segmentation information and dependency trees for Chinese Named Entity Recognition (NER) as extra features.”). We note that this is not necessarily an issue with the retrieval system, rather a potential indication that the question is irrelevant to the papers cited.

Overall, we find that this pipeline does not produce high quality (q, D) pairs needed for **KIWI**. Hence we proceed with human annotations as described in §6.3.1.

Require: Language model M , Retrieval model R , User U

Input: question q and a set of relevant papers D

Output: a series of interaction $\mathcal{I}$, each consists of (instruction i , answer a , rating r , comment c , edited answer a')

```
1:  $\mathcal{I} \leftarrow \emptyset$ 
    $a_0 = M(q, R(q, D))$  {Generate an initial answer}
    $a_p \leftarrow a_0$  {Set initial answer as previous answer}
2: for  $t \in \{1, \dots, 10\}$  do
3:    $i_t, I_t = U(q, a_0, \dots, a_{t-1}, D)$  {User issues an instruction  $i_t$ , and  $I_t$ , the type of the instruction.}
4:    $p_t \leftarrow \emptyset$ 
5:   if  $I_t == \text{info}$  then
6:      $p_t = R(i_t, D)$ 
7:   end if
8:    $a_t = M(q, a_p, p_t, i_t)$ 
9:    $r_t, c_t = U(q, i_t, \text{prev\_ans}, a_t)$  {User provides a rating and comment for the updated answer.}
10:   $a'_t \leftarrow a_t$ 
11:  if  $r_t \in \{\text{neutral}, \text{bad}\}$  then
12:     $a'_t = U(q, i_t, a_{t-1})$  {User optionally edits the updated answer.}
13:  end if
14:   $\mathcal{I} \leftarrow \mathcal{I} \cup \{(i_t, a_t, r_t, c_t, a'_t)\}$ 
15:  if  $U(q, a'_t) == \text{Good}$  then
16:    break {Interaction ends if the user is satisfy with the answer at this turn.}
17:  else if  $r_t \neq \text{Bad}$  or  $a_t \neq a'_t$  then
18:     $a_p \leftarrow a'_t$  {Update previous answer if answer in this turn is not rated as bad OR edited by user.}
19:  end if
20: end for
```

Figure B1: We present the workflow for collecting **KIWI**. Detailed description for each step is in §6.2.

Rating	Example
good	Original question: Are there any datasets available for table-to-text generation tasks in NLP? Previous answer: Yes, there are datasets available for table-to-text generation tasks in NLP. State-of-the-art models for data-to-text generation tasks have been trained and evaluated on various datasets. One commonly used dataset is the WebNLG dataset [0]. The DART model, for example, has achieved new state-of-the-art results on all WebNLG 2017 test set splits by fine-tuning T5-large on DART [0]. Other competitive models, both pretrained and not pretrained, have also been evaluated on this dataset [0]. The DART dataset has been used for data augmentation and has shown to improve the performance of existing models on the WebNLG 2017 dataset [0]. The TOTTO dataset is another dataset used for table-to-text generation tasks [5]. However, state-of-the-art neural models have been found to struggle in generating faithful results on the TOTTO dataset, despite the high quality of the training data [5]. The performance gap between the models and the oracle upper-bound, which treats one of the references as a candidate, has been shown to be considerable [5]. The WikiTableT dataset focuses on generating Wikipedia article sections based on structured data [4]. However, the performance of state-of-the-art models on this dataset has not been explicitly mentioned in the provided documents. Overall, while state-of-the-art models have achieved impressive results on these datasets, there is still room for improvement and further research in the field of data-to-text generation [0] [4] [5]. Instruction: What metric was used to measure performance on these datasets? Updated answer: Yes, there are datasets available for table-to-text generation tasks in NLP. State-of-the-art models for data-to-text generation tasks have been trained and evaluated on various datasets. One commonly used dataset is the WebNLG dataset [0]. The participating systems on the WebNLG dataset were evaluated using three automatic metrics [1]. The DART model, for example, has achieved new state-of-the-art results on all WebNLG 2017 test set splits by fine-tuning T5-large on DART [0]. Other competitive models, both pretrained and not pretrained, have also been evaluated on this dataset [0]. The DART dataset has been used for data augmentation and has shown to improve the performance of existing models on the WebNLG 2017 dataset [0]. The TOTTO dataset is another dataset used for table-to-text generation tasks [5]. The evaluation on the TOTTO dataset used the BLEU metric, which measures n-gram overlap between the reference and the prediction at the corpus level [5]. However, state-of-the-art neural models have been found to struggle in generating faithful results on the TOTTO dataset, despite the high quality of the training data [5]. The performance gap between the models and the oracle upper-bound, which treats one of the references as a candidate, has been shown to be considerable [5]. The WikiTableT dataset focuses on generating Wikipedia article sections based on structured data [4]. The performance of state-of-the-art models on this dataset has not been explicitly mentioned in the provided documents. Overall, while state-of-the-art models have achieved impressive results on these datasets, their performance was measured using various metrics such as BLEU and REP [1] [4] [5]. These metrics provide quantitative measures to assess the performance of models on these datasets and compare different approaches. However, there is still room for improvement and further research in the field of data-to-text generation [0] [4] [5]. Free-form comment: The model provided the requested metrics.
neutral	Original question: What approaches have been proposed to edit a generated text to be factually consistent with a collection of evidence? Previous answer: In the context of factuality correction, a "collection of evidence" refers to a set of information or data that supports or refutes a claim or statement. This collection typically consists of retrieved snippets or passages from various sources, such as web pages or documents, which are used to assess the factual consistency of the generated text and make necessary revisions [1]. Approaches that have been proposed to edit a generated text to be factually consistent with a collection of evidence include: 1. Fact extraction or attention on fact triples: This approach involves extracting and incorporating factual information from the evidence collection into the generated text. Fact triples, which consist of subject-predicate-object statements, can be used to represent these extracted facts [5]. 2. Natural language inference or question-answering models for consistency checking: These models can be employed to verify the factual consistency of the generated text with the evidence collection. By comparing the generated text with the evidence, these models can identify any inconsistencies and suggest appropriate modifications [5]. 3. Training the model on artificial datasets: In order to improve factuality correction, the model can be trained on artificial datasets that contain both generated text and corresponding evidence. This approach helps the model learn to align the generated text with the evidence and make necessary revisions [5]. 4. Adversarial data generation process using Infilling Language Models: This approach involves generating adversarial examples by filling in missing information in the generated text. By iteratively generating and updating the text, the model can be fine-tuned to ensure factual consistency with the evidence [4]. 5. Factual error correction by editing inconsistent parts in generated summaries: This approach focuses on identifying and editing specific parts of the generated text that are inconsistent with the evidence. By revising these inconsistent parts, the overall factuality of the text can be improved [5]. 6. Factual error correction by rewriting claims to better align with retrieved evidence: This approach involves rewriting or modifying the claims made in the generated text to align them more closely with the evidence. By making these adjustments, the generated text can be made more factually consistent [6]. These approaches from papers [0], [2], [3], [4], [5], and [6] aim to enhance the factuality of generated texts by incorporating evidence and making necessary changes to ensure factual consistency. Instruction: Now keep the content and the order of the text as it is, but re-write the text into 1 paragraph only Updated answer: Approaches that have been proposed to edit a generated text to be factually consistent with a collection of evidence include: extracting and incorporating factual information from the evidence collection into the generated text using fact extraction or attention on fact triples [5]; employing natural language inference or question-answering models to verify the factual consistency and suggest modifications [5]; training the model on artificial datasets that contain both generated text and corresponding evidence to align the text with the evidence [5]; generating adversarial examples by filling in missing information in the text and fine-tuning the model for factual consistency [4]; identifying and editing specific parts of the text that are inconsistent with the evidence to improve factuality [5]; rewriting or modifying the claims made in the text to align them more closely with the evidence [6]. These approaches aim to enhance the factuality of generated texts by incorporating evidence and making necessary changes to ensure factual consistency [0] [2] [3] [4] [5] [6]. [1] Free-form comment: It removed a lot from the text, including the definition

Table B4: Example turns rated as good, neutral. The bad example is in Table B5.

Rating	Example
bad	Original question: Is it possible to extract the data used to train a language model and if so, how to do it? Previous answer: Yes, it is possible to extract the data used to train a language model through extraction attacks. These attacks involve recovering specific training examples from a language model by leveraging its output [0]. The goal is to reconstruct verbatim training examples rather than just representative or "fuzzy" examples [0]. The process of performing extraction attacks typically involves several steps. First, the attacker generates probing queries designed to elicit specific responses from the language model [0]. These queries are carefully crafted to contain clues about the training data. Then, the attacker analyzes the responses generated by the language model to identify potential matches with the training data [0]. By observing patterns and similarities in the model's output, the attacker can infer the presence of specific training examples. The attacker iteratively refines their probing queries based on the model responses, gradually narrowing down the potential training examples [0]. This iterative refinement process involves generating new queries that are tailored to elicit more precise responses from the language model. Once the attacker has identified potential training examples, they can reconstruct the verbatim content by combining multiple queries and responses [0]. By carefully analyzing the model's behavior, the attacker can piece together the training data that the language model has memorized. Extraction attacks have been shown to be practical and efficient, allowing adversaries to extract hundreds of training examples from a language model [0][5][7]. These attacks have been successfully performed against various language models, including GPT-2 and clinical language models like ClinicalBERT [5][7]. Furthermore, extraction attacks can succeed even when a specific training example appears only a few times in the data [0]. In addition to extraction attacks, membership inference attacks can also be used to extract training data from language models. These attacks focus on determining whether a given example is part of the training data [5]. Adversaries train shadow models that mimic the behavior of the attacked model and use them to train an auditing model that can predict membership in the training dataset [5]. Membership inference attacks have been performed on different NLP models, including masked language models like GPT-2 and clinical language models such as ClinicalBERT, successfully extracting training data from these models [5][7]. Therefore, both extraction attacks and membership inference attacks can be used to extract the data used to train a language model [0][5][7]. Instruction: Combine the second and third paragraphs into one paragraph of no more than five sentences in which the steps to perform an extraction attack are listed. Upadted answer: Yes, it is possible to extract the data used to train a language model through extraction attacks. These attacks involve recovering specific training examples from a language model by leveraging its output [0]. The process of performing extraction attacks typically involves several steps. First, the attacker generates probing queries designed to elicit specific responses from the language model [0]. These queries are carefully crafted to contain clues about the training data. Then, the attacker analyzes the responses generated by the language model to identify potential matches with the training data [0]. By observing patterns and similarities in the model's output, the attacker can infer the presence of specific training examples. The attacker iteratively refines their probing queries based on the model responses, gradually narrowing down the potential training examples [0]. This iterative refinement process involves generating new queries that are tailored to elicit more precise responses from the language model. Once the attacker has identified potential training examples, they can reconstruct the verbatim content by combining multiple queries and responses [0]. By carefully analyzing the model's behavior, the attacker can piece together the training data that the language model has memorized. Extraction attacks have been shown to be practical and efficient, allowing adversaries to extract hundreds of training examples from a language model [0][5][7]. These attacks have been successfully performed against various language models, including GPT-2 and clinical language models like ClinicalBERT [5][7]. Furthermore, extraction attacks can succeed even when a specific training example appears only a few times in the data [0]. In addition to extraction attacks, membership inference attacks can also be used to extract training data from language models. These attacks focus on determining whether a given example is part of the training data [5]. Adversaries train shadow models that mimic the behavior of the attacked model and use them to train an auditing model that can predict membership in the training dataset [5]. Membership inference attacks have been performed on different NLP models, including masked language models like GPT-2 and clinical language models such as ClinicalBERT, successfully extracting training data from these models [5][7]. Therefore, both extraction attacks and membership inference attacks can be used to extract the data used to train a language model [0][5][7]. [0] Free-form comment: The model did not shorten the answer as requested.

Table B5: Example turns rated as bad by the annotator.

You are given an instruction to improve an answer to a question. Your job is to assign the instruction into one of the category below.

Example 1

Original question: Are there any datasets available for table-to-text generation tasks in NLP? # Instruction: How well do state-of-the-art models perform on these datasets?

Category: asking for more information

Explanation: This category of instruction asks for more information related to the question (e.g. example, limitation, evaluation results, background information, citations, definition).

Example 2

Original question: How has prior work studied and evaluated robustness of multimodal models?

Instruction: As first sentence of the text, add a very short one sentence definition of what are "multimodal models"

Category: asking for more information

Explanation: This category of instruction asks for more information related to the question (e.g. example, limitation, evaluation results, background information, citations, definition).

Example 3

Original question: How did prior work explore learning from human feedback for different NLP tasks?

Instruction: Try to add very short information also from papers 0,1,3,5,6,7,8. Keep the rest of the text exactly as it is.

Category: expand answer coverage

Explanation: This category of instruction asks for more information directly answering the question, usually from multiple papers.

Example 4

Original question: What are the existing approaches for the lip-reading task?

Instruction: You mention that some methods use a network to predict phoneme probabilities and were trained using CTC loss and that it was trained using YouTube videos. Does this mean that such methods require phoneme level labels for the data to be trained, or does word level transcriptions work?

Category: asking for clarification

Explanation: This category of instruction asks for clarification of a concept already in the answer.

Example 5

Original question: What are the prior efforts that develop RL environments with language-informed tasks?

Instruction: Organize the papers better in categories of what aspect of language-conditioned RL tasks they are tackling.

Category: categorization

Explanation: This category of instruction requests to categorize the content in the answer.

Example 6

Original question: How can information stored in knowledge bases be injected into language models task-specific model fine-tuning?

Instruction: Please shorten the answer to a maximum of 6 paragraphs.

Category: condense

Explanation: This category of instruction requests to shorten/summarize the answer.

Example 7

Original question: How can I quantize the weights of a generative language model and still achieve reasonable model performance?

Instruction: Without changing anything else, delete every instance of the string "(Feedback)", and delete the string "[1] "Compression of Generative Pre-trained Language Models via Quantization"" at the end of the answer.

Category: remove

Explanation: This category of instruction requests to remove/delete certain part of the answer.

Example 8

Original question: What techniques have been proposed to measure and evaluate the effectiveness of task-oriented dialogue systems?

Instruction: Move the paragraph starting with "Finally, there are also various task-specific evaluation metrics" to after the discussion of user satisfaction modeling, and make sure to cite paper [2] in that paragraph.

Category: reorganize

Explanation: This category of instruction requests to reorganize the answer(e.g. move things around, etc.).

Example 9

Original question: What methods have been proposed to categorize the kinds of editing operations that occur during text simplification?

Instruction: Immediately after this sentence: "In the process of text simplification, different text-editing operations are used." add this sentence "The editing operations that occur during text simplification can be usually categorized after doing literature review and reading previous publications on text simplification."

Category: direct verbatim edit

Explanation: This category of instruction requests to directly add information that are provided in the instruction verbatim to the answer.

Example 10

You are given an instruction for a model to improve an answer to a question and a comment on how the updated answer followed the instruction. Your job is to assign the comment into one of the category below.

Example 1

Instruction: Now add this information: "[2] introduce a controllable summarization model that provides a mechanism for users to specify high level attributes such as length, style, or entities of interest. This enables personalized generation."
Comment: It added the information, but in the wrong place, so it looks as an extension of another method. # Category: added information but made answer less coherent/correct
Explanation: This category of comment will mention that the instruction is followed, but the updated answer is less coherent (for example, become too long; has a weird structure, or is not correct).
In the example, the comment mentioned that the updated answer has the information added but the updated answer is confusing.

Example 2

Instruction: Remove the tenth sentence.
Comment: The sentence was not removed as requested.
Category: instruction is ignored
Explanation: This category of comment will mention that the instruction is not followed at all. In this example, the comment mentioned that requested change was not performed.

Example 3

Instruction: As a next sentence after this one "These parameters include the weights and biases of the model's layers, such as the embedding layer, encoder, decoder, and attention mechanisms [0]." add a short sentence, which explains why parameters sharing is necessary in multilingual models used for machine translation
Comment: It added an explanation, but deleted a whole paragraph
Category: introduced unrequested changes
Explanation: This category of comment will mention that the instruction is followed but also there are unrequested changes performed. In this example, the comment mentioned that the requested change (explanation) was incorporated but it also introduced unrequested change (a whole paragraph was deleted).

Example 4

Instruction: Great answer! Without changing anything else, in the last paragraph, mention some tasks that benefit from using explanations, and other tasks where it impedes performance.
Comment: The answer is overall still good but the new material is too long, and the model disregarded the part of the prompt where it was asked to put this material in the last paragraph. I will try to re-prompt.
Category: failed to follow hard constraint
Explanation: This category of comment will mention one or more hard constraint in the instruction is not followed by the model. The hard constraint can be a specific location (at the beginning of the answer), length of the added/final answer (a very short definition), or an action to avoid ("do not...").
In this example, the comment mentioned that the updated answer didn't follow the constraint in the instruction (which mention the added information should be in the last paragraph).

Example 5

Instruction: I notice a lot of redundant sentences, such as repeated mention of something like "compress the model while maintaining performance." Can you make your answer more concise?
Comment: model mostly just added paragraphs together without cutting actual content
Category: others
Explanation: This category of comment will mention that the model tried to follow the instruction (thus not completely ignore the instruction) but the updated answer is not satisfactory.
The updated answer doesn't has issue mentioned above (unrequested change or failed to follow hard constraint), but just in general has subpar quality.

Now, assign a category for the example below. ONLY ASSIGN ONE OF THE CATEGORY ABOVE. DONOT INVENT YOUR OWN CATEGORY. DON'T PROVIDE AN EXPLANATION.

Table B7: Prompt for automatic comment analysis.

Long-form QA Interaction Study

Imagine you are an NLP researcher and your colleague has asked you for a paragraph-length answer about a research topic of interest. You will work with a large language model to generate an answer to send to the colleague. You will first see the question, an initial answer generated by the model, and a set of relevant papers, based on which the model generates answers.

Your task is to provide feedback for improving the generated answer through several rounds of interaction. Detailed instructions can be found [here](#)

This process can last up to 10 interactions, or you can end it early by clicking “Ready to Send” if you find the answer satisfactory.

Question: How are features of text and images fused in multimodal summarization models?

Answer:

Chatbot

In multimodal summarization models, features of text and images are fused through a process that involves using OCR transcripts in all multimodal baselines. The OCR-generated transcripts guide the text-embeddings to focus on the most important parts in a long ASR transcript. The Factorized Multimodal Transformer (FMT) based Language Model (FLORAL) proposed in the research, takes a video, its corresponding audio and text transcript as input and generates a textual summary. The first part of FLORAL extracts unimodal features using respective unimodal feature extraction networks. Then, these unimodal features are processed using the FMT based decoder-only network over multiple steps, which in turn generates one summary word in each step. After every step, the generated word is appended to the source text with a delimiter[2].

Enter feedback

Does this question asked for more information?
☐ More Info

Submit Feedback

Rating
How satisfied are you with this updated answer?

Comments
Enter comment

Edit (this will be enabled if the rating is or)
In multimodal summarization models, features of text and images are fused through a process that involves using OCR transcripts in all multimodal baselines. The OCR-generated transcripts guide the text-embeddings to focus on the most important parts in a long ASR transcript. The Factorized Multimodal Transformer (FMT) based Language Model (FLORAL) proposed in the research, takes a video, its corresponding audio and text transcript as input and generates a textual summary. The first part of FLORAL extracts unimodal features using respective unimodal feature extraction networks. Then, these unimodal features are processed using the FMT based decoder-only network over multiple steps, which in turn generates one summary word in each step. After every step, the generated word is appended to the source text with a delimiter[2].

Other relevant passages

Submit rating & comment

Ready to send

Current turn: 1

Status: Please read the answer and input feedback

Papers:

- [0] [Multimodal Sentence Summarization via Multimodal Selective Encoding](#)
Haoran Li, Junnan Zhu, Jiajun Zhang, Xiaodong He, Chengqing Zong
- [1] [Multi-modal Sentence Summarization with Modality Attention and Image Filtering](#)
Haoran Li, Junnan Zhu, Tianshang Liu, Jiajun Zhang, Chengqing Zong
- [2] [See, Hear, Read: Leveraging Multimodality with Guided Attention for Abstractive Text Summarization](#)
Yash Atri, Shraman Pramanick, Vikram Goyal, Tanmoy Chakraborty
- [3] [DMS^2SS^2S: Deep Multi-Modal Sequence Sets with Hierarchical Modality Attention](#)
Shunsuke Kitada, Yuki Iwazaki, Riku Togashi, Hitoshi Iyatomi
- [4] [Multimodal Abstractive Summarization for How2 Videos](#)
Shruti Palaskar, Jindřich Libovický, Spandana Gella, Florian Metzger
- [5] [Multistage Fusion with Forget Gate for Multimodal Summarization in Open-Domain Videos](#)
Nayu Liu, Xian Sun, Hongfeng Yu, Wenkai Zhang, Guangluan Xu

Retrieved documents:

[2] See, Hear, Read: Leveraging Multimodality with Guided Attention for Abstractive Text Summarization

Influenced by the performance of unimodal summarization models, we incorporate the OCR transcripts into all of our multimodal baselines. Supporting our intuition, the multimodal systems obtain significant performance enhancement with OCR transcripts as shown in Table 2. The multimodal hierarchical attention model, MulT, and FMT-based encoder-decoder models show [0.8 – 1.5] points improvement in the R-L score. Our proposed FLORAL model yields the highest performance boost with OCR among all the multimodal systems, showing 3.87, 4.66, and 2.95 point enhancement in R-1, R-2, and R-L scores respectively. The performance boost can be easily attributed to the keywords in the OCR-generated transcript, which guides the text-embeddings to attend the most important portions in a very long ASR transcript. Hence, in the rest of our discussion, we always report results with (ASR + OCR) transcript, fused with guided attention, as the textual modality. Table 3 shows the ROUGE scores for different unimodal and multimodal text summarization systems on the How2 and AVIATE datasets. Among the unimodal variants, the abstractive text summarization systems generally perform much better than the extractive systems, especially on AVIATE. Note that despite being a very strong extractive baseline, Lead3 does not perform well on AVIATE, as the text transcripts of academic presentation videos do not tend to be structured with the most important information at the beginning. The two video-only models, simple conv-pool action features and action features with RNN perform very close to the abstractive textonly baselines, which clearly indicates the necessity of visual modality in addition to the textual modality. 12 As presented in Table 3, the MulT, and FMT multimodal baselines and the proposed FLORAL model beat most of the unimodal systems by a large margin, on both the datasets. This result is expected because of the inherent ability of MulT and FMT to capture the intra-model and inter-modal dynamics within asynchronous multimodal sequences and incorporate diverse information in a single network. Overall, the combination of visual, acoustic, and textual signals significantly improves over the unimodal variants, with an improvement of 1.57, 3.04, and 3 R-1, R-2, and R-L points on How2 and 6.86, 7.1 and 4.41 on AVIATE.

[2] See, Hear, Read: Leveraging Multimodality with Guided Attention for Abstractive Text Summarization

In this section, we describe our proposed system, Factorized Multimodal Transformer (Zadeh et al., 2020) based Language Model (FLORAL) for abstractive text summarization using multimodal signals. Figure 3 shows the overall architecture of FLORAL. It takes a video, its corresponding audio and text transcript as input and generates an abstractive textual summary. A video generally has three distinct modalities -visual, textual, and acoustic, which supplement each other by providing complementary information, and thus when fused, separately contribute to generating richer and more fluent summaries. The first part of FLORAL extracts unimodal features using respective unimodal feature extraction networks. This phase does not consider the contextual relationship between the three different modalities. In the next part, unimodal features are processed using the Factorized Multimodal Transformer (FMT) based decoder-only network over multiple steps, which in turn generates one summary word in each step. After every step, the generated word is appended to the source text with a delimiter. Therefore, FLORAL considers the entire summarization problem as a language modeling task, simplifying traditional encoder-decoder architecture. The remaining part of this section discusses individual modules of FLORAL in detail.

Figure B2: A screenshot of our annotation interface.

BIBLIOGRAPHY

- Adnan, Muhammad, Akhil Arunkumar, Gaurav Jain, Prashant Nair, Ilya Soloveychik, and Purushotham Kamath [2024]. “Keyformer: Kv cache reduction through key tokens selection for efficient generative inference”. In: *Proceedings of Machine Learning and Systems* 6, pp. 114–127.
- Ainslie, Joshua, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebr’on, and Sumit K. Sanghai [2023]. “GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints”. In: *ArXiv* abs/2305.13245.
- Allen-Zhu, Zeyuan, Yuanzhi Li, and Zhao Song [Sept. 2019]. “A Convergence Theory for Deep Learning via Over-Parameterization”. In: *Proceedings of the 36th International Conference on Machine Learning*. Ed. by Kamalika Chaudhuri and Ruslan Salakhutdinov. Vol. 97. Proceedings of Machine Learning Research. PMLR, pp. 242–252.
- An, Chenxin, Shansan Gong, Ming Zhong, Xingjian Zhao, Mukai Li, Jun Zhang, Lingpeng Kong, and Xipeng Qiu [2023]. *L-Eval: Instituting Standardized Evaluation for Long Context Language Models*.
- Anthropic [2026]. *Claude Opus 4.7*. Large language model. Model ID: claude-opus-4-7.
- Appel, Ruth, Peter McCrory, Alex Tamkin, Miles McCain, Tyler Neylon, and Michael Stern [2025]. *Anthropic Economic Index report: Uneven geographic and enterprise AI adoption*.
- Arora, Daman and Andrea Zanette [2025]. *Training Language Models to Reason Efficiently*.
- Asai, Akari, Jacqueline He, Rulin Shao, Weijia Shi, Amanpreet Singh, Joseph Chee Chang, Kyle Lo, Luca Soldaini, Sergey Feldman, Mike D’arcy, David Wadden, Matt Latzke, Minyang Tian,

- Pan Ji, Shengyan Liu, Hao Tong, Bohao Wu, Yanyu Xiong, Luke Zettlemoyer, Graham Neubig, Dan Weld, Doug Downey, Wen-tau Yih, Pang Wei Koh, and Hannaneh Hajishirzi [2024a]. *OpenScholar: Synthesizing Scientific Literature with Retrieval-augmented LMs*.
- Asai, Akari, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi [2024b]. “Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection”. In: *The Twelfth International Conference on Learning Representations*.
- Bai, Yushi, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li [2023]. “LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding”. In: *arXiv preprint arXiv:2308.14508*.
- Balepur, Nishant, Vishakh Padmakumar, Fumeng Yang, Shi Feng, Rachel Rudinger, and Jordan Lee Boyd-Graber [July 2025]. “Whose Boat Does it Float? Improving Personalization in Preference Tuning via Inferred User Personas”. In: *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Ed. by Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar. Vienna, Austria: Association for Computational Linguistics, pp. 3371–3393. ISBN: 979-8-89176-251-0.
- Barrantes, Mario, Benedikt Herudek, and Richard Wang [2020]. “Adversarial nli for factual correctness in text summarisation models”. In: *arXiv preprint arXiv:2005.11739*.
- Beltagy, Iz, Matthew E. Peters, and Arman Cohan [2020a]. “Longformer: The Long-Document Transformer”. In: *arXiv:2004.05150*.
- Beltagy, Iz, Matthew E. Peters, and Arman Cohan [2020b]. “Longformer: The Long-Document Transformer”. In: *ArXiv abs/2004.05150*.
- Bertsch, Amanda, Uri Alon, Graham Neubig, and Matthew Gormley [2023]. “Unlimiformer: Long-Range Transformers with Unlimited Length Input”. In: *Advances in Neural Information Processing Systems*. Ed. by A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine. Vol. 36. Curran Associates, Inc., pp. 35522–35543.

- Borgeaud, Sebastian, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George Bm Van Den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, Diego De Las Casas, Aurelia Guy, Jacob Menick, Roman Ring, Tom Hennigan, Saffron Huang, Loren Maggiore, Chris Jones, Albin Cassirer, Andy Brock, Michela Paganini, Geoffrey Irving, Oriol Vinyals, Simon Osindero, Karen Simonyan, Jack Rae, Erich Elsen, and Laurent Sifre [17–23 Jul 2022]. “Improving Language Models by Retrieving from Trillions of Tokens”. In: *Proceedings of the 39th International Conference on Machine Learning*. Ed. by Kamalika Chaudhuri, Stefanie Jegelka, Le Song, Csaba Szepesvari, Gang Niu, and Sivan Sabato. Vol. 162. Proceedings of Machine Learning Research. PMLR, pp. 2206–2240.
- Brown, Tom, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. [2020a]. “Language models are few-shot learners”. In: *Advances in neural information processing systems* 33, pp. 1877–1901.
- Brown, Tom B., Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, T. J. Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeff Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei [2020b]. “Language Models are Few-Shot Learners”. In: *ArXiv abs/2005.14165*.
- Cai, Zefan, Yichi Zhang, Bofei Gao, Yuliang Liu, Tianyu Liu, Keming Lu, Wayne Xiong, Yue Dong, Baobao Chang, Junjie Hu, et al. [2024]. “PyramidKV: Dynamic KV Cache Compression based on Pyramidal Information Funneling”. In: *CoRR*.
- Campos, Daniel Fernando, Tri Nguyen, Mir Rosenberg, Xia Song, Jianfeng Gao, Saurabh Tiwary, Rangan Majumder, Li Deng, and Bhaskar Mitra [2016]. “MS MARCO: A Human Generated MACHine Reading COMprehension Dataset”. In: *ArXiv abs/1611.09268*.

- Chen, Anthony, Gabriel Stanovsky, Sameer Singh, and Matt Gardner [Nov. 2020]. “MOCHA: A Dataset for Training and Evaluating Generative Reading Comprehension Metrics”. In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Online: Association for Computational Linguistics, pp. 6521–6532.
- Chen, Jifan, Grace Kim, Aniruddh Sriram, Greg Durrett, and Eunsol Choi [2023a]. “Complex Claim Verification with Evidence Retrieved in the Wild”. In: *ArXiv abs/2305.11859*.
- Chen, Shouyuan, Sherman Wong, Liangjian Chen, and Yuandong Tian [2023b]. *Extending Context Window of Large Language Models via Positional Interpolation*.
- Chen, Yilong, Guoxia Wang, Junyuan Shang, Shiyao Cui, Zhenyu Zhang, Tingwen Liu, Shuohuan Wang, Yu Sun, Dianhai Yu, and Hua Wu [Aug. 2024]. “NACL: A General and Effective KV Cache Eviction Framework for LLM at Inference Time”. In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Ed. by Lun-Wei Ku, Andre Martins, and Vivek Srikumar. Bangkok, Thailand: Association for Computational Linguistics, pp. 7913–7926.
- Chevalier, Alexis, Alexander Wettig, Anirudh Ajith, and Danqi Chen [2023]. “Adapting Language Models to Compress Contexts”. In: *arXiv preprint arXiv:2305.14788*.
- Chiang, Cheng-Han and Hung-Yi Lee [2023]. “Can Large Language Models Be an Alternative to Human Evaluations?” In: *Annual Meeting of the Association for Computational Linguistics*.
- Child, Rewon, Scott Gray, Alec Radford, and Ilya Sutskever [2019]. “Generating Long Sequences with Sparse Transformers”. In: *ArXiv abs/1904.10509*.
- Choi, Eunbi, Yongrae Jo, Joel Jang, and Minjoon Seo [2022]. “Prompt Injection: Parameterization of Fixed Inputs”. In: *ArXiv abs/2206.11349*.
- Chung, Hyung Won, Le Hou, S. Longpre, Barret Zoph, Yi Tay, William Fedus, Eric Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, Albert Webson, Shixiang Shane Gu, Zhuyun Dai, Mirac Suzgun, Xinyun Chen, Aakanksha Chowdhery, Dasha Valter, Sharan Narang, Gaurav Mishra, Adams Wei Yu, Vincent Zhao, Yanping Huang, Andrew M. Dai, Hongkun Yu, Slav Petrov,

- Ed Huai-hsin Chi, Jeff Dean, Jacob Devlin, Adam Roberts, Denny Zhou, Quoc V. Le, and Jason Wei [2022]. “Scaling Instruction-Finetuned Language Models”. In: *ArXiv abs/2210.11416*.
- Clark, Kevin, Minh-Thang Luong, Quoc V. Le, and Christopher D. Manning [2020]. “ELECTRA: Pre-training Text Encoders as Discriminators Rather Than Generators”. In: *ICLR*.
- Cohen, Jacob [1960]. “A Coefficient of Agreement for Nominal Scales”. In: *Educational and Psychological Measurement* 20, pp. 37–46.
- Collins, Allan and Dedre Gentner [1980]. “A framework for a cognitive theory of writing”. English. In: *Cognitive Processes in Writing*. Erlbaum, pp. 51–72.
- Conover, Mike, Matt Hayes, Ankit Mathur, Jianwei Xie, Jun Wan, Sam Shah, Ali Ghodsi, Patrick Wendell, Matei Zaharia, and Reynold Xin [2023]. *Free Dolly: Introducing the World’s First Truly Open Instruction-Tuned LLM*.
- Damani, Mehul, Idan Shenfeld, Andi Peng, Andreea Bobu, and Jacob Andreas [2025]. “Learning How Hard to Think: Input-Adaptive Allocation of LM Computation”. In: *The Thirteenth International Conference on Learning Representations*.
- Dao, Tri [2024]. “FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning”. In: *International Conference on Learning Representations (ICLR)*.
- Dasigi, Pradeep, Kyle Lo, Iz Beltagy, Arman Cohan, Noah A. Smith, and Matt Gardner [June 2021]. “A Dataset of Information-Seeking Questions and Answers Anchored in Research Papers”. In: *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Ed. by Kristina Toutanova, Anna Rumshisky, Luke Zettlemoyer, Dilek Hakkani-Tur, Iz Beltagy, Steven Bethard, Ryan Cotterell, Tanmoy Chakraborty, and Yichao Zhou. Online: Association for Computational Linguistics, pp. 4599–4610.
- DeepSeek-AI [2025]. *DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models*.
- Dettmers, Tim, Mike Lewis, Sam Shleifer, and Luke Zettlemoyer [2021]. “8-bit Optimizers via Block-wise Quantization”. In: *CoRR abs/2110.02861*.

- Du, Simon S., Xiyu Zhai, Barnabas Póczos, and Aarti Singh [2019]. “Gradient Descent Provably Optimizes Over-parameterized Neural Networks”. In: *International Conference on Learning Representations*.
- Du, Wanyu, Vipul Raheja, Dhruv Kumar, Zae Myung Kim, Melissa Lopez, and Dongyeop Kang [2022]. “Understanding Iterative Revision from Human-Written Text”. In: *ArXiv abs/2203.03802*.
- Dubois, Yann, Xuechen Li, Rohan Taori, Tianyi Zhang, Ishaan Gulrajani, Jimmy Ba, Carlos Guestrin, Percy Liang, and Tatsunori Hashimoto [2023]. “AlpacaFarm: A Simulation Framework for Methods that Learn from Human Feedback”. In: *ArXiv abs/2305.14387*.
- Dwivedi-Yu, Jane, Timo Schick, Zhengbao Jiang, Maria Lomeli, Patrick Lewis, Gautier Izacard, Edouard Grave, Sebastian Riedel, and Fabio Petroni [2022]. “EditEval: An Instruction-Based Benchmark for Text Improvements”. In: *ArXiv abs/2209.13331*.
- Fabbri, Alexander, Chien-Sheng Wu, Wenhao Liu, and Caiming Xiong [July 2022]. “QAFactEval: Improved QA-Based Factual Consistency Evaluation for Summarization”. In: *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Seattle, United States: Association for Computational Linguistics, pp. 2587–2601.
- Fan, Angela, Yacine Jernite, Ethan Perez, David Grangier, Jason Weston, and Michael Auli [July 2019a]. “ELI5: Long Form Question Answering”. In: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Florence, Italy: Association for Computational Linguistics, pp. 3558–3567.
- Fan, Angela, Yacine Jernite, Ethan Perez, David Grangier, Jason Weston, and Michael Auli [July 2019b]. “ELI5: Long Form Question Answering”. In: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Florence, Italy: Association for Computational Linguistics, pp. 3558–3567.
- Fleiss, Joseph L [1971]. “Measuring nominal scale agreement among many raters.” In: *Psychological bulletin* 76.5, p. 378.

- Fleiss, Joseph L, Bruce Levin, and Myunghee Cho Paik [2013]. *Statistical methods for rates and proportions*. John Wiley & Sons.
- Fleming, S., Alejandro Lozano, William J. Haberkorn, Jenelle A. Jindal, Eduardo Pontes Reis, Rahul Thapa, Louis Blankemeier, Julian Z. Genkins, Ethan H. Steinberg, Ashwin Nayak, Birju S. Patel, Chia-Chun Chiang, Alison Callahan, Zepeng Huo, Sergios Gatidis, Scott J. Adams, Oluseyi Fayanju, Shreya Shah, Thomas Savage, Ethan Goh, Akshay S. Chaudhari, Nima Aghaeepour, Christopher D. Sharp, Michael A. Pfeffer, Percy Liang, Jonathan H. Chen, Keith E. Morse, Emma Brunskill, Jason Alan Fries, and Nigam H. Shah [2023]. “MedAlign: A Clinician-Generated Dataset for Instruction Following with Electronic Medical Records”. In: *AAAI Conference on Artificial Intelligence*.
- Freitag, Markus, George Foster, David Grangier, Viresh Ratnakar, Qijun Tan, and Wolfgang Macherey [2021]. “Experts, Errors, and Context: A Large-Scale Study of Human Evaluation for Machine Translation”. In: *Transactions of the Association for Computational Linguistics* 9, pp. 1460–1474.
- Gao, Ze-Feng, Kun Zhou, Peiyu Liu, Wayne Xin Zhao, and Ji-rong Wen [2023]. “Small Pre-trained Language Models Can be Fine-tuned as Large Models via Over-Parameterization”. In: *Annual Meeting of the Association for Computational Linguistics*.
- Gao, Luyu, Zhuyun Dai, Panupong Pasupat, Anthony Chen, Arun Tejasvi Chaganty, Yicheng Fan, Vincent Zhao, N. Lao, Hongrae Lee, Da-Cheng Juan, and Kelvin Guu [2022a]. “Attributed Text Generation via Post-hoc Research and Revision”. In: *ArXiv* abs/2210.08726.
- Gao, Tianxiang, Hailiang Liu, Jia Liu, Hriday Rajan, and Hongyang Gao [2022b]. “A global convergence theory for deep ReLU implicit networks via over-parameterization”. In: *International Conference on Learning Representations*.
- Ge, Suyu, Yunan Zhang, Liyuan Liu, Minjia Zhang, Jiawei Han, and Jianfeng Gao [2024]. “Model Tells You What to Discard: Adaptive KV Cache Compression for LLMs”. In: *The Twelfth International Conference on Learning Representations*.

- Gemini [2024]. “Google. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context”. In: *arXiv preprint arXiv:2403.05530*.
- Gillick, Dan and Yang Liu [June 2010]. “Non-Expert Evaluation of Summarization Systems is Risky”. In: *Proceedings of the NAACL HLT 2010 Workshop on Creating Speech and Language Data with Amazon’s Mechanical Turk*. Los Angeles: Association for Computational Linguistics, pp. 148–151.
- Google [2023]. “Gemini: A Family of Highly Capable Multimodal Models”. In: *ArXiv abs/2312.11805*.
- Google [2025]. *Gemini 2.5: Pushing the Frontier with Advanced Reasoning, Multimodality, Long Context, and Next Generation Agentic Capabilities*.
- Goyal, Tanya and Greg Durrett [Nov. 2020]. “Evaluating Factuality in Generation with Dependency-level Entailment”. In: *Findings of the Association for Computational Linguistics: EMNLP 2020*. Online: Association for Computational Linguistics, pp. 3592–3603.
- Goyal, Tanya, Junyi Jessy Li, and Greg Durrett [2022a]. “News Summarization and Evaluation in the Era of GPT-3”. In: *arXiv preprint arXiv:2209.12356*.
- Goyal, Tanya, Junyi Jessy Li, and Greg Durrett [2022b]. “SNaC - Coherence Error Detection for Narrative Summarization”. In: *Proceedings of EMNLP*.
- Gu, Albert, Karan Goel, and Christopher Re [2022]. “Efficiently Modeling Long Sequences with Structured State Spaces”. In: *International Conference on Learning Representations*.
- Guha, Neel, Julian Nyarko, Daniel Ho, Christopher Ré, Adam Chilton, Aditya K, Alex Chohlas-Wood, Austin Peters, Brandon Waldon, Daniel Rockmore, Diego Zambrano, Dmitry Talisman, Enam Hoque, Faiz Surani, Frank Fagan, Galit Sarfaty, Gregory Dickinson, Haggai Porat, Jason Hegland, Jessica Wu, Joe Nudell, Joel Niklaus, John Nay, Jonathan Choi, Kevin Tobia, Margaret Hagan, Megan Ma, Michael Livermore, Nikon Rasumov-Rahe, Nils Holzenberger, Noam Kolt, Peter Henderson, Sean Rehaag, Sharad Goel, Shang Gao, Spencer Williams, Sunny Gandhi, Tom Zur, Varun Iyer, and Zehua Li [2023]. “LegalBench: A Collaboratively Built Benchmark for Measuring Legal Reasoning in Large Language Models”. In: *Advances in Neural Information*

- Processing Systems*. Ed. by A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine. Vol. 36. Curran Associates, Inc., pp. 44123–44279.
- Guo, Daya et al. [2025]. “DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning”. In: *Nature* 645.8081, pp. 633–638. ISSN: 1476-4687.
- Guu, Kelvin, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang [2020]. “REALM: Retrieval-Augmented Language Model Pre-Training”. In: *arXiv preprint 2002.08909*.
- Han, Jesse Michael, Igor Babuschkin, Harrison Edwards, Arvind Neelakantan, Tao Xu, Stanislas Polu, Alex Ray, Pranav Shyam, Aditya Ramesh, Alec Radford, et al. [2021]. “Unsupervised neural machine translation with generative language models only”. In: *arXiv preprint arXiv:2110.05448*.
- He, Junxian, Graham Neubig, and Taylor Berg-Kirkpatrick [Nov. 2021]. “Efficient Nearest Neighbor Language Models”. In: *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Online and Punta Cana, Dominican Republic: Association for Computational Linguistics, pp. 5703–5714.
- Hendrycks, Dan, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt [2021]. “Measuring Massive Multitask Language Understanding”. In: *International Conference on Learning Representations*.
- Hermann, Karl Moritz, Tomás Kociský, Edward Grefenstette, Lasse Espeholt, Will Kay, Mustafa Suleyman, and Phil Blunsom [2015]. “Teaching Machines to Read and Comprehend”. In: *ArXiv abs/1506.03340*.
- Hinton, Geoffrey E., Oriol Vinyals, and Jeffrey Dean [2015]. “Distilling the Knowledge in a Neural Network”. In: *ArXiv abs/1503.02531*.
- Hoffmann, Jordan, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katherine Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Oriol Vinyals, Jack William Rae, and Laurent Sifre [2022]. “An empirical analysis of compute-optimal large language model training”.

- In: *Advances in Neural Information Processing Systems*. Ed. by Alice H. Oh, Alekh Agarwal, Danielle Belgrave, and Kyunghyun Cho.
- Holtzman, Ari, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi [2019]. “The Curious Case of Neural Text Degeneration”. In: *International Conference on Learning Representations*.
- Honnibal, Matthew, Ines Montani, Sophie Van Landeghem, and Boyd Adriane [2020]. *spaCy: Industrial-strength Natural Language Processing in Python*.
- Hooper, Coleman, Sehoon Kim, Hiva Mohammadzadeh, Michael W Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami [2024]. “KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization”. In: *arXiv preprint arXiv:2401.18079*.
- Hsieh, Cheng-Ping, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg [2024]. “RULER: What’s the Real Context Size of Your Long-Context Language Models?” In: *arXiv preprint arXiv:2404.06654*.
- Hsu, Chao-Chun and Chenhao Tan [Nov. 2021]. “Decision-Focused Summarization”. In: *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Online and Punta Cana, Dominican Republic: Association for Computational Linguistics, pp. 117–132.
- Huang, Luyang, Shuyang Cao, Nikolaus Parulian, Heng Ji, and Lu Wang [June 2021]. “Efficient Attentions for Long Document Summarization”. In: *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Ed. by Kristina Toutanova, Anna Rumshisky, Luke Zettlemoyer, Dilek Hakkani-Tur, Iz Beltagy, Steven Bethard, Ryan Cotterell, Tanmoy Chakraborty, and Yichao Zhou. Online: Association for Computational Linguistics, pp. 1419–1436.
- Huh, Mina, Fangyuan Xu, Yi-Hao Peng, Chongyan Chen, Hansika Murugu, Danna Gurari, Eunsol Choi, and Amy Pavel [2024]. “Long-Form Answers to Visual Questions from Blind and Low Vision People”. In: *First Conference on Language Modeling*.
- Iskender, Neslihan, Tim Polzehl, and Sebastian Möller [Nov. 2020]. “Best Practices for Crowd-based Evaluation of German Summarization: Comparing Crowd, Expert and Automatic Evaluation”.

- In: *Proceedings of the First Workshop on Evaluation and Comparison of NLP Systems*. Online: Association for Computational Linguistics, pp. 164–175.
- Izacard, Gautier, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave [2021]. *Unsupervised Dense Information Retrieval with Contrastive Learning*.
- Izacard, Gautier, Patrick Lewis, Maria Lomeli, Lucas Hosseini, Fabio Petroni, Timo Schick, Jane A. Yu, Armand Joulin, Sebastian Riedel, and Edouard Grave [2022]. “Few-shot Learning with Retrieval Augmented Language Models”. In: *ArXiv* abs/2208.03299.
- Jiang, Bowen, Zhuoqun Hao, Young Min Cho, Bryan Li, Yuan Yuan, Sihao Chen, Lyle Ungar, Camillo Jose Taylor, and Dan Roth [2025]. “Know Me, Respond to Me: Benchmarking LLMs for Dynamic User Profiling and Personalized Responses at Scale”. In: *Second Conference on Language Modeling*.
- Jiang, Huiqiang, Yucheng Li, Chengruidong Zhang, Qianhui Wu, Xufang Luo, Surin Ahn, Zhenhua Han, Amir H Abdi, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu [2024]. “MInference 1.0: Accelerating Pre-filling for Long-Context LLMs via Dynamic Sparse Attention”. In: *arXiv preprint arXiv:2407.02490*.
- Jiang, Huiqiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu [2023]. “LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression”. In: *ArXiv* abs/2310.06839.
- Jiang, Yuchen Eleanor, Tianyu Liu, Shuming Ma, Dongdong Zhang, Jian Yang, Haoyang Huang, Rico Sennrich, Ryan Cotterell, Mrinmaya Sachan, and Ming Zhou [July 2022]. “BlonDe: An Automatic Evaluation Metric for Document-level Machine Translation”. In: *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies [NAACL]*. Association for Computational Linguistics.

- Jimenez, Carlos E, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan [2024]. “SWE-bench: Can Language Models Resolve Real-world Github Issues?” In: *The Twelfth International Conference on Learning Representations*.
- Jin, Bowen, Jinsung Yoon, Jiawei Han, and Sercan Arik [2025]. “Long-Context LLMs Meet RAG: Overcoming Challenges for Long Inputs in RAG”. In: *International Conference on Learning Representations*. Ed. by Y. Yue, A. Garg, N. Peng, F. Sha, and R. Yu. Vol. 2025, pp. 37784–37822.
- Jin, Qiao, Bhuwan Dhingra, Zhengping Liu, William W. Cohen, and Xinghua Lu [2019]. “PubMedQA: A Dataset for Biomedical Research Question Answering”. In: *Conference on Empirical Methods in Natural Language Processing*.
- Joshi, Mandar, Eunsol Choi, Daniel S Weld, and Luke Zettlemoyer [2017a]. “Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension”. In: *arXiv preprint 1705.03551*.
- Joshi, Mandar, Eunsol Choi, Daniel S. Weld, and Luke Zettlemoyer [July 2017b]. “TriviaQA: A Large Scale Distantly Supervised Challenge Dataset for Reading Comprehension”. In: *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics*. Vancouver, Canada: Association for Computational Linguistics.
- Jung, Jaehun, Peter West, Liwei Jiang, Faeze Brahman, Ximing Lu, Jillian Fisher, Taylor Sorensen, and Yejin Choi [2023]. “Impossible Distillation: from Low-Quality Model to High-Quality Dataset & Model for Summarization and Paraphrasing”. In: *arXiv preprint arXiv:2305.16635*.
- Kamradt, Gregory [2023]. *Needle In A Haystack - pressure testing LLMs, Commercially Usable LLMs*. URL: <https://github.com/gkamradt/LLMTest%20NeedleInAHaystack/tree/main..>
- Kaplan, Jared, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei [2020]. *Scaling Laws for Neural Language Models*.
- Karpinska, Marzena, Katherine Thai, Kyle Lo, Tanya Goyal, and Mohit Iyyer [2024]. *One Thousand and One Pairs: A "novel" challenge for long-context language models*.

- Karpukhin, Vladimir, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih [Nov. 2020a]. “Dense Passage Retrieval for Open-Domain Question Answering”. In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Online: Association for Computational Linguistics, pp. 6769–6781.
- Karpukhin, Vladimir, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih [Nov. 2020b]. “Dense Passage Retrieval for Open-Domain Question Answering”. In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Online: Association for Computational Linguistics, pp. 6769–6781.
- Karpukhin, Vladimir, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Yu Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih [2020c]. “Dense Passage Retrieval for Open-Domain Question Answering”. In: *Conference on Empirical Methods in Natural Language Processing*.
- Kasai, Jungo, Keisuke Sakaguchi, Ronan Le Bras, Lavinia Dunagan, Jacob Morrison, Alexander Fabbri, Yejin Choi, and Noah A. Smith [July 2022]. “Bidimensional Leaderboards: Generate and Evaluate Language Hand in Hand”. In: *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Seattle, United States: Association for Computational Linguistics, pp. 3540–3557.
- Khandelwal, Urvashi, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis [2019]. “Generalization through Memorization: Nearest Neighbor Language Models”. In: *ArXiv abs/1911.00172*.
- Kingma, Diederik P. and Jimmy Ba [2014]. “Adam: A Method for Stochastic Optimization”. In: *CoRR abs/1412.6980*.
- Kopf, Andreas, Yannic Kilcher, Dimitri von Rutte, Sotiris Anagnostidis, Zhi Rui Tam, Keith Stevens, Abdullah Barhoum, Nguyen Minh Duc, Oliver Stanley, Rich’ard Nagyfi, ES Shahul, Sameer Suri, David Glushkov, Arnav Dantuluri, Andrew Maguire, Christoph Schuhmann, Huu Nguyen, and Alexander Mattick [2023]. “OpenAssistant Conversations - Democratizing Large Language Model Alignment”. In: *ArXiv abs/2304.07327*.

- Krishna, Kalpesh, Yapei Chang, John Wieting, and Mohit Iyyer [2022]. “RankGen: Improving Text Generation with Large Ranking Models”. In: *arXiv preprint arXiv:2205.09726*.
- Krishna, Kalpesh, Aurko Roy, and Mohit Iyyer [June 2021]. “Hurdles to Progress in Long-form Question Answering”. In: *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Online: Association for Computational Linguistics, pp. 4940–4957.
- Kryscinski, Wojciech, Bryan McCann, Caiming Xiong, and Richard Socher [Nov. 2020]. “Evaluating the Factual Consistency of Abstractive Text Summarization”. In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Online: Association for Computational Linguistics, pp. 9332–9346.
- Kwiatkowski, Tom, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei Chang, Andrew M. Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov [2019]. “Natural Questions: A Benchmark for Question Answering Research”. In: *Transactions of the Association for Computational Linguistics* 7, pp. 452–466.
- Laban, Philippe, Tobias Schnabel, Paul N. Bennett, and Marti A. Hearst [2022]. “SummaC: Re-Visiting NLI-based Models for Inconsistency Detection in Summarization”. In: *Transactions of the Association for Computational Linguistics* 10, pp. 163–177.
- Landis, J Richard and Gary G Koch [1977]. “The measurement of observer agreement for categorical data”. In: *biometrics*, pp. 159–174.
- Lee, Mina, Megha Srivastava, Amelia Hardy, John Thickstun, Esin Durmus, Ashwin Paranjape, Ines Gerard-Ursin, Xiang Lisa Li, Faisal Ladhak, Frieda Rong, Rose E. Wang, Minae Kwon, Joon Sung Park, Hancheng Cao, Tony Lee, Rishi Bommasani, Michael S. Bernstein, and Percy Liang [2022]. “Evaluating Human-Language Model Interaction”. In: *ArXiv abs/2212.09746*.

- Lee, Yoonjoo, Kyungjae Lee, Sunghyun Park, Dasol Hwang, Jaehyeon Kim, Ho Hin Lee, and Moon-tae Lee [2023]. “QASA: Advanced Question Answering on Scientific Articles”. In: *International Conference on Machine Learning*.
- Lewis, Mike, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Veselin Stoyanov, and Luke Zettlemoyer [July 2020a]. “BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension”. In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Online: Association for Computational Linguistics, pp. 7871–7880.
- Lewis, Patrick, Ethan Perez, Aleksandara Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela [2020b]. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *ArXiv abs/2005.11401*.
- Li, Fengfu and Bin Liu [2016]. “Ternary Weight Networks”. In: *ArXiv abs/1605.04711*.
- Li, Xuechen, Tianyi Zhang, Yann Dubois, Rohan Taori, Ishaan Gulrajani, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto [2023a]. *AlpacaEval: An Automatic Evaluator of Instruction-following Models*. https://github.com/tatsu-lab/alpaca_eval.
- Li, Yucheng, Bo Dong, Chenghua Lin, and Frank Guerin [2023b]. “Compressing Context to Enhance Inference Efficiency of Large Language Models”. In: *Conference on Empirical Methods in Natural Language Processing*.
- Li, Yuhong, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen [2024]. “SnapKV: LLM Knows What You are Looking for Before Generation”. In: *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.
- Lian, Long, Sida Wang, Felix Juefei-Xu, Tsu-Jui Fu, Xiuyu Li, Adam Yala, Trevor Darrell, Alane Suhr, Yuandong Tian, and Xi Victoria Lin [2025]. “ThreadWeaver: Adaptive Threading for Efficient Parallel Reasoning in Language Models”. In: *arXiv preprint arXiv:2512.07843*.

- Liang, Percy, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, Deepak Narayanan, Yuhuai Wu, Ananya Kumar, Benjamin Newman, Binhang Yuan, Bobby Yan, Ce Zhang, Christian Cosgrove, Christopher D. Manning, Christopher Ré, Diana Acosta-Navas, Drew A. Hudson, Eric Zelikman, Esin Durmus, Faisal Ladhak, Frieda Rong, Hongyu Ren, Huaxiu Yao, Jue Wang, Keshav Santhanam, Laurel Orr, Lucia Zheng, Mert Yuksekgonul, Mirac Suzgun, Nathan Kim, Neel Guha, Niladri Chatterji, Omar Khattab, Peter Henderson, Qian Huang, Ryan Chi, Sang Michael Xie, Shibani Santurkar, Surya Ganguli, Tatsunori Hashimoto, Thomas Icard, Tianyi Zhang, Vishrav Chaudhary, William Wang, Xuechen Li, Yifan Mai, Yuhui Zhang, and Yuta Koreeda [2023]. *Holistic Evaluation of Language Models*.
- Lin, Chin-Yew [July 2004]. “ROUGE: A Package for Automatic Evaluation of Summaries”. In: *Text Summarization Branches Out*. Barcelona, Spain: Association for Computational Linguistics, pp. 74–81.
- Liu, Akide, Jing Liu, Zizheng Pan, Yefei He, Gholamreza Haffari, and Bohan Zhuang [2024a]. “MiniCache: KV Cache Compression in Depth Dimension for Large Language Models”. In: *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.
- Liu, Junteng, Yunji Li, Chi Zhang, Jingyang Li, Aili Chen, Ke Ji, Weiyu Cheng, Zijia Wu, Chengyu Du, Qidi Xu, Jiayuan Song, Zhengmao Zhu, Wenhui Chen, Pengyu Zhao, and Junxian He [2025]. *WebExplorer: Explore and Evolve for Training Long-Horizon Web Agents*.
- Liu, Nelson F, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang [2023a]. “Lost in the middle: How language models use long contexts”. In: *arXiv preprint arXiv:2307.03172*.
- Liu, Xiang, Xuming Hu, Xiaowen Chu, and Eunsol Choi [2026a]. “DiffAdapt: Difficulty-Adaptive Reasoning for Token-Efficient LLM Inference”. In: *The Fourteenth International Conference on Learning Representations*.

- Liu, Yixin, A. R. Fabbri, Jiawen Chen, Yilun Zhao, Simeng Han, Shafiq R. Joty, Pengfei Liu, Dragomir Radev, Chien-Sheng Wu, and Arman Cohan [2023b]. “Benchmarking Generation and Evaluation Capabilities of Large Language Models for Instruction Controllable Summarization”. In: *ArXiv* abs/2311.09184.
- Liu, Yixin, Alexander R. Fabbri, Pengfei Liu, Yilun Zhao, Linyong Nan, Ruilin Han, Simeng Han, Shafiq R. Joty, Chien-Sheng Wu, Caiming Xiong, and Dragomir R. Radev [2022]. “Revisiting the Gold Standard: Grounding Summarization Evaluation with Robust Human Evaluation”. In: *ArXiv* abs/2212.07981.
- Liu, Yuhan, Fangyuan Xu, Vishakh Padmakumar, Daphne Ippolito, and Eunsol Choi [2026b]. *No Single Best Model for Diversity: Learning a Router for Sample Diversity*.
- Liu, Zirui, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu [2024b]. “KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache”. In: *ArXiv* abs/2402.02750.
- Lo, Kyle, Lucy Lu Wang, Mark Neumann, Rodney Kinney, and Daniel Weld [July 2020]. “S2ORC: The Semantic Scholar Open Research Corpus”. In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Online: Association for Computational Linguistics, pp. 4969–4983.
- Lu, Miao, Weiwei Sun, Weihua Du, Zhan Ling, Xuesong Yao, Kang Liu, and Jiecao Chen [2025]. *Scaling LLM Multi-turn RL with End-to-end Summarization-based Context Management*.
- Mallen, Alex, Akari Asai, Victor Zhong, Rajarshi Das, Hannaneh Hajishirzi, and Daniel Khashabi [2022]. “When Not to Trust Language Models: Investigating Effectiveness and Limitations of Parametric and Non-Parametric Memories”. In: *ArXiv* abs/2212.10511.
- Martin-Boyle, Anna, Aahan Tyagi, Marti A. Hearst, and Dongyeop Kang [2024]. “Shallow Synthesis of Knowledge in GPT-Generated Texts: A Case Study in Automatic Related Work Composition”. In: *ArXiv* abs/2402.12255.

- Martins, Pedro Henrique, Zita Marinho, and André F. T. Martins [Dec. 2022]. “Chunk-based Nearest Neighbor Machine Translation”. In: *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*. Abu Dhabi, United Arab Emirates: Association for Computational Linguistics, pp. 4228–4245.
- Maynez, Joshua, Shashi Narayan, Bernd Bohnet, and Ryan McDonald [July 2020]. “On Faithfulness and Factuality in Abstractive Summarization”. In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Online: Association for Computational Linguistics, pp. 1906–1919.
- Merity, Stephen, Caiming Xiong, James Bradbury, and Richard Socher [2016]. “Pointer Sentinel Mixture Models”. In: *ArXiv abs/1609.07843*.
- Meta [2024]. “The Llama 3 Herd of Models”. In: *ArXiv abs/2407.21783*.
- Min, Sewon, Julian Michael, Hannaneh Hajishirzi, and Luke Zettlemoyer [2020]. “AmbigQA: Answering Ambiguous Open-domain Questions”. In: *Conference on Empirical Methods in Natural Language Processing*.
- MosaicML, NLP Team [2023]. *Introducing MPT-7B: A New Standard for Open-Source, Commercially Usable LLMs*. URL: <https://www.mosaicml.com/blog/mpt-7b>.
- Mu, Jesse, Xiang Lisa Li, and Noah D. Goodman [2023]. “Learning to Compress Prompts with Gist Tokens”. In: *ArXiv abs/2304.08467*.
- Muennighoff, Niklas, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto [2025]. *s1: Simple test-time scaling*.
- Nakano, Reiichiro, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, et al. [2021]. “WebGPT: Browser-assisted question-answering with human feedback”. In: *arXiv preprint arXiv:2112.09332*.
- Nallapati, Ramesh, Bing Xiang, and Bowen Zhou [2016]. “Sequence-to-Sequence RNNs for Text Summarization”. In: *CoRR abs/1602.06023*.

- Niklaus, Joel, Veton Matoshi, Pooja Rani, Andrea Galassi, Matthias Stürmer, and Ilias Chalkidis [Dec. 2023]. “LEXTREME: A Multi-Lingual and Multi-Task Benchmark for the Legal Domain”. In: *Findings of the Association for Computational Linguistics: EMNLP 2023*. Ed. by Houda Bouamor, Juan Pino, and Kalika Bali. Singapore: Association for Computational Linguistics, pp. 3016–3054.
- OpenAI [2023]. “GPT-4 technical report”. In: *arXiv*, pp. 2303–08774.
- OpenAI [2026]. *OpenAI GPT-5 System Card*.
- Ouyang, Long, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe [2022]. *Training language models to follow instructions with human feedback*.
- Ouyang, Siru, Shuo Wang, Yang Liu, Ming Zhong, Yizhu Jiao, Dan Iter, Reid Pryzant, Chenguang Zhu, Heng Ji, and Jiawei Han [2023]. “The Shifted and The Overlooked: A Task-oriented Investigation of User-GPT Interactions”. In: *Conference on Empirical Methods in Natural Language Processing*.
- Padmanabhan, Shankar, Yasumasa Onoe, Michael J.Q. Zhang, Greg Durrett, and Eunsol Choi [2023]. “Propagating Knowledge Updates to LMs Through Distillation”. In: *ArXiv abs/2306.09306*.
- Pan, Jiayi, Xiuyu Li, Long Lian, Charlie Victor Snell, Yifei Zhou, Adam Yala, Trevor Darrell, Kurt Keutzer, and Alane Suhr [2025]. “Learning Adaptive Parallel Reasoning with Language Models”. In: *Second Conference on Language Modeling*.
- Papineni, Kishore, Salim Roukos, Todd Ward, and Wei-Jing Zhu [July 2002]. “Bleu: a Method for Automatic Evaluation of Machine Translation”. In: *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*. Philadelphia, Pennsylvania, USA: Association for Computational Linguistics, pp. 311–318.
- Pedregosa, F., G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot,

- and E. Duchesnay [2011]. “Scikit-learn: Machine Learning in Python”. In: *Journal of Machine Learning Research* 12, pp. 2825–2830.
- Petroni, Fabio, Aleksandra Piktus, Angela Fan, Patrick Lewis, Majid Yazdani, Nicola De Cao, James Thorne, Yacine Jernite, Vladimir Karpukhin, Jean Maillard, Vassilis Plachouras, Tim Rocktäschel, and Sebastian Riedel [June 2021]. “KILT: a Benchmark for Knowledge Intensive Language Tasks”. In: *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Online: Association for Computational Linguistics, pp. 2523–2544.
- Pillutla, Krishna, Swabha Swayamdipta, Rowan Zellers, John Thickstun, Sean Welleck, Yejin Choi, and Zaïd Harchaoui [2021]. “MAUVE: Measuring the Gap Between Neural Text and Human Text using Divergence Frontiers”. In: *Neural Information Processing Systems*.
- Potluri, Abhilash, Fangyuan Xu, and Eunsol Choi [July 2023]. “Concise Answers to Complex Questions: Summarization of Long-form Answers”. In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Toronto, Canada: Association for Computational Linguistics, pp. 9709–9728.
- Qi, Peng, Yuhao Zhang, Yuhui Zhang, Jason Bolton, and Christopher D. Manning [July 2020]. “Stanza: A Python Natural Language Processing Toolkit for Many Human Languages”. In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*. Online: Association for Computational Linguistics, pp. 101–108.
- Qin, Lianhui, Sean Welleck, Daniel Khashabi, and Yejin Choi [2022]. “COLD decoding: Energy-based constrained text generation with langevin dynamics”. In: *arXiv preprint arXiv:2202.11705*.
- Radford, Alec, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever [2019]. “Language Models are Unsupervised Multitask Learners”. In.
- Raffel, Colin, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu [2020a]. “Exploring the limits of transfer learning with a unified text-to-text transformer”. In: *The Journal of Machine Learning Research* 21.1, pp. 5485–5551.

- Raffel, Colin, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu [2020b]. “Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer”. In: *Journal of Machine Learning Research* 21.140, pp. 1–67.
- Raffel, Colin, Noam M. Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu [2019]. “Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer”. In: *J. Mach. Learn. Res.* 21, 140:1–140:67.
- Raheja, Vipul, Dhruv Kumar, Ryan Koo, and Dongyeop Kang [2023]. “CoEdIT: Text Editing by Task-Specific Instruction Tuning”. In: *ArXiv* abs/2305.09857.
- Rajpurkar, Pranav, Jian Zhang, Konstantin Lopyrev, and Percy Liang [2016]. “SQuAD: 100, 000+ Questions for Machine Comprehension of Text”. In: *EMNLP*.
- Ram, Ori, Yoav Levine, Itay Dalmedigos, Dor Muhlgay, Amnon Shashua, Kevin Leyton-Brown, and Yoav Shoham [2023]. “In-Context Retrieval-Augmented Language Models”. In: *ArXiv* abs/2302.00083.
- Reimers, Nils and Iryna Gurevych [Nov. 2019]. “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks”. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Hong Kong, China: Association for Computational Linguistics, pp. 3982–3992.
- Research, Cursor, : Aaron Chan, Ahmed Shalaby, Alexander Wettig, Aman Sanger, Andrew Zhai, Anurag Ajay, Ashvin Nair, Charlie Snell, Chen Lu, Chen Shen, Emily Jia, Federico Cassano, Hanpeng Liu, Haoyu Chen, Henry Wildermuth, Jacob Jackson, Janet Li, Jediah Katz, Jiajun Yao, Joey Hejna, Josh Warner, Julius Vering, Kevin Frans, Lee Danilek, Less Wright, Lujing Cen, Luke Melas-Kyriazi, Michael Truell, Michiel de Jong, Naman Jain, Nate Schmidt, Nathan Wang, Niklas Muennighoff, Oleg Rybkin, Paul Loh, Phillip Kravtsov, Rishabh Yadav, Sahil Shah, Sam Kottler, Alexander M Rush, Shengtong Zhang, Shomil Jain, Sriram Sankar, Stefan Heule,

- Stuart H. Sul, Sualeh Asif, Victor Rong, Wanqi Zhu, William Lin, Yuchen Wu, Yuri Volkov, Yury Zemlyanskiy, Zack Holbrook, and Zhiyuan Zhang [2026]. *Composer 2 Technical Report*.
- Ribar, Luka, Ivan Chelombiev, Luke Hudlass-Galley, Charlie Blake, Carlo Luschi, and Douglas Orr [21–27 Jul 2024]. “SparQ Attention: Bandwidth-Efficient LLM Inference”. In: *Proceedings of the 41st International Conference on Machine Learning*. Ed. by Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp. Vol. 235. Proceedings of Machine Learning Research. PMLR, pp. 42558–42583.
- Robertson, Stephen E. and Hugo Zaragoza [2009]. “The Probabilistic Relevance Framework: BM25 and Beyond”. In: *Found. Trends Inf. Retr.* 3, pp. 333–389.
- Roy, Aurko, Mohammad Saffar, Ashish Vaswani, and David Grangier [2021]. “Efficient Content-Based Sparse Attention with Routing Transformers”. In: *Transactions of the Association for Computational Linguistics* 9, pp. 53–68.
- Rubin, Ohad, Jonathan Herzig, and Jonathan Berant [July 2022]. “Learning To Retrieve Prompts for In-Context Learning”. In: *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Seattle, United States: Association for Computational Linguistics, pp. 2655–2671.
- Sachan, Devendra Singh, Mike Lewis, Mandar Joshi, Armen Aghajanyan, Wen-tau Yih, Joelle Pineau, and Luke Zettlemoyer [2022]. “Improving Passage Retrieval with Zero-Shot Question Generation”. In: *arXiv preprint arXiv:2204.07496*.
- Sanh, Victor, Albert Webson, Colin Raffel, Stephen Bach, Lintang Sutawika, Zaid Alyafeai, Antoine Chaffin, Arnaud Stiegler, Teven Le Scao, Arun Raja, et al. [2022]. “Multitask Prompted Training Enables Zero-Shot Task Generalization”. In: *The Tenth International Conference on Learning Representations*.
- Sellam, Thibault, Dipanjan Das, and Ankur P Parikh [2020]. “BLEURT: Learning Robust Metrics for Text Generation”. In: *Proceedings of ACL*.

- Shaib, Chantal, Millicent Li, Sebastian Joseph, Iain Marshall, Junyi Jessy Li, and Byron Wallace [July 2023]. “Summarizing, Simplifying, and Synthesizing Medical Evidence using GPT-3 (with Varying Success)”. In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*. Toronto, Canada: Association for Computational Linguistics, pp. 1387–1407.
- Shao, Yijia, Humishka Zope, Yucheng Jiang, Jiaxin Pei, David Nguyen, Erik Brynjolfsson, and Diyi Yang [2026]. *Future of Work with AI Agents: Auditing Automation and Augmentation Potential across the U.S. Workforce*.
- Shen, Judy Hanwen and Alex Tamkin [2026]. *How AI Impacts Skill Formation*.
- Shen, Zejiang, Tal August, Pao Siangliulue, Kyle Lo, Jonathan Bragg, Jeff Hammerbacher, Doug Downey, and Joseph Chee Chang [2023]. “Beyond Summarization: Designing AI Support for Real-World Expository Writing Tasks”. In: *ArXiv abs/2304.02623*.
- Shen, Zejiang, Kyle Lo, Lauren Yu, Nathan Dahlberg, Margo Schlanger, and Doug Downey [2022]. “Multi-LexSum: Real-world Summaries of Civil Rights Lawsuits at Multiple Granularities”. In: *Advances in Neural Information Processing Systems*. Ed. by S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh. Vol. 35. Curran Associates, Inc., pp. 13158–13173.
- Shi, Freda, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed Huai-hsin Chi, Nathanael Scharli, and Denny Zhou [2023a]. “Large Language Models Can Be Easily Distracted by Irrelevant Context”. In: *International Conference on Machine Learning*.
- Shi, Weijia, Sewon Min, Michihiro Yasunaga, Minjoon Seo, Rich James, Mike Lewis, Luke Zettlemoyer, and Wen-tau Yih [2023b]. “REPLUG: Retrieval-Augmented Black-Box Language Models”. In: *ArXiv abs/2301.12652*.
- Shu, Lei, Liangchen Luo, Jayakumar Hoskerc, Yun Zhu, Canoe Liu, Simon Tong, Jindong Chen, and Lei Meng [2023]. “RewriteLM: An Instruction-Tuned Large Language Model for Text Rewriting”. In: *ArXiv abs/2305.15685*.

- Si, Chenglei, Zhe Gan, Zhengyuan Yang, Shuohang Wang, Jianfeng Wang, Jordan L. Boyd-Graber, and Lijuan Wang [2022]. “Prompting GPT-3 To Be Reliable”. In: *ArXiv* abs/2210.09150.
- Snell, Charles Burton, Dan Klein, and Ruiqi Zhong [2022]. “Learning by Distilling Context”. In: *ArXiv* abs/2209.15189.
- Stelmakh, Ivan, Yi Luan, Bhuwan Dhingra, and Ming-Wei Chang [2022]. “ASQA: Factoid Questions Meet Long-Form Answers”. In: *arXiv preprint arXiv:2204.06092*.
- Su, Dan, Yan Xu, Tiezheng Yu, Farhad Bin Siddique, Elham Barber, and Pascale Fung [2022]. “Read before Generate! Faithful Long Form Question Answering with Machine Reading”. In: *Findings of the Association for Computational Linguistics: ACL 2022*.
- Su, Hongjin, Shizhe Diao, Ximing Lu, Mingjie Liu, Jiacheng Xu, Xin Dong, Yonggan Fu, Peter Belcak, Hanrong Ye, Hongxu Yin, Yi Dong, Evelina Bakhturina, Tao Yu, Yejin Choi, Jan Kautz, and Pavlo Molchanov [2025]. *ToolOrchestra: Elevating Intelligence via Efficient Model and Tool Orchestration*.
- Su, Jianlin, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu [2023]. *RoFormer: Enhanced Transformer with Rotary Position Embedding*.
- Sun, Jiao, Yufei Tian, Wangchunshu Zhou, Nan Xu, Qian Hu, Rahul Gupta, John Frederick Wieting, Nanyun Peng, and Xuezhe Ma [2023]. “Evaluating Large Language Models on Controlled Generation Tasks”. In: *Conference on Empirical Methods in Natural Language Processing*.
- Sun, Simeng, Ori Shapira, Ido Dagan, and Ani Nenkova [2019]. “How to Compare Summarizers without Target Length? Pitfalls, Solutions and Re-Examination of the Neural Summarization Literature”. In: *Proceedings of the Workshop on Methods for Optimizing and Evaluating Neural Language Generation*.
- Tang, Jiaming, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han [2024]. *Quest: Query-Aware Sparsity for Efficient Long-Context LLM Inference*.

- Taori, Rohan, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto [2023]. *Stanford Alpaca: An Instruction-following LLaMA model*. https://github.com/tatsu-lab/stanford_alpaca.
- Together [2023]. *RedPajama: an Open Dataset for Training Large Language Models*.
- Touvron, Hugo, Louis Martin, Kevin R. Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Daniel M. Bikel, Lukas Blecher, Cristian Cantón Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony S. Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel M. Kloumann, A. V. Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, R. Subramanian, Xia Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zhengxu Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom [2023]. “Llama 2: Open Foundation and Fine-Tuned Chat Models”. In: *ArXiv abs/2307.09288*.
- Wan, Zhongwei, Xinjian Wu, Yu Zhang, Yi Xin, Chaofan Tao, Zhihong Zhu, Xin Wang, Siqi Luo, Jing Xiong, Longyue Wang, and Mi Zhang [2025]. “ D_2O : Dynamic Discriminative Operations for Efficient Long-Context Inference of Large Language Models”. In: *The Thirteenth International Conference on Learning Representations*.
- Wang, Ben and Aran Komatsuzaki [May 2021]. *GPT-J-6B: A 6 Billion Parameter Autoregressive Language Model*. <https://github.com/kingoflolz/mesh-transformer-jax>.
- Wang, Hongru, Cheng Qian, Wanjuan Zhong, Xiushi Chen, Jiahao Qiu, Shijue Huang, Bowen Jin, Mengdi Wang, Kam-Fai Wong, and Heng Ji [2025a]. *Acting Less is Reasoning More! Teaching Model to Act Efficiently*.

- Wang, Shufan, Fangyuan Xu, Laure Thompson, Eunsol Choi, and Mohit Iyyer [2022]. “Modeling Exemplification in Long-form Question Answering via Retrieval”. In: *North American Chapter of the Association for Computational Linguistics*.
- Wang, Xingyao, Zihan Wang, Jiateng Liu, Yangyi Chen, Lifan Yuan, Hao Peng, and Heng Ji [2023a]. “MINT: Evaluating LLMs in Multi-turn Interaction with Tools and Language Feedback”. In: *ArXiv abs/2309.10691*.
- Wang, Yizhong, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A Smith, Daniel Khashabi, and Hannaneh Hajishirzi [2023b]. “Self-Instruct: Aligning Language Models with Self-Generated Instructions”. In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 13484–13508.
- Wang, Zheng, Boxiao Jin, Yuming Chang, Zhongzhi Yu, and Minjia Zhang [2025b]. *Model Tells You Where to Merge: Adaptive KV Cache Merging for LLMs on Long-Context Tasks*.
- Wang, Zhiruo, Jun Araki, Zhengbao Jiang, Md. Rizwan Parvez, and Graham Neubig [2023c]. “Learning to Filter Context for Retrieval-Augmented Generation”. In: *ArXiv abs/2311.08377*.
- West, Peter, Chandra Bhagavatula, Jack Hessel, Jena Hwang, Liwei Jiang, Ronan Le Bras, Ximing Lu, Sean Welleck, and Yejin Choi [July 2022]. “Symbolic Knowledge Distillation: from General Language Models to Commonsense Models”. In: *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Seattle, United States: Association for Computational Linguistics, pp. 4602–4625.
- Wingate, David, Mohammad Shoeybi, and Taylor Sorensen [Dec. 2022]. “Prompt Compression and Contrastive Conditioning for Controllability and Toxicity Reduction in Language Models”. In: *Findings of the Association for Computational Linguistics: EMNLP 2022*. Abu Dhabi, United Arab Emirates: Association for Computational Linguistics, pp. 5621–5634.
- Wolf, Thomas, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, and Jamie Brew [2019]. “Hugging-Face’s Transformers: State-of-the-art Natural Language Processing”. In: *ArXiv abs/1910.03771*.

- Wu, Xixi, Kuan Li, Yida Zhao, Liwen Zhang, Litu Ou, Huifeng Yin, Zhongwang Zhang, Xinmiao Yu, Dingchu Zhang, Yong Jiang, Pengjun Xie, Fei Huang, Minhao Cheng, Shuai Wang, Hong Cheng, and Jingren Zhou [2026]. *ReSum: Unlocking Long-Horizon Search Intelligence via Context Summarization*.
- Xiao, Chaojun, Pengl Zhang, Xu Han, Guangxuan Xiao, Yankai Lin, Zhengyan Zhang, Zhiyuan Liu, Song Han, and Maosong Sun [2024a]. “InfLLM: Unveiling the Intrinsic Capacity of LLMs for Understanding Extremely Long Sequences with Training-Free Memory”. In: *arXiv*.
- Xiao, Guangxuan, Ji Lin, Mickael Seznec, Julien Demouth, and Song Han [2022]. “SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models”. In: *ArXiv abs/2211.10438*.
- Xiao, Guangxuan, Jiaming Tang, Jingwei Zuo, Junxian Guo, Shang Yang, Haotian Tang, Yao Fu, and Song Han [2024b]. “DuoAttention: Efficient Long-Context LLM Inference with Retrieval and Streaming Heads”. In: *arXiv*.
- Xiao, Guangxuan, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis [2023]. “Efficient Streaming Language Models with Attention Sinks”. In: *ArXiv abs/2309.17453*.
- Xie, Tianbao, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu [2024]. “OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments”. In: *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.
- Xu, Fangyuan, Sihao Chen, Zinan Lin, Taiwei Shi, Sydney Graham, Pei Zhou, Mengting Wan, Alex Stein, Virginia Estellers, Charles Chen, Morris Sharp, Richard Speyer, Tadas Baltrusaitis, Jennifer Neville, Eunsol Choi, and Longqi Yang [2026a]. *DP-RFT: Learning to Generate Synthetic Text via Differentially Private Reinforcement Fine-Tuning*.
- Xu, Fangyuan, Tanya Goyal, and Eunsol Choi [2025]. “RefreshKV: Updating Small KV Cache During Long-form Generation”. In: *Proceedings of the 63rd Annual Meeting of the Associa-*

tion for Computational Linguistics (Volume 1: Long Papers). Vienna, Austria: Association for Computational Linguistics.

- Xu, Fangyuan, Rujun Han, Yanfei Chen, Zifeng Wang, I-Hung Hsu, Jun Yan, Vishy Tirumalashetty, Eunsol Choi, Tomas Pfister, and Chen-Yu Lee [2026b]. “SAGE: Steerable Agentic Data Generation for Deep Search with Execution Feedback”. In: *Findings of the Association for Computational Linguistics: EACL 2026*. Rabat, Morocco: Association for Computational Linguistics.
- Xu, Fangyuan, Kyle Lo, Luca Soldaini, Bailey Kuehl, Eunsol Choi, and David Wadden [2024a]. “KIWI: A Dataset of Knowledge-Intensive Writing Instructions for Answering Research Questions”. In: *Findings of the Association for Computational Linguistics: ACL 2024*. Bangkok, Thailand: Association for Computational Linguistics.
- Xu, Fangyuan, Weijia Shi, and Eunsol Choi [2024b]. “RECOMP: Improving Retrieval-Augmented LMs with Context Compression and Selective Augmentation”. In: *The Twelfth International Conference on Learning Representations*.
- Xu, Fangyuan, Yixiao Song, Mohit Iyyer, and Eunsol Choi [2023]. “A Critical Evaluation of Evaluations for Long-form Question Answering”. In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Toronto, Canada: Association for Computational Linguistics.
- Xu, Yumo and Mirella Lapata [2020]. “Coarse-to-Fine Query Focused Multi-Document Summarization”. In: *Conference on Empirical Methods in Natural Language Processing*.
- Yang, An, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, Guanting Dong, Haoran Wei, Huan Lin, Jialong Tang, Jialin Wang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Ma, Jin Xu, Jingren Zhou, Jinze Bai, Jinzheng He, Junyang Lin, Kai Dang, Keming Lu, Ke-Yang Chen, Kexin Yang, Mei Li, Min Xue, Na Ni, Pei Zhang, Peng Wang, Ru Peng, Rui Men, Ruize Gao, Runji Lin, Shijie Wang, Shuai Bai, Sinan Tan, Tianhang Zhu, Tianhao Li, Tianyu Liu, Wenbin Ge, Xiaodong Deng, Xiaohuan Zhou, Xingzhang Ren, Xinyu Zhang, Xipin Wei, Xuancheng Ren, Yang Fan, Yang

- Yao, Yichang Zhang, Yunyang Wan, Yunfei Chu, Zeyu Cui, Zhenru Zhang, and Zhi-Wei Fan [2024a]. “Qwen2 Technical Report”. In: *ArXiv* abs/2407.10671.
- Yang, Dongjie, Xiaodong Han, Yan Gao, Yao Hu, Shilin Zhang, and Hai Zhao [Aug. 2024b]. “PyramidInfer: Pyramid KV Cache Compression for High-throughput LLM Inference”. In: *Findings of the Association for Computational Linguistics ACL 2024*. Ed. by Lun-Wei Ku, Andre Martins, and Vivek Srikumar. Bangkok, Thailand and virtual meeting: Association for Computational Linguistics, pp. 3258–3270.
- Yang, Songlin, Jan Kautz, and Ali Hatamizadeh [2025]. “Gated Delta Networks: Improving Mamba2 with Delta Rule”. In: *The Thirteenth International Conference on Learning Representations*.
- Yang, Zhilin, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning [2018]. “HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering”. In: *Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- Yao, Shunyu, Howard Chen, Austin W. Hanjie, Runzhe Yang, and Karthik R Narasimhan [2024]. “COLLIE: Systematic Construction of Constrained Text Generation Tasks”. In: *The Twelfth International Conference on Learning Representations*.
- Ye, Xi, Fangcong Yin, Yinghui He, Joie Zhang, Yen Howard, Tianyu Gao, Greg Durrett, and Danqi Chen [2025]. “LongProc: Benchmarking Long-Context Language Models on Long Procedural Generation”. In: *arXiv preprint*.
- Yoran, Ori, Tomer Wolfson, Ori Ram, and Jonathan Berant [2024]. “Making Retrieval-Augmented Language Models Robust to Irrelevant Context”. In: *The Twelfth International Conference on Learning Representations*.
- Yuan, Weizhe, Ilia Kulikov, Ping Yu, Kyunghyun Cho, Sainbayar Sukhbaatar, Jason E Weston, and Jing Xu [Nov. 2025]. “Following Length Constraints in Instructions”. In: *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. Ed. by Christos

- Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng. Suzhou, China: Association for Computational Linguistics, pp. 24232–24243. ISBN: 979-8-89176-332-6.
- Yuan, Weizhe, Graham Neubig, and Pengfei Liu [2021]. “BARTScore: Evaluating Generated Text as Text Generation”. In: *Advances in Neural Information Processing Systems*. Ed. by M. Ranzato, A. Beygelzimer, Y. Dauphin, P.S. Liang, and J. Wortman Vaughan. Vol. 34. Curran Associates, Inc., pp. 27263–27277.
- Zaheer, Manzil, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontañón, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed [2020]. “Big Bird: Transformers for Longer Sequences”. In: *ArXiv abs/2007.14062*.
- Zeng, Zhiyuan, Jiatong Yu, Tianyu Gao, Yu Meng, Tanya Goyal, and Danqi Chen [2023]. “Evaluating Large Language Models at Evaluating Instruction Following”. In: *ArXiv abs/2310.07641*.
- Zhang, Chen, L. F. D’Haro, Qiquan Zhang, Thomas Friedrichs, and Haizhou Li [2022a]. “FineD-Eval: Fine-grained Automatic Dialogue-Level Evaluation”. In: *ArXiv abs/2210.13832*.
- Zhang, Haopeng, Hayate Iso, Sairam Gurajada, and Nikita Bhutani [2023]. “XATU: A Fine-grained Instruction-based Benchmark for Explainable Text Updates”. In: *ArXiv abs/2309.11063*.
- Zhang, Maosen, Nan Jiang, Lei Li, and Yexiang Xue [Nov. 2020]. “Language Generation via Combinatorial Constraint Satisfaction: A Tree Search Enhanced Monte-Carlo Approach”. In: *Findings of the Association for Computational Linguistics: EMNLP 2020*. Online: Association for Computational Linguistics, pp. 1286–1298.
- Zhang, Michael JQ, Zhilin Wang, Jena D. Hwang, Yi Dong, Olivier Delalleau, Yejin Choi, Eunsol Choi, Xiang Ren, and Valentina Pyatkin [2025]. “Diverging Preferences: When do Annotators Disagree and do Models Know?” In: *Forty-second International Conference on Machine Learning*.
- Zhang, Shiyue, David Wan, and Mohit Bansal [2022b]. “Extractive is not Faithful: An Investigation of Broad Unfaithfulness Problems in Extractive Summarization”. In: *ArXiv abs/2209.03549*.

- Zhang, Tianyi, Varsha Kishore, Felix Wu, Kilian Q Weinberger, and Yoav Artzi [2019]. “BERTScore: Evaluating Text Generation with BERT”. In: *International Conference on Learning Representations*.
- Zhang, Xinrong, Yingfa Chen, Shengding Hu, Zihang Xu, Junhao Chen, Moo Hao, Xu Han, Zhen Thai, Shuo Wang, Zhiyuan Liu, and Maosong Sun [Aug. 2024a]. “ ∞ Bench: Extending Long Context Evaluation Beyond 100K Tokens”. In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Ed. by Lun-Wei Ku, Andre Martins, and Vivek Srikumar. Bangkok, Thailand: Association for Computational Linguistics, pp. 15262–15277.
- Zhang, Zhenyu, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, et al. [2024b]. “H2o: Heavy-hitter oracle for efficient generative inference of large language models”. In: *Advances in Neural Information Processing Systems* 36.
- Zhao, Wenting, Xiang Ren, Jack Hessel, Claire Cardie, Yejin Choi, and Yuntian Deng [2024]. “WildChat: 1M ChatGPT Interaction Logs in the Wild”. In: *The Twelfth International Conference on Learning Representations*.
- Zhao, Wenting, Xiang Ren, Jack Hessel, Yejin Choi, Claire Cardie, and Yuntian Deng [2023a]. *(InThe)WildChat: 650K ChatGPT Interaction Logs in the Wild*.
- Zhao, Yanli, Andrew Gu, Rohan Varma, Liangchen Luo, Chien-chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, Alban Desmaison, Can Balioglu, Bernard Nguyen, Geeta Chauhan, Yuchen Hao, and Shen Li [2023b]. “PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel”. In: *Proc. VLDB Endow.* 16, pp. 3848–3860.
- Zheng, Lianmin, Wei-Lin Chiang, Ying Sheng, Tianle Li, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zhuohan Li, Zi Lin, Eric P. Xing, Joseph E. Gonzalez, Ion Stoica, and Haotong Zhang [2023a]. “LMSYS-Chat-1M: A Large-Scale Real-World LLM Conversation Dataset”. In: *ArXiv abs/2309.11998*.

- Zheng, Lianmin, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Haotong Zhang, Joseph Gonzalez, and Ion Stoica [2023b]. “Judging LLM-as-a-judge with MT-Bench and Chatbot Arena”. In: *ArXiv abs/2306.05685*.
- Zhong, Ming, Yang Liu, Da Yin, Yuning Mao, Yizhu Jiao, Peng Liu, Chenguang Zhu, Heng Ji, and Jiawei Han [2022]. “Towards a Unified Multi-Dimensional Evaluator for Text Generation”. In: *ArXiv abs/2210.07197*.
- Zhong, Ming, Da Yin, Tao Yu, Ahmad Zaidi, Mutethia Mutuma, Rahul Jha, Ahmed Hassan Awadallah, Asli Celikyilmaz, Yang Liu, Xipeng Qiu, and Dragomir Radev [June 2021]. “QMSum: A New Benchmark for Query-based Multi-domain Meeting Summarization”. In: *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Ed. by Kristina Toutanova, Anna Rumshisky, Luke Zettlemoyer, Dilek Hakkani-Tur, Iz Beltagy, Steven Bethard, Ryan Cotterell, Tanmoy Chakraborty, and Yichao Zhou. Online: Association for Computational Linguistics, pp. 5905–5921.
- Zhou, Shuyan, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig [2024]. “WebArena: A Realistic Web Environment for Building Autonomous Agents”. In: *The Twelfth International Conference on Learning Representations*.
- Zhou, Wangchunshu, Yuchen Jiang, Ethan Gotlieb Wilcox, Ryan Cotterell, and Mrinmaya Sachan [2023]. “Controlled Text Generation with Natural Language Instructions”. In: *ArXiv abs/2304.14293*.
- Zhu, Yaoming, Sidi Lu, Lei Zheng, Jiaxian Guo, Weinan Zhang, Jun Wang, and Yong Yu [2018]. “Texygen: A benchmarking platform for text generation models”. In: *The 41st International ACM SIGIR Conference on Research & Development in Information Retrieval*, pp. 1097–1100.